

SC4063: Network Security

Week 10 : Intro to Network Security

Chua Zong Fu





Architectural foundations and modern threat landscape



Modern Application Architecture

- **Monolithic Architecture:** Traditional "three-tier" model (Presentation, Application, Persistence) where all components are tightly coupled and deployed as a single unit.
- **Microservices:** Decomposing applications into small, loosely coupled services organized around business capabilities. Each service runs its own process and communicates via lightweight APIs (often REST or gRPC).
- **Serverless (FaaS):** Event-driven execution (e.g. AWS Lambda, Azure Functions) where the cloud provider manages the underlying infrastructure. Code runs only when triggered by events (HTTP requests, database changes).
- **SaaS (Software as a Service):** Fully managed applications (e.g. Salesforce, Microsoft 365) where security responsibility lies primarily with the provider, leaving the customer to manage access and data.
- **Hybrid Architectures:** A mix of on-premises legacy infrastructure and private/public cloud services, often requiring complex orchestration to maintain integration during transitions.

Introduction to Application Architecture

Application architecture serves as the blueprint for how computing and storage resources are organised, directly impacting how traffic flows and how security must be implemented. As organisations migrate to the cloud, they must choose models that balance agility with the ability to defend a varied attack surface.

Monolithic Architecture

The **monolithic model** is the traditional "three-tier" approach, consisting of a **presentation tier** (web server), an **application tier** (business logic), and a **persistence tier** (database). In this architecture, all components are tightly coupled and deployed as a single unit. While this centralises security controls, it creates a "all eggs in one basket" risk; if an attacker compromises the web server, they often gain a direct path to the database credentials stored within the application code. Furthermore, monoliths suffer from slow release cycles, as a security hotfix for one component requires rebuilding and redeploying the entire package.

Microservices

Microservices decompose applications into a suite of small, independent services organised around specific business capabilities. Each service runs its own process and communicates via lightweight **APIs**, typically using **REST or**

gRPC. This modularity allows for "polyglot" environments where different teams use the best technology stack for their specific task. From a security perspective, microservices facilitate **micro-segmentation**, allowing architects to isolate high-risk data into restricted subnets. However, this model significantly increases complexity; while a monolith may have three primary trust boundaries, a microservice architecture can have over forty connection points, each requiring its own authentication and authorization.

Serverless (FaaS)

Serverless computing, or **Function as a Service (FaaS)**, represents a shift where the cloud provider manages the underlying infrastructure, including the operating system, scaling, and runtime environment. Code executes only when triggered by specific events, such as **HTTP requests** or **database changes**. This model offers significant security benefits: the attack surface is smaller because there are no long-running servers to scan or patch, and malware persistence is difficult because each request spins up a "clean" temporary container. However, organisations must be wary of "**Denial of Wallet**" attacks, where an adversary triggers excessive function executions to inflate cloud costs.

SaaS (Software as a Service)

In the **SaaS model**, users access fully managed applications over the internet on a subscription basis. The **Shared Responsibility Model** dictates that the provider secures the entire stack, including the application code and infrastructure. The customer's primary responsibility is managing **user access** and the **data** stored within the service. Because customers lack visibility into the backend, they often employ **Cloud Access Security Brokers (CASBs)** to monitor user activity and ensure compliance.

Hybrid Architectures

Hybrid architectures combine on-premises legacy infrastructure with private or public cloud services. This is a common strategy for organisations undergoing long-term migrations or those required by regulation to keep sensitive data locally. These environments require complex **orchestration** and robust connectivity, such as **Site-to-Site VPNs** or dedicated links like **ExpressRoute**, to ensure that data transitions securely between the data centre and the cloud. The primary challenge is maintaining a consistent security policy across disparate environments, as attackers may exploit gaps at the point where traffic transitions between the two.



Distinguishing Attack Surfaces

- **Not Interchangeable:** While they share underlying technologies, the attack vectors and defensive tools differ significantly. Traditional Application Security (AppSec) tools like SAST and DAST often lack the context to effectively test APIs.
- **The API Gap:** Traditional network firewalls and Web Application Firewalls (WAFs) struggle to protect APIs because they lack awareness of the specific API logic and expected traffic patterns.
- **Headless Nature:** Unlike web apps, APIs are "headless" (no UI) and intended for automated consumption, making them highly susceptible to automated script attacks and credential abuse.
- **Overlapping Risks:** Attacks often chain vulnerabilities; for example, an injection flaw (AppSec) might be exploited to compromise a backend database (Web Security), or a broken object level authorization (BOLA) in an API can lead to mass data exfiltration.

Introduction: The Expanding Attack Surface

An **attack surface** represents the total sum of reachable and exploitable vulnerabilities within a system, encompassing network protocols, software code, and human elements. While traditional web applications and APIs share underlying technologies like HTTP, they are **not interchangeable** from a security perspective. APIs have fundamentally expanded the attack surface of modern applications, often acting as the "Death Star's thermal exhaust port", a direct, poorly defended path to an organisation's most sensitive data.

The Limitations of Traditional AppSec Tools

Traditional Application Security (AppSec) relies on tools like **SAST** (Static Analysis) and **DAST** (Dynamic Analysis).

SAST identifies flaws in source code during development but frequently suffers from a **lack of contextual awareness**. It may catch a generic injection flaw but will miss API-specific logic issues like **Broken Object Level Authorization (BOLA)** because the engine does not understand the underlying business logic.

DAST tests running applications but similarly lacks the context to fully interpret API-specific requests and responses. Generic vulnerability scans often yield **false negatives** when applied to APIs, providing a false sense of security while leaving catastrophic weaknesses undetected.

The API Gap in Perimeter Defense

Traditional network firewalls operate at the **Network and Transport layers (Layers 3 and 4)**, inspecting IP addresses and ports. These are increasingly insufficient, as they cannot prevent attacks employing application-specific functions.

Web Application Firewalls (WAFs) operate at the **Application layer (Layer 7)** to block known malicious signatures. However, WAFs struggle with APIs because they typically use a **negative security model** (blocklists), which only stops "known bad" patterns. They lack awareness of the specific **API logic** and the intended data contract (the OpenAPI Specification), making them ineffective against sophisticated bots or logic-based exploits that appear as legitimate traffic.

The Vulnerabilities of "Headless" Architecture

Unlike traditional web apps, APIs are **"headless,"** meaning they lack a user interface and are intended for **automated machine-to-machine communication**. This nature makes them highly susceptible to:

Automated Script Attacks: Adversaries use scripts and botnets to probe APIs at scale, which is significantly easier and faster than attacking a GUI.

Credential Abuse: APIs are prime targets for **credential stuffing**, where attackers use large-scale automation to test leaked usernames and passwords until they find a match. Because APIs are designed for ease of use, they are often **over-permissioned**, granting attackers broad access once a single identity is compromised.

Overlapping Risks and Chained Vulnerabilities

Modern breaches rarely stem from a single flaw; they result from **chaining multiple vulnerabilities**. For example, a **Server-Side Request Forgery (SSRF)** flaw (an infrastructure/web risk) was used in the Capital One breach to query a cloud metadata service and steal credentials. These stolen keys then allowed the attacker to exploit a **BOLA** vulnerability in the API to exfiltrate the data of millions of users. This illustrates that a failure in one discipline (Network or Web Security) often serves as a foothold to exploit deeper logic flaws in the API.



HTTP as the Foundational Protocol

- **Stateless Protocol:** HTTP/1.1 is stateless by design; each request is independent and the server retains no knowledge of previous interactions.
- **The "State" Hack:** To create cohesive sessions (like a logged-in user experience), applications must artificially maintain state using cookies or tokens.
- **Security Consequence:** This reliance on client-side tokens makes session hijacking and Cross-Site Request Forgery (CSRF) possible. If an attacker steals the session cookie, they effectively become the user.
- **Cleartext Legacy:** HTTP/1.1 is text-based and readable by humans. Without TLS (HTTPS), all headers, credentials, and data are transmitted in cleartext, vulnerable to interception.

These speaker notes provide an in-depth analysis of the slide "Distinguishing Attack Surfaces", grounded in the provided sources.

Introduction: The Expanding Attack Surface

An **attack surface** represents the total sum of reachable and exploitable vulnerabilities within a system, encompassing network protocols, software code, and human elements. While traditional web applications and APIs share underlying technologies like HTTP, they are **not interchangeable** from a security perspective. APIs have fundamentally expanded the attack surface of modern applications, often acting as the "Death Star's thermal exhaust port", a direct, poorly defended path to an organisation's most sensitive data.

The Limitations of Traditional AppSec Tools

Traditional Application Security (AppSec) relies on tools like **SAST** (Static Analysis) and **DAST** (Dynamic Analysis).

SAST identifies flaws in source code during development but frequently suffers from a **lack of contextual awareness**. It may catch a generic injection flaw but will miss API-specific logic issues like **Broken Object Level Authorization (BOLA)** because the engine does not understand the underlying business logic. **DAST** tests running applications but similarly lacks the context to fully interpret API-specific requests and responses. Generic vulnerability scans often yield

false negatives when applied to APIs, providing a false sense of security while leaving catastrophic weaknesses undetected.

The API Gap in Perimeter Defense

Traditional network firewalls operate at the **Network and Transport layers (Layers 3 and 4)**, inspecting IP addresses and ports. These are increasingly insufficient, as they cannot prevent attacks employing application-specific functions.

Web Application Firewalls (WAFs) operate at the **Application layer (Layer 7)** to block known malicious signatures. However, WAFs struggle with APIs because they typically use a **negative security model** (blocklists), which only stops "known bad" patterns. They lack awareness of the specific **API logic** and the intended data contract (the OpenAPI Specification), making them ineffective against sophisticated bots or logic-based exploits that appear as legitimate traffic.

The Vulnerabilities of "Headless" Architecture

Unlike traditional web apps, APIs are **"headless,"** meaning they lack a user interface and are intended for **automated machine-to-machine communication**. This nature makes them highly susceptible to:

Automated Script Attacks: Adversaries use scripts and botnets to probe APIs at scale, which is significantly easier and faster than attacking a GUI.

Credential Abuse: APIs are prime targets for **credential stuffing**, where attackers use large-scale automation to test leaked usernames and passwords until they find a match. Because APIs are designed for ease of use, they are often **over-permissioned**, granting attackers broad access once a single identity is compromised.

Overlapping Risks and Chained Vulnerabilities

Modern breaches rarely stem from a single flaw; they result from **chaining multiple vulnerabilities**. For example, a **Server-Side Request Forgery (SSRF)** flaw (an infrastructure/web risk) was used in the Capital One breach to query a cloud metadata service and steal credentials. These stolen keys then allowed the attacker to exploit a **BOLA** vulnerability in the API to exfiltrate the data of millions of users. This illustrates that a failure in one discipline (Network or Web Security) often serves as a foothold to exploit deeper logic flaws in the API.



Protocol Evolution (HTTP/2 & HTTP/3)

- **HTTP/2 Efficiency & Obfuscation:**

- Introduced binary framing and header compression (HPACK), making traffic non-human-readable without specialized tools.
- Multiplexing: Allows multiple concurrent requests over a single TCP connection, complicating flow analysis for legacy security tools.

- **HTTP/3 (QUIC):**

- Moves transport from TCP to UDP to reduce latency and head-of-line blocking.
- Encryption by Default: TLS 1.3 is mandatory and integrated into the transport layer, encrypting not just the payload but also packet numbers and handshake metadata.

- **Visibility Challenges:** Newer protocols like DNS over HTTPS (DoH) run inside encrypted HTTP/2 streams, concealing the very existence of DNS traffic from traditional network monitoring.

The evolution of web protocols represents a paradigm shift in both performance and security visibility. **HTTP/2, based on the Google SPDY protocol**, was designed to fix the inefficiencies of HTTP/1.1. Its primary feature is **binary framing**, which means that unlike the human-readable plaintext of previous versions, traffic on the wire is now binary and **non-human-readable without specialised analysis tools**. This obfuscation is deepened by **HPACK header compression**, which uses Huffman coding to compress binary header tags, significantly reducing overhead but complicating manual inspection. Furthermore, **multiplexing** allows a client to use a single TCP connection to handle up to 100 concurrent request/response streams simultaneously, making traditional flow analysis for legacy security tools much more difficult. **HTTP/3 represents an even more radical shift by adopting the QUIC protocol**, which moves transport from TCP to **UDP**. This change was specifically intended to **reduce latency and eliminate head-of-line blocking**, a performance bottleneck where a single lost packet could stall an entire TCP connection. Critically, HTTP/3 mandates **encryption by default**, integrating **TLS 1.3** directly into the transport layer. While older versions of TLS left sequence and acknowledgment numbers visible, QUIC encrypts nearly all metadata, including **packet numbers and handshake metadata**, creating a significant blind spot for

network-based defenders.

These advancements lead to substantial **visibility challenges**, perhaps best exemplified by **DNS over HTTPS (DoH)**. Because DoH runs inside encrypted **HTTP/2 or HTTP/3 streams on port 443**, it conceals the very existence of DNS traffic from traditional network monitoring systems. Unlike DNS over TLS (DoT), which uses the easily identifiable port 853, DoH looks exactly like standard web traffic. This creates a "**crypto plus steganography**" effect that allows both legitimate browsers and malicious actors, such as the **Godlua malware**, to bypass internal DNS security controls and monitoring entirely.



Protocol Artifacts for Attackers & Defenders

- **Methods:** Attackers use obscure methods like OPTIONS for fingerprinting servers or DELETE/PUT to modify content, whereas normal users primarily use GET and POST.
- **Status Codes:** Bursts of 404 (Not Found) indicate scanning/reconnaissance. 500 (Server Error) series errors can indicate successful SQL injection probing.
- **Headers:**
 - **User-Agent:** Often spoofed by attackers to look like legitimate browsers or mobile devices (e.g. "IPHONE8.5").
 - **Server/X-Powered-By:** Leaks backend technology details (e.g. "ASP.NET", "Apache/2.4") useful for attackers targeting specific CVEs.
- **Cookies:** Critical for session state. Flags like HttpOnly (prevents XSS access) and Secure (HTTPS only) are vital defensive artifacts.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to Protocol Artifacts

The Hypertext Transfer Protocol (HTTP) is a stateless protocol, meaning each request and response pair is independent. Because of this, both attackers and defenders rely on specific **protocol artifacts** to understand the intent and state of a connection. For security professionals, these artifacts found in logs and packet captures are essential for distinguishing between a legitimate user and a malicious actor.

Request Methods as Indicators of Intent

Standard web traffic is primarily comprised of **GET and POST** methods.

Defenders should be alert for obscure methods like **OPTIONS**, which attackers use to "fingerprint" a server by retrieving a list of all supported HTTP methods. Verbs such as **PUT and DELETE** are rare in normal browsing; they allow a client to upload or remove resources and may indicate an attempt to deface a site or install a backdoor.

Status Codes and Reconnaissance

HTTP response status codes reveal the outcome of a request and are vital for **anomaly detection**. A burst of **404 Not Found** errors from a single IP address is a classic signature of a website crawl or directory brute-force attack, where an adversary is scanning for hidden files or administrative portals. Conversely, **500-**

series Server Errors can indicate that an attacker is successfully probing for vulnerabilities; for example, an unhandled SQL syntax error from the backend database often triggers a 500 response, signaling a potential SQL injection vulnerability.

Headers: Profiling and Information Leakage

Headers provide metadata that is often manipulated by adversaries. The **User-Agent** header identifies the client software, but it is easily spoofed; an attacker might use a non-standard string like "IPHONE8.5" while connecting from a laptop to evade simple filtering rules. On the server side, headers like **Server** and **X-Powered-By** can leak specific version details (e.g. "Apache/2.4.24" or "ASP.NET"), which helps attackers identify exactly which known exploits to use against the backend technology.

Cookies and Defensive Hardening

Because HTTP is stateless, **cookies** are used to maintain session state. They are high-value targets because stealing a session cookie allows an attacker to impersonate a user without needing their password. Defenders must use **HttpOnly** and **Secure** flags as vital artifacts of a secure architecture. The **HttpOnly** flag prevents client-side JavaScript from accessing the cookie, mitigating the risk of theft via Cross-Site Scripting (XSS), while the **Secure** flag ensures the cookie is only transmitted over encrypted HTTPS connections, preventing it from being sniffed in cleartext.



Traffic Inspection and Protocol Literacy

- **Interception Proxies:** Tools like Burp Suite and OWASP ZAP act as "intentional Man-in-the-Middle" points, allowing testers to pause, modify, and replay requests to test application logic.
- **Packet Capture:** Tools like Wireshark and tcpdump capture raw traffic. Analysts must understand stream reconstruction to make sense of fragmented packets.
- **Encryption Limits:**
 - **The "Middlebox" Problem:** With TLS 1.3 and Perfect Forward Secrecy (PFS), passive decryption (sniffing) is largely impossible even with the server's private key.
 - **Inspection Points:** Inspection must move to endpoints (EDR) or authorized termination points (WAFs, Load Balancers) where decryption occurs.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Traffic Inspection and Protocol Literacy

Effective security analysis requires a deep understanding of how protocols behave on the wire to distinguish malicious activity from benign noise.

Interception Proxies: Tools like **Burp Suite** and **OWASP ZAP** are essential for web and API security testing, acting as **intentional Man-in-the-Middle (MITM)** points. These proxies allow practitioners to **intercept, modify, and replay** individual HTTP/HTTPS requests and responses in real-time. This capability is critical for testing complex application logic, as it enables testers to bypass client-side restrictions and observe how the backend handles malformed or unexpected data.

Packet Capture: For a lower-level view, tools such as **Wireshark** and **tcpdump** capture raw network traffic. Because TCP/IP traffic is often fragmented or received out of order, analysts must understand **stream reconstruction**. Wireshark's "Follow Stream" feature is particularly valuable here, as it reassembles individual packets into a cohesive, human-readable transcript of the entire conversation.

Encryption Limits and The "Middlebox" Problem

The widespread adoption of modern encryption has created a significant "blind spot" for traditional network security monitoring.

The Middlebox Problem: Historical inspection relied on "middleboxes" (like NIDS) sniffing traffic and decrypting it using the server's private key. However, **TLS 1.3** and the mandatory use of **Perfect Forward Secrecy (PFS)** have rendered this passive decryption largely impossible.

Perfect Forward Secrecy (PFS): PFS ensures that even if a server's long-term private key is compromised, an attacker cannot decrypt past or future sessions because the session keys are **ephemeral** and generated randomly for every connection.

Evolved Inspection Points

Due to these encryption limits, security teams must strategically move their inspection points.

Endpoint Detection: Inspection is increasingly moving to the host via **Endpoint Detection and Response (EDR)** systems or **Endpoint Policy Enforcement Points (E-PEPs)**. Because the endpoint is a terminal node, it has access to the data in its unencrypted state before it is wrapped in TLS.

Authorized Termination Points: Alternatively, inspection occurs at authorized termination points such as **API Gateways, Load Balancers, or Web Application Firewalls (WAFs)**. These devices act as a "terminating proxy," establishing one encrypted connection with the client and a second with the server, allowing for full plaintext inspection in the middle.



Attacker Objectives & Architecture

- **Data Theft (Exfiltration):** Targeting APIs (BOLA vulnerabilities) to scrape mass volumes of PII or financial data.
- **Account Takeover (ATO):** Exploiting broken authentication mechanisms or credential stuffing against login endpoints.
- **Fraud:** Abusing business logic flaws (e.g. excessive coupon redemption, inventory hoarding) rather than technical exploits.
- **Lateral Movement:** Using Server-Side Request Forgery (SSRF) to pivot from a public-facing web server to internal metadata services (e.g. AWS IMDS) or internal databases.
- **Persistence:** Injecting webshells, creating "shadow" API keys, or manipulating scheduled tasks to maintain access after the initial breach.

Data Theft (Exfiltration)

Exfiltration is the act of stealing sensitive data from a system without the owner's awareness. In the context of APIs, the most common and devastating method is targeting **Broken Object Level Authorization (BOLA)**, also known as **Insecure Direct Object Reference (IDOR)**. Attackers manipulate object identifiers, such as user IDs or account numbers, in API requests to access resources belonging to other users. Because API interaction is **headless** and intended for machine-to-machine communication, these attacks are easily **automated with scripts**, allowing for the **mass scraping** of Personally Identifiable Information (PII) or financial records. Real-world examples include the **USPS breach**, where 60 million records were exposed, and **T-Mobile**, where 37 million customer records were stolen due to improper access controls.

Account Takeover (ATO)

Account Takeover (ATO) occurs when an attacker successfully hijacks a legitimate user's identity. This is primarily achieved by exploiting **Broken Authentication** mechanisms. Attackers frequently use **credential stuffing**, an automated process where large lists of leaked usernames and passwords from previous breaches are "stuffed" into login endpoints to identify reused credentials. Alternatively, they may use **brute-force attacks** or **password**

spraying (trying a few common passwords across many accounts) to bypass weak authentication checks. If an API does not enforce **rate limiting** or MFA, these automated attempts can proceed undetected until a match is found.

Fraud and Business Logic Flaws

Attackers often bypass technical exploits in favour of abusing **business logic flaws**, which are intended features used in unintended ways to gain financial advantages. Examples include **scalping**, where automated bots buy out limited-edition stocks (like concert tickets) to resell them at higher prices, or **promo fraud**, where users create fake accounts to hoard referral rewards. These flaws are particularly pernicious because they do not involve implementation bugs but rather **design loopholes** that automated scanners typically fail to detect.

Lateral Movement (SSRF/IMDS)

Lateral movement is the process of moving from an initial foothold to other systems within a network to get closer to the "crown jewels". A critical vector for this in cloud environments is **Server-Side Request Forgery (SSRF)**. An attacker tricks a public-facing web server into making internal requests to sensitive services, most notably the **Instance Metadata Service (IMDS)**. By querying the IMDS (typically at 169.254.169.254), attackers can **exfiltrate temporary cloud credentials** (IAM tokens) assigned to that server. This tactic was famously used in the **Capital One breach**, where the attacker leveraged a misconfigured WAF to steal credentials and access 700 S3 buckets containing customer data.

Persistence

Once access is established, adversaries deploy mechanisms to **maintain their foothold** across restarts or credential changes.

Webshells: Attackers upload or modify server-side script files (e.g. PHP or ASP.NET) to execute arbitrary OS commands on the web server.

Shadow API Keys: In cloud environments, attackers may create **new IAM user keys** or backdoor credentials as a secondary access mechanism, ensuring they can re-enter even if the primary account password is reset.

Scheduled Tasks: Adversaries use built-in tools like **Windows Task Scheduler (schtasks)** or **Linux cron** to re-execute malicious payloads at boot or specific intervals, effectively surviving reboots.



Trust Boundaries in Modern Systems

- **Browser to Backend:** The classic perimeter. Zero Trust requires treating the browser as untrusted, enforcing strict Content Security Policy (CSP) and input validation.
- **Frontend to API:** Modern SPAs and mobile apps consume APIs directly. Authentication tokens (JWTs) must be handled securely (e.g. not stored in local storage).
- **Service-to-Service (East-West):** In microservices, internal traffic is often implicitly trusted. Zero Trust mandates mutual TLS (mTLS) and service-level authorization to prevent lateral movement.
- **Third-Party Integrations:** "Unsafe Consumption of APIs" is a growing risk. Trusting data from third-party partners without validation can lead to injection attacks or data corruption.

Introduction to Trust Boundaries

In modern computing, **trust boundaries** represent the points where data flows between different zones of varying trust levels. While traditional security focused on a "crunchy shell" network perimeter, the transition to cloud and microservices has created a **perimeter-less environment** where trust must be established per request, regardless of location. Identifying these boundaries is critical because they represent the primary **attack surface** where adversaries exploit flaws in access-control logic.

Browser to Backend: The Classic Perimeter

The browser was historically viewed as the start of a trusted session, but **Zero Trust** mandates that we treat the browser environment as inherently **hostile and untrusted**. To secure this boundary, organisations must enforce a strict **Content Security Policy (CSP)**, which specifies exactly which resources and script endpoints a browser is permitted to load, thereby mitigating **Cross-Site Scripting (XSS)**. Furthermore, every piece of data received must undergo rigorous **input validation and sanitisation** to ensure malicious user queries do not reach the business or database layers.

Frontend to API: Managing Stateless Credentials

Modern **Single Page Applications (SPAs)** and mobile apps interact directly with

APIs, typically utilising **JSON Web Tokens (JWTs)** as "digital passports" for authentication. A major security pitfall is storing these tokens in **browser local storage**, which is highly vulnerable to exfiltration via XSS. Best practices for this boundary include using **HTTP-only, Secure, and SameSite cookies** to prevent scripts from accessing tokens, or storing tokens strictly in-memory.

Service-to-Service (East-West): Eliminating Internal Implicit Trust

In microservice architectures, internal or "**East-West**" traffic is often implicitly trusted if it originates from within the same network. Zero Trust eliminates this by mandating **Mutual TLS (mTLS)** for all service-to-service communications. By requiring both the client and server workloads to present **X.509 certificates**, organisations ensure that every internal request is authenticated. Additionally, **service-level authorisation** using token claims ensures **least-privilege** access, which limits the "blast radius" and prevents **lateral movement** if a single service is breached.

Third-Party Integrations: The Risk of Unsafe Consumption

"Unsafe Consumption of APIs" is a rising threat where developers blindly trust data received from third-party partners. If a partner's API is compromised, malicious data (such as **SQL injection** strings) can be passed into our system, leading to data corruption or exfiltration. To secure this boundary, organisations must treat third-party data with the same suspicion as human input, requiring strict **validation, filtering, and sanitisation** before the data is processed or stored.



Web application architecture and security model



Classical Three-Tier Architecture & Modern Systems

- **Architectural Separation:** The classical model divides applications into three distinct tiers: Presentation (frontend), Application (logic), and Persistence (data storage).
- **Modern Mapping:** This structure **persists** in modern systems, often referred to as "monoliths," where all components required for a business function exist within a single structure, though they may scale horizontally.
- **Security Implications:**
 - **Tier Isolation:** Proper architecture requires physical or logical separation between tiers to enable network-level access control and monitoring.
 - **Compromise Containment:** In a three-tier design, compromising the web server (presentation) does not automatically grant full access to the database; the attacker must pivot through the application logic tier.
 - **Collapsed Tiers:** Two-tier designs (combining presentation and application) reduce infrastructure costs but increase risk ("all eggs in one basket"), as compromising the web server exposes the application logic directly.

The classical **three-tier architecture** remains a cornerstone of application design, dividing functionality into the **Presentation tier** (frontend user interaction), the **Application tier** (business logic processing), and the **Persistence tier** (long-term data storage). In the presentation tier, web servers such as Apache or Nginx serve static pages and collect user input, which is then passed to application servers like Tomcat or JBoss for "heavy-duty" processing according to programming logic. Finally, the persistence tier, comprised of databases like MySQL or PostgreSQL, handles data retrieval and storage for the application.

In modern contexts, these structures are often implemented as **monoliths**, where all components required to deliver a specific business function exist within a single structure. While monoliths can scale horizontally by adding more instances, they typically require synchronous releases of all components due to their tight coupling.

The security implications of this model are significant. Proper architecture necessitates **tier isolation**, allowing for physical or logical separation that enables granular network-level access control and monitoring via firewalls and Intrusion Detection Systems (IDS). This design facilitates **compromise containment**; in a well-implemented three-tier environment, the presentation

tier is never permitted to talk directly to the persistent tier. If an attacker breaches the web server, they do not gain immediate access to the database because they must first pivot through the application logic tier.

Conversely, some systems use **collapsed two-tier designs**, where the presentation and application layers are combined into a single host. While this reduces hardware requirements and deployment costs, it creates an "**all eggs in one basket**" risk. In such setups, a compromise of the web server exposes the application logic and database credentials directly, potentially resulting in the compromise of all three logical tiers simultaneously.



Presentation, Application, and Persistence Tiers

- **Presentation Tier:**

- Components: Web servers (Nginx, Apache, IIS) and client-side code running in browsers.
- Evolution: Modern Single Page Applications (SPAs) move significant presentation logic to the browser, fetching code on the first load and using API calls for subsequent data, effectively making the browser an extension of the server application.

- **Application Tier:**

- Function: Middleware and frameworks (e.g. Spring Boot, Django, Rails) execute business logic and process user input before interacting with the database.
- Role: Acts as a dynamic gateway to generate content and enforce authorization rules.

- **Persistence Tier:**

- Storage Types: Ranging from traditional SQL RDBMS (Oracle, MySQL) to NoSQL document stores (MongoDB) and object storage (AWS S3, Azure Blob).
- Cloud Native: In cloud environments, persistence is often offloaded to managed services (e.g. RDS, DynamoDB), altering the security responsibility model regarding OS patching and backup.

The **three-tier architecture** is a well-established model designed to provide strong security, wonderful scalability, and tight control over sensitive information. By physically and logically separating the **Presentation, Application, and Persistence tiers**, organisations can implement granular network-layer access controls, firewalls, and Intrusion Detection Systems (IDS) between each layer.

The Presentation Tier and SPA Evolution

The **Presentation Tier** is the frontend of the application, historically consisting of web servers like **Nginx, Apache, or Microsoft IIS**. Its primary role is to collect user input and present output in HTML. In traditional models, this tier acts as a proxy for dynamic content, passing data to the application tier for processing. However, modern **Single Page Applications (SPAs)**, built on frameworks such as React or AngularJS, have shifted significant logic into the browser. In this model, the first page load downloads the entire frontend logic to the client. Subsequent interactions occur via **API calls** to the server to fetch data rather than reloading the entire page. This evolution effectively makes the **browser an extension of the server application**.

The Application Tier: The Dynamic Gateway

The **Application Tier** (or business logic tier) is where heavy-duty processing

occurs. Using middleware and frameworks like **Spring Boot, Django, or Rails**, this tier executes the programming logic and processes user input. Critically, it acts as a **dynamic gateway** or "gatekeeper". It is responsible for enforcing **authorization rules**, performing entitlement checks, and validating all requests before they interact with the persistence layer. In a well-implemented three-tier system, the application tier is the only layer permitted to communicate directly with the database, and it must never communicate directly with the internet without passing through the presentation tier.

The Persistence Tier and Cloud-Native Shifts

The **Persistence Tier** serves as the data repository. This includes **Relational Databases (SQL)** like MySQL and Oracle for structured data, and **NoSQL stores** like MongoDB or DynamoDB for unstructured data.

In **cloud-native environments**, persistence is frequently offloaded to managed services such as **AWS RDS, DynamoDB, or object storage like Amazon S3 and Azure Blobs**. This transition alters the **Shared Responsibility Model**:

Infrastructure: The Cloud Service Provider (CSP) becomes responsible for the "security of the cloud," including **OS patching**, hardware maintenance, and physical security.

Data and Access: The customer remains responsible for "security in the cloud," which includes **managing access credentials**, setting proper permissions (IAM), and ensuring data encryption.

Losing visibility into the underlying infrastructure is a common trade-off when moving to these managed services, requiring defenders to shift their focus toward monitoring **cloud-control plane logs** and API activity.



Client-Server Interaction Models & State

- **Stateless Protocol:** HTTP is inherently stateless; each request is independent, and the server retains no knowledge of prior transactions by default.
- **State Management:**
 - **Cookies:** Small text files sent by the server and stored by the client to track state (e.g authentication status) across requests.
 - **Session IDs:** A unique identifier exchanged via cookies or URLs to link a client to a server-side session object.
- **RESTful APIs:** Modern REST APIs emphasize statelessness, where the client must provide all necessary context (authentication tokens) in every request, allowing the server to scale without managing session memory.
- **Interaction Flow:** The client initiates communication (request), and the server provides the resource (response) or an error code (e.g 401 Unauthorized).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Web applications operate on a **client-server architecture**, where the client (typically a browser) initiates a request for resources, and the server responds with the requested data or an informative status code. The foundational protocol for this interaction is **HTTP**, which is defined as inherently **stateless**. This design means that every request is independent; the server retains no memory of prior transactions, and each transaction must stand entirely on its own.

Managing State: Cookies and Session IDs

Because a rich user experience (such as maintaining a shopping cart or remaining logged in) requires continuity, developers must **simulate state**. This is primarily achieved through:

Cookies: Small text files or JSON objects sent by the server via Set-Cookie headers and stored by the client. The browser then automatically includes these in subsequent requests to the same domain to identify the user.

Session IDs: The server assigns a **unique identifier** to an authenticated account, which acts as a "short-term secret" or **continuous re-authentication token**. These IDs can be exchanged via cookies, hidden form fields, or URL parameters.

Statelessness in RESTful APIs

Modern **REST (REpresentational State Transfer)** APIs emphasize a return to

"pure" statelessness to enhance **scalability and performance**. In a RESTful model, the server does not store client-specific session memory; instead, the client is responsible for providing all necessary context, such as **authentication tokens (JWTs)**, within every single request. This allows the backend to scale horizontally with ease, as any server instance can process any request without needing to share complex session data across a cluster.

The Interaction Flow

The standard interaction follows a **request/response model**. The client sends an HTTP method (such as GET or POST) to an endpoint, and the server provides the resource (often as JSON or XML) or an **error code**. Critical among these is the **401 Unauthorized** status code, which signals that authentication is required and has either failed or was not provided. These status codes are essential for the client to understand the outcome of its request and determine the next logical step in the application flow.



Browser-Enforced Security Boundaries

- Same-Origin Policy (SOP):
 - Definition: A critical security mechanism where a browser permits scripts to access data only from the same origin (defined by Protocol, Host, and Port).
 - Restriction: Prevents malicious scripts on one site from reading data or interacting with the DOM of another site (e.g. a banking application).
- CORS (Cross-Origin Resource Sharing):
 - Mechanism: A standard allowing servers to explicitly relax SOP restrictions via HTTP headers (e.g. Access-Control-Allow-Origin).
 - Failures: Misconfigurations, such as setting the allowed origin to a wildcard (*) while allowing credentials, create vulnerabilities that bypass SOP protections.
 - Preflight: Browsers send OPTIONS requests to verify permissions before executing potentially risky cross-origin actions (e.g. POST with custom headers).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The **Same-Origin Policy** is a foundational security mechanism implemented by all modern web browsers to restrict how a script or document loaded from one "origin" can interact with resources from another origin,. An **origin** is strictly defined by a three-part tuple consisting of the **Protocol (Scheme)**, **Host**, and **Port**,,. For two origins to be considered the same, all three components must match exactly; for example, `https://example.com` and `http://example.com` are different origins due to the protocol, while `https://example.com:8080` is distinct due to the port,,.

The primary purpose of SOP is to protect user data by preventing malicious scripts on one site from reading sensitive data, such as **session cookies, local storage, or the Document Object Model (DOM)** of another site,. Without SOP, a script on a malicious website could theoretically "reach into" a banking application tab in the same browser to steal session identifiers or manipulate account data,. However, SOP does not block all interactions; it generally allows the embedding of resources (like images or external scripts) but prevents a script from **reading the response** from a different origin,,.

Cross-Origin Resource Sharing (CORS)

As modern web applications frequently need to fetch data from different sources (e.g. a frontend app on one domain calling an API on another), **CORS** was

introduced to safely relax the rigid restrictions of SOP,. CORS is a header-based standard that allows a server to explicitly declare which origins are permitted to access its resources,,.

The most critical header is **Access-Control-Allow-Origin**, which the server returns to tell the browser if the requesting origin is authorized,. Security failures often occur due to **misconfigurations**. For example, developers frequently use a **wildcard (*)**, which allows any origin to read the response,,. This is particularly dangerous because if the server also allows credentials (like cookies or tokens) via Access-Control-Allow-Credentials: true, a wildcard cannot be used, the server must reflect a specific, validated origin instead,,.

Preflight Requests and OPTIONS

To protect against potentially risky actions that could alter state on the server, browsers utilize a **preflight request** for "non-simple" actions (e.g using methods like **PUT** or **DELETE**, or adding custom headers),,. Before the actual request is sent, the browser automatically issues an **HTTP OPTIONS** request to the server,,. This preflight request uses headers like Access-Control-Request-Method to ask the server for permission,. The server must then respond with headers like Access-Control-Allow-Methods and Access-Control-Allow-Headers,,. If the server grants permission, the browser proceeds with the original request; otherwise, it blocks the transaction entirely to prevent unauthorized cross-origin actions,,.



Security Implications of Modern Frontend Frameworks

- **Logic Shift:** Frameworks like Angular, React, and Vue move MVC (Model-View-Controller) logic to the client-side, increasing the attack surface exposed to the user.
- **Single Page Application Vulnerabilities:**
 - **XSS:** SPAs relying on DOM manipulation must carefully handle data binding to prevent Cross-Site Scripting (XSS), though many frameworks include sanitization by default.
 - **Token Storage:** SPAs often store authentication tokens (JWTs) in local storage (vulnerable to XSS) or cookies (vulnerable to CSRF), requiring trade-offs in security design.
- **API Reliance:** The frontend becomes strictly a consumer of APIs, necessitating robust backend validation since client-side controls can be easily bypassed.

The transition to modern frontend development has fundamentally altered the application security landscape, moving away from traditional server-centric models to highly interactive, client-heavy architectures.

The Logic Shift to Client-Side MVC

Modern frameworks such as **Angular, React, and Vue** have shifted the **Model-View-Controller (MVC)** pattern, traditionally a server-side responsibility, directly into the user's browser. This evolution makes the browser effectively an **extension of the server application**. However, this shift significantly expands the **attack surface** because the entirety of the application's frontend source code is delivered to the untrusted client environment. Adversaries can scrutinise this code to identify hidden features, internal API endpoints, and logic intended for different privilege levels that the developer "forgot" to hide from the client-side delivery.

Single Page Application (SPA) Vulnerabilities

Cross-Site Scripting (XSS): SPAs are particularly susceptible to **DOM-based XSS**, where untrusted user data is used to update the Document Object Model (DOM) through unsafe JavaScript calls. While modern frameworks like **React** provide "secure-by-default" features, such as automatic encoding of strings going to the DOM, these are not invincible. Vulnerabilities persist if developers

use "escape hatches" like `dangerouslySetInnerHTML` or dynamically generate URLs without proper sanitisation.

Token Storage Trade-offs: SPAs face a critical dilemma regarding the storage of authentication tokens like **JWTs**. Storing tokens in **Local Storage** is common for usability, as it persists across browser restarts; however, it is highly vulnerable to exfiltration via XSS because it is accessible to JavaScript. Conversely, using **Cookies** with the `HttpOnly` flag prevents JavaScript access and mitigates token theft, but it introduces the risk of **Cross-Site Request Forgery (CSRF)**, requiring additional defences such as `SameSite` attributes or double-submit cookies.

API Reliance and Backend Validation

In a modern architecture, the frontend is strictly an **API consumer**. Because users can easily intercept, modify, and replay requests using proxies like **Burp Suite** or **ZAP**, all client-side security controls must be viewed as purely for user experience and performance. **Robust backend validation** is mandatory; the server must independently verify every request's authentication, authorisation, and data integrity, treating all incoming data from the client as untrusted and potentially malicious.



TLS for Application Practitioners

- **Termination Points:** In modern architectures, TLS is often terminated at the load balancer or reverse proxy, meaning traffic within the internal network (East-West traffic) may be unencrypted unless mutual TLS (mTLS) or service meshes are implemented.
- **Trust Assumptions:** Relying on the perimeter for termination assumes the internal network is trusted; Zero Trust models require re-encrypting traffic to the application server.
- **Misconfigurations:**
 - **Weak Suites:** Supporting legacy protocols (SSLv3, TLS 1.0) or weak ciphers exposes traffic to downgrade attacks.
 - **Certificate Chains:** Improperly configured chains or failure to renew certificates can cause service outages or trust errors for clients.

In modern application architecture, **TLS termination** (or offloading) typically occurs at a **load balancer or reverse proxy** situated at the network perimeter. While this centralises certificate management and offloads expensive cryptographic processing from application servers, it often leaves "**East-West**" (**internal service-to-service**) **traffic unencrypted** as it traverses the internal network. In a traditional "castle-and-moat" security model, this internal network is implicitly trusted; however, the **Zero Trust** paradigm assumes the network is always hostile. To adhere to Zero Trust, practitioners must implement **re-encryption** from the load balancer to the backend application server or utilize a **service mesh**. Service meshes like Istio or Linkerd automate this by injecting **sidecar proxies** into pods, which transparently upgrade internal communications to **Mutual TLS (mTLS)**, ensuring that both the client and server workloads are authenticated and their data remains confidential.

Security misconfigurations at the TLS layer introduce severe risks. Supporting legacy protocols such as **SSLv3 or TLS 1.0** makes systems vulnerable to **downgrade attacks** (like Logjam or POODLE), where an adversary forces the peers to negotiate weak, often export-grade, encryption that can be compromised. Practitioners should disable these weak suites and instead mandate **TLS 1.2 or 1.3**, preferring **ECDHE** cipher suites to ensure **Perfect**

Forward Secrecy (PFS). PFS ensures that even if a server's long-term private key is later stolen, an attacker cannot decrypt past captured sessions.

Finally, the integrity of the **certificate chain** is paramount. Clients verify the "path of trust" from the server's certificate through any **intermediate CAs** back to a **trusted root CA**. If this chain is improperly configured or if an administrator **fails to renew** a certificate before it expires, it results in the "dreaded 'Certificate has Expired' message," causing immediate **service outages** and trust errors for clients. Utilizing managed services like **AWS Certificate Manager (ACM)** or automated providers like **Let's Encrypt** can help mitigate these risks by automating the renewal and deployment of valid certificates.



Reverse Proxies, CDNs, and Load Balancers

- **Placement & Function:**
 - **Load Balancers:** Distribute traffic across healthy instances; Layer 7 (Application) balancers can route based on content (URL, headers), while Layer 4 (Network) balancers route based on IP/Port.
 - **CDNs:** Cache static content (images, JS, CSS) at the edge to reduce latency and absorb DDoS traffic.
- **WAF Integration:** Web Application Firewalls (WAFs) are typically deployed at this edge layer to inspect incoming traffic for malicious patterns (SQLi, XSS) before it reaches the origin.
- **Trust Confusion:** Shared infrastructure can lead to risks where a WAF or CDN is misconfigured to trust traffic based on headers (like X-Forwarded-For) that can be spoofed if not properly sanitized at the boundary.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The modern web perimeter is no longer a single firewall but a sophisticated edge layer comprising **load balancers, Content Delivery Networks (CDNs), and reverse proxies**. These components work in tandem to ensure **high availability, performance, and security** by brokering traffic between external clients and backend origin servers.

Load Balancing and Routing Logic

Load balancers distribute incoming requests across a pool of healthy backend instances to prevent any single resource from becoming overwhelmed. They operate at different layers of the OSI model:

Layer 4 (Network) Balancers: These operate at the transport layer, making extremely fast forwarding decisions based strictly on **IP addresses and TCP/UDP port numbers**.

Layer 7 (Application) Balancers: These operate at the application layer, allowing for more **granular, informed decisions**. They can route traffic based on specific content within the HTTP request, such as **URL paths, headers, or cookies**.

Content Delivery Networks (CDNs) at the Edge

A **CDN** is a geographically distributed network of servers designed to serve content from **edge nodes** located close to the end-user. By **caching static content**, such as HTML, JavaScript, and images, at these nodes, CDNs

significantly **reduce network latency** and improve page load speeds. Furthermore, because CDNs possess massive bandwidth capacity, they act as a critical defensive layer that can **absorb volumetric Distributed Denial of Service (DDoS) traffic**, protecting the origin server from being crashed by floods of malicious requests.

WAF Integration and Pattern Inspection

Web Application Firewalls (WAFs) are typically integrated at this edge layer, often fronting the CDN or load balancer. Unlike traditional firewalls, a WAF performs **Deep Packet Inspection (DPI)** on Layer 7 traffic to identify and block malicious patterns, such as **SQL Injection (SQLi)**, **Cross-Site Scripting (XSS)**, and **command injection**, before they ever reach the application logic.

The Risk of Trust Confusion

A significant security challenge arises from the use of **shared infrastructure**. When a request passes through a proxy or CDN, the backend server sees the **IP address of the proxy** rather than the true client. To preserve the original source identity, proxies often insert an **X-Forwarded-For** header. However, this introduces **trust confusion**: because these headers are arbitrary, an attacker can **spoof** them to make a request appear as if it originated from a trusted internal IP address. If a WAF or backend system is misconfigured to **implicitly trust these headers** without proper sanitisation at the network boundary, an adversary can **bypass IP-based access controls** or elude attribution. This "confused deputy" scenario highlights the necessity of **validating all metadata** at every trust boundary.



Browser Security Headers

- **HSTS (HTTP Strict Transport Security):** Forces browsers to communicate with the server only over HTTPS, mitigating SSL stripping attacks by refusing unencrypted connections.
- **Content-Type Protections:**
 - **X-Content-Type-Options: nosniff:** Prevents browsers from "sniffing" the response type, forcing them to adhere to the declared Content-Type, which mitigates certain XSS vectors involving file uploads.
- **Referrer Policy:** Controls how much information is included in the Referer header when navigating away from a page, reducing data leakage (e.g. exposing session tokens in URLs).
- **Content Security Policy (CSP):** Restricts the sources from which a browser is allowed to load resources (scripts, images), significantly reducing the impact of XSS attacks.

Web security headers are **HTTP response headers** used by web servers to instruct browsers on how to handle content and enforce specific security policies. These headers act as a **secondary layer of defence**, mitigating common vulnerabilities such as Cross-Site Scripting (XSS), clickjacking, and session hijacking.

HSTS (HTTP Strict Transport Security)

HTTP Strict Transport Security (HSTS) is a mechanism that forces a browser to communicate with a server exclusively over **encrypted SSL/TLS connections**.

Once a browser receives the Strict-Transport-Security header, it will automatically convert any future insecure `http://` requests to secure `https://` requests for a duration defined by the `max-age` parameter. This is vital for mitigating **SSL stripping attacks**, where an adversary attempts to downgrade a user's secure connection to cleartext. Additionally, HSTS ensures that users cannot bypass **certificate error warnings**, preventing them from connecting to potentially compromised servers.

Content-Type Protections

The **X-Content-Type-Options: nosniff** header is used to disable a browser feature known as **MIME sniffing**. Without this protection, browsers might ignore the server's declared Content-Type and attempt to guess the file type based on

its content. This allows attackers to execute **XSS attacks** by uploading malicious files, such as a graphic containing HTML or script code, that the browser mistakenly executes as a script. Setting this header forces the browser to strictly adhere to the **MIME type** provided in the HTTP header.

Referrer Policy

The **Referrer Policy** provides granular control over how much information is included in the **Referer header** when a user navigates from one page to another. Historically, the Referer header would leak the entire URL of the previous page, which could contain **sensitive information** such as session tokens, user identifiers, or private data within query parameters. By configuring policies like no-referrer or origin, developers can restrict this data sharing to prevent **unintended information disclosure** to third-party sites or server logs.

Content Security Policy (CSP)

Content Security Policy (CSP) is a highly effective security header designed to prevent and mitigate **XSS and clickjacking attacks**. It allows site owners to define an **allowlist of trusted sources** from which the browser is permitted to load and execute dynamic resources, such as JavaScript, CSS, and images. A robust CSP can **disable inline scripts** and the use of dangerous functions like `eval()`, ensuring that only vetted, pre-approved code runs in the user's browser context. Furthermore, the **frame-ancestors** directive within CSP serves as a modern replacement for X-Frame-Options, effectively securing applications against UI redressing or clickjacking.



Core authentication and authorization pillars



Identity, Authentication, and Authorization

- **Separation of Concerns:** Security architectures must distinguish between who an entity is (Identity), verifying that claim (Authentication), and determining what they can do (Authorization), as coupling them leads to logic flaws and privilege escalation.
- **Identity (The "Who"):** This is the user profile or subject record (e.g. a User ID or email) existing within a directory; it establishes the claim of who the entity is before any verification occurs.
- **Authentication (The "Verify"):** The process of validating the identity claim using credentials, such as passwords, tokens, or biometrics, to prove the user is who they claim to be.
- **Authorization (The "What"):** The decision logic that occurs after successful authentication to determine if the verified identity has permission to access specific resources or perform actions (e.g. READ access to /invoices).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Separation of Concerns

A robust security architecture depends on the **separation of concerns** between these three functions. While they are closely related and used in conjunction to provide granular access, they are distinctly different processes. **Muddling or coupling these concepts is a significant security risk**; for instance, if the same technique is used for both authentication and authorization, a breach in one model inevitably leads to a breach in the other. Furthermore, security designs must ensure that although a user is successfully authenticated, it does not automatically imply they have open access to all functions, as this leads to **broken function-level authorization (BFLA)** and **privilege escalation**.

Identity (The "Who")

Identity is the unique representation of a subject, whether a human, machine, or application, engaged in a digital transaction. It defines **who the entity is** before any verification occurs. In modern systems, this identity often exists as a **user profile or subject record** in a centralized directory, such as Active Directory or Entra ID. This identity is associated with various **attributes**, such as job title, department, or location, which are later used to inform access decisions.

Authentication (The "Verify")

Authentication is the process of **verifying the truth of an identity claim**. It

answers the question, "Are you who you say you are?". This is typically achieved through the presentation of **credentials** based on one or more of four factors:

Something you know: Passwords, passphrases, or PINs.

Something you have: Smart cards, physical tokens, or mobile devices (e.g. TOTP).

Something you are: Static biometrics like fingerprints or facial recognition.

Something you do: Dynamic biometrics like typing rhythm or voice patterns.

Using multiple factors (**MFA**) is a security best practice, as it provides higher identity confidence than relying on a single factor.

Authorization (The "What")

Authorization (or access control) is the decision logic that occurs **only after successful authentication**. It stipulates **what an authenticated user is permitted to do** within a system boundary. This process involves evaluating the permissions, roles, and privileges associated with the verified identity. Effective authorization should follow the **principle of least privilege**, granting only the minimum permissions necessary for a task and employing a "**deny by default**" policy to prevent unauthorized access. For example, a user may be authenticated as an employee but explicitly denied **READ access to sensitive financial files** (/invoices) if their role does not require it.



Traditional Authentication Mechanisms

- **Forms, Sessions, and Cookies**
- **Form-Based Authentication:** The user submits credentials (username/password) via a web form; if valid, the server creates a stateful session.
- **HTTP Basic Authentication:** A legacy method where credentials are Base64 encoded (not encrypted) and sent in the HTTP header; easily compromised without TLS.
- **Session Management:** The server generates a Session ID (reference token) stored in a database and sends it to the client, usually as a cookie; the server tracks the state of the user.
- **Cookie Security Risks:** Session cookies are vulnerable to Cross-Site Scripting (XSS) theft and Cross-Site Request Forgery (CSRF) if flags like HttpOnly and SameSite are not properly configured.

Form-based authentication is the most prevalent method used by modern web applications. Unlike protocol-level schemes, it is **handled by the application developer** or a framework rather than the web server itself. The process typically involves an HTML form where a user provides credentials, which are then transmitted to a server-side processing component, usually via an **HTTP POST request**. This method is highly flexible, allowing developers to implement custom logic, such as multifactor authentication (MFA) or complex credential requirements. Because POST requests are transmitted in cleartext by default, it is a mandatory best practice to use **SSL/TLS encryption** to ensure the confidentiality of the credentials during transit.

HTTP Basic Authentication

HTTP Basic Authentication is a legacy, built-in scheme defined by the HTTP protocol. When a client attempts to access a restricted resource, the server responds with an **HTTP 401 Unauthorized** status code and a WWW-Authenticate: Basic header. The browser then prompts the user for a username and password through a standard dialogue box. These credentials are concatenated with a colon and **Base64 encoded** before being sent in the Authorization header of subsequent requests. It is critical to understand that **Base64 is an encoding, not encryption**; it can be trivially decoded with simple

tools. Consequently, Basic authentication is completely insecure unless protected by a **TLS tunnel**. Furthermore, this method lacks a native logout function; credentials remain cached in the browser's memory until it is closed entirely.

Session Management

Because the **HTTP protocol is inherently stateless**, servers have no built-in way to associate multiple requests as part of the same conversation. **Session management** simulates state by assigning a unique **Session ID** (or reference token) to an authenticated user. This ID is typically a cryptographically random string generated with high entropy to ensure it is **unguessable**. The server stores this ID in a database or cache, while the client retains it, most commonly within a **cookie**. For every subsequent request, the browser includes this token, which acts as a form of **continuous re-authentication**, allowing the server to recognize the user without requiring their password again.

Cookie Security Risks and Mitigations

Session cookies are high-value targets for adversaries. If an attacker steals a valid session cookie, they can **impersonate the user** without knowing their credentials.

Cross-Site Scripting (XSS): Attackers use malicious scripts to exfiltrate cookies from the browser. Setting the **HttpOnly flag** prevents client-side scripts (like JavaScript) from accessing the cookie, serving as a primary defence against token theft.

Cross-Site Request Forgery (CSRF): This attack tricks a logged-in user's browser into sending an unsolicited request to a vulnerable API. Because browsers automatically attach cookies to requests for a given domain, the server may process the forged request as legitimate.

Mitigation Flags: The **SameSite attribute** (Strict or Lax) is essential for mitigating CSRF by regulating whether cookies are sent in cross-site contexts. Additionally, the **Secure flag** must be applied to ensure the cookie is only transmitted over encrypted HTTPS connections, preventing network sniffing.



Federated Authentication

- **OAuth 2.0 and OpenID Connect (OIDC)**
- **Delegated Access (OAuth 2.0):** An authorization framework that allows third-party applications to obtain limited access to a user's resources without exposing credentials.
- **Identity Layer (OIDC):** OpenID Connect sits on top of OAuth 2.0 to provide authentication; it issues an ID Token (JWT) that asserts the user's identity to the client, solving the "who are you" problem that OAuth 2.0 alone does not address.
- **Centralized Identity:** Organizations use Identity Providers (IdPs) like Okta or Entra ID to manage credentials centrally, reducing the attack surface of individual applications.

Federated identity is an architectural model that allows users to access multiple systems or applications using a **single set of credentials** by linking identities across different management systems. This approach establishes trust between an **Identity Provider (IdP)**, which serves as the authoritative source of identities, and an **Identity Consumer** (or Service Provider), which relies on the IdP for authentication decisions.

Delegated Access (OAuth 2.0)

OAuth 2.0 is an extensible **authorization framework** rather than a specific authentication protocol. Its primary purpose is to solve the **access delegation problem**, allowing a third-party application (the Client) to obtain limited access to a user's resources (hosted on a Resource Server) without the user having to share their login credentials with that third party.

This is often explained using the **"valet key" concept**: a car owner can give a valet a special key that allows them to drive the car but prevents them from accessing the boot or glove box. In the digital realm, OAuth issues a **time-bound Access Token** that is restricted to specific **scopes**, well-defined purposes or permissions approved by the user.

The Identity Layer (OpenID Connect)

While OAuth 2.0 handles authorization ("What are you allowed to do?"), it does

not inherently address authentication ("Who are you?"). **OpenID Connect (OIDC)** is a standardised **identity layer** built directly on top of the OAuth 2.0 protocol to provide this missing functionality.

OIDC enables the IdP to issue an **ID Token**, which is a cryptographically signed **JSON Web Token (JWT)** that asserts the user's identity to the client. This token contains **claims**, key-value pairs of data such as the user's unique identifier (sub), name, or email address, which the client can verify using the IdP's public key.

Centralised Identity and IdPs

Modern organisations utilise **Identity Providers (IdPs)** such as **Okta**, **Microsoft Entra ID** (formerly Azure AD), or **AWS Identity Center** to manage credentials in a centralised repository. Centralising identity offers significant security advantages:

Reduced Attack Surface: It eliminates "identity silos", ensuring users do not have to manage (and potentially reuse) different passwords for every individual application.

Administrative Efficiency: Administrators can perform **centralised provisioning and deprovisioning**; when an employee leaves, their access to all federated resources can be revoked in a single action at the IdP level.

Enhanced Protection: IdPs can enforce consistent, robust security policies across all applications, such as **Multi-Factor Authentication (MFA)** and **Conditional Access** rules based on risk factors like location or device health.



OAuth Flows in Context

- **Selecting the Right Flow to Prevent Misuse**
- **Authorization Code Flow:** The gold standard for user-facing applications (web and mobile); it exchanges a temporary code for a token via a back-channel, preventing token leakage in the browser.
- **Client Credentials Flow:** Used strictly for machine-to-machine (M2M) communication where no user is present; services authenticate using their own secrets to access resources.
- **PKCE (Proof Key for Code Exchange):** A mandatory security extension for the Code Flow that prevents authorization code interception attacks; it cryptographically binds the code request to the token exchange.
- **Misuse & Deprecation:** The Implicit Flow (returning tokens directly in the URL) and Resource Owner Password Credentials (ROPC) flow (handling passwords directly) are deprecated due to severe security risks and should no longer be used.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Selecting the correct **OAuth 2.0 flow** is essential for maintaining a secure environment and preventing the exposure of sensitive credentials. The **Authorization Code Flow** is the established **gold standard** for user-facing applications, such as web and mobile apps, because it employs a multi-step process that separates the browser from the final token acquisition. By returning a short-lived **authorization code** (valid for approximately one minute) to the browser instead of the token itself, the flow allows the client to perform a **back-channel** exchange for the access token directly with the authorisation server. This design ensures that **access tokens** are not leaked through browser history, referrer headers, or web server logs.

For **machine-to-machine (M2M)** scenarios where no user is present, such as automated background processes or service-to-service communication, the **Client Credentials Flow** is utilised. In this model, the service acts as its own resource owner and authenticates using its own **client ID and secret** directly at the token endpoint. Because there is no user session to maintain, this flow typically does not issue **refresh tokens**.

To address vulnerabilities where a malicious application might intercept an authorization code, **PKCE (Proof Key for Code Exchange)** is a mandatory extension for the Code Flow. It functions by generating a random runtime secret

called a **code_verifier**, which is hashed to create a **code_challenge** sent with the initial request. The authorisation server binds the code to this challenge, and the client must provide the original verifier during the token exchange to prove it is the same entity that initiated the flow, effectively defeating **code interception and injection attacks**.

Finally, modern standards have **deprecated** several legacy flows due to high risk. The **Implicit Flow** is obsolete because it returns tokens directly in the **URL fragment**, making them vulnerable to theft via browser history or cross-site scripting (XSS). Similarly, the **Resource Owner Password Credentials (ROPC)** flow is deprecated for new systems because it requires users to share their passwords directly with the application, violating the core security principles of OAuth. Organizations should transition these legacy implementations to the more secure **Authorization Code Flow with PKCE**.



Token-Based Authentication

- **JSON Web Token Structure (JWTs)**
- **Stateless Authentication:** JWTs are self-contained tokens that carry identity and authorization data (claims), allowing APIs to validate requests without querying a central database for every call.
- **Structure:** Composed of three Base64Url encoded parts separated by dots:
 - Header: Specifies the token type and signing algorithm (e.g. HS256, RS256).
 - Payload: Contains Claims such as sub (subject), exp (expiration), iss (issuer), and custom permissions.
 - Signature: Cryptographic proof generated using the header, payload, and a secret/key to ensure data integrity.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Stateless Authentication with JWTs

JSON Web Tokens (JWTs) are defined as a **compact, self-contained** format for securely transmitting information, represented as a JSON object, between parties. A primary advantage of JWTs is that they enable **stateless authentication**. Unlike traditional session management, where a server must store session data in a database and query it for every incoming request, a JWT carries all the necessary identity and authorisation data (claims) within the token itself. This allows APIs to validate requests independently, which significantly improves **scalability** by allowing any server in a distributed environment to process the request without needing access to a centralised session store.

The Structure of a JWT

A JWT is composed of three distinct parts, each **Base64Url encoded** and separated by a dot (.).

The Header: This is the first part of the token, identifying the metadata. It typically specifies the **token type** (usually "JWT") and the **signing algorithm** used to secure the token. Common algorithms include **HS256** (HMAC with SHA-256), which is symmetric and uses a shared secret, and **RS256** (RSA with SHA-256), which is asymmetric and utilizes a private/public key pair.

The Payload: The middle section contains **claims**, which are assertions about

the subject (typically the user) and additional metadata. Standard registered claims include:

iss (Issuer): Identifies the authority that issued the token.

sub (Subject): The unique identifier for the user or service account.

exp (Expiration): A timestamp indicating when the token becomes invalid, mitigating risks if the token is leaked.

Custom Claims: Developers can include bespoke data, such as **user roles or specific permissions**, to enable fine-grained authorisation logic within the API.

The Signature: This is the final component, providing **cryptographic proof** of the token's origin and integrity. It is generated by taking the encoded header and payload and signing them with a secret key or a private key using the algorithm specified in the header.

Data Integrity and Verification

The signature ensures that the payload has not been **tampered with** during transit. When an API receives a JWT, it recomputes the signature using its own copy of the secret or public key; if the recomputed signature matches the one on the token, the API can trust that the claims are **authentic and unaltered**.

However, it is important to note that Base64Url encoding is not encryption; the contents of a standard JWT are **world-readable**, meaning sensitive PII (Personally Identifiable Information) should not be included unless the token is further protected via **JSON Web Encryption (JWE)**.



Signing Trade-offs

- **Symmetric vs. Asymmetric Signing**
- **Symmetric Signing (HMAC/HS256):** Uses a single shared secret for both signing and verification.
 - Trade-off: Faster computation but risky for distributed systems; if an external service needs to verify the token, you must share the secret key with them, increasing the risk of key compromise.
- **Asymmetric Signing (RSA/RS256 or ECDSA):** Uses a Private Key to sign and a Public Key to verify.
 - Trade-off: The Authorization Server holds the private key; all downstream APIs use the widely available public key to verify. This enables a zero-trust architecture where services verify tokens without being able to forge them.

Symmetric Signing (HMAC/HS256)

Symmetric cryptography is a method where a **single secret key** is shared between two or more parties and is used for both the encryption and decryption, or in this context, the signing and verification, of data. A common implementation in web security is **HS256**, which stands for **HMAC (Hash-based Message Authentication Code)** using the **SHA-256** hashing algorithm.

Trade-off: Speed vs. Distribution Risk. The primary advantage of symmetric signing is its performance; it is **computationally "cheap" and significantly faster** than asymmetric alternatives. However, its greatest weakness lies in **key distribution**. Because the same secret is required for both creation and validation, an **Authorization Server (AS)** must share its secret key with any external or downstream service that needs to verify a token. This drastically increases the **risk of key compromise**; if any one service in a distributed system is breached, the shared secret is exposed, allowing an attacker to **forge valid tokens** for the entire system.

Asymmetric Signing (RSA/RS256 or ECDSA)

Asymmetric encryption, often called **public-key cryptography**, utilises a mathematically linked **key pair**: a **Private Key** and a **Public Key**. Common examples include **RS256** (RSA with SHA-256) and **ECDSA** (Elliptic Curve Digital

Signature Algorithm), the latter of which provides equivalent security with much **smaller key sizes**.

Trade-off: Security vs. Performance. In this model, the **Authorization Server holds the private key** and uses it to sign tokens. Downstream APIs and services use the **widely available public key** to verify the signature. This design is the bedrock of a **zero-trust architecture** because it allows services to verify the integrity and origin of a token without ever having the ability to **forge or alter** it. While this improves security by narrowing the "blast radius" of a potential compromise, it comes at a **high performance cost**, as asymmetric operations are orders of magnitude slower than symmetric ones.



Common Implementation Failures

- **Verifying JWTs Incorrectly**
- **Algorithm Confusion:** Attackers can force the server to use a symmetric algorithm (HS256) while providing the public key as the "secret," allowing them to forge valid signatures.
- **The "None" Algorithm:** Critical vulnerability where libraries accept tokens with alg: none, allowing attackers to bypass signature verification entirely by stripping the signature section.
- **Missing Claim Validation:** Failing to check the aud (audience) claim allows a token issued for Service A to be used against Service B. Failing to check exp (expiration) allows replay attacks with old tokens.

Algorithm Confusion Attacks

Algorithm confusion is a sophisticated attack that exploits a server's failure to enforce a specific cryptographic algorithm during token validation. In a secure asymmetric setup (such as **RS256**), the authorization server signs a token with a **private key**, and the API verifies it using a **public key**. However, if the API's validation library is poorly implemented, an attacker can modify the JWT header to specify a symmetric algorithm like **HS256**. The attacker then uses the server's **public key**, which is, by definition, public, as the HMAC "**secret**" to sign a forged token. Because the library sees "HS256" in the header, it performs a symmetric check using the provided public key as the secret, successfully validating the attacker's forged signature. To mitigate this, practitioners must **ignore the alg header** from the client and strictly hardcode the expected algorithm on the server side.

The "None" Algorithm Vulnerability

The "**None**" algorithm represents a critical flaw where a library is configured to accept tokens with an alg field set to none. While the specification allows for unsigned tokens in rare cases where integrity is already guaranteed, allowing them in an API context effectively **bypasses signature verification**. An adversary can decode a legitimate JWT, modify the payload to **escalate privileges** (e.g

changing a user role to "admin"), change the algorithm to none, and strip the signature section entirely. Some implementations have also failed due to **case-sensitivity flaws**, where they rejected none but mistakenly accepted variants like nonE.

Failure in Claim Validation

Even if a signature is valid, failing to validate specific **claims** within the payload leads to severe security gaps:

Audience (aud) claim: This claim identifies the intended recipient of the token. If an API does not verify that it is the intended audience, a token issued for a low-security "Service A" could be **replayed** to gain access to a high-security "Service B," provided they trust the same issuer.

Expiration (exp) claim: This timestamp indicates when the token must no longer be accepted. Missing this check allows **replay attacks** using old, potentially compromised tokens that should have been retired, indefinitely extending the window for a threat actor to impersonate a user.



Multi-Factor Authentication (MFA) Strength

- **Not All Factors Are Created Equal**
- **SMS/Voice (Weakest):** Vulnerable to SIM swapping and SS7 interception attacks; considered a deprecated form of MFA by modern standards.
- **TOTP (Moderate):** Time-based One-Time Passwords (e.g. Google Authenticator) prevent SIM swapping but are still susceptible to real-time phishing proxies (AiTM).
- **Push Notifications (Moderate/Strong):** User approves a prompt on a device; convenient but vulnerable to "MFA Fatigue" (Prompt Bombing) where attackers spam requests until the user accepts.
- **FIDO2/Hardware Keys (Strongest):** Hardware-backed (e.g. YubiKey, Passkeys) uses cryptography bound to the domain, making it technically un-phishable.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Multi-factor authentication (MFA) is a critical security measure requiring users to present two or more independent factors, **something they know** (passwords), **something they have** (tokens), or **something they are** (biometrics), to verify their identity. However, as this slide illustrates, the security assurance provided by different MFA factors varies significantly.

SMS and Voice (Weakest)

While popular due to ease of use, **SMS and voice-based authentication** are considered the least secure methods. They are highly vulnerable to **SIM swapping (or SIM jacking)**, where an attacker socially engineers a mobile carrier to transfer a victim's phone number to a new SIM card. Additionally, these methods are susceptible to **SS7 interception attacks**, which exploit vulnerabilities in the underlying telephony signaling protocol to capture codes in transit. Due to these inherent risks, **NIST officially deprecated SMS** as a secure form of authentication in 2016.

TOTP (Moderate)

Time-based One-Time Passwords (TOTP), utilized by apps like **Google Authenticator**, offer moderate security by generating codes that change periodically (e.g. every 60 seconds). This prevents SIM swapping because the "secret" is stored locally on the device rather than tied to a phone number.

However, TOTP remains **vulnerable to real-time phishing proxies**, also known as **Adversary-in-the-Middle (AiTM)** attacks. In this scenario, an attacker tricks a user into entering both their password and the current TOTP code into a fraudulent site, which the attacker immediately relays to the legitimate service.

Push Notifications (Moderate/Strong)

Push notifications provide a more convenient user experience by prompting a user to "Approve" or "Deny" a login request on an already-authenticated mobile app. While stronger than SMS, this method is susceptible to **"MFA Fatigue" (or Prompt Bombing)**. In such an attack, an adversary who has already compromised a user's password repeatedly triggers MFA prompts until the frustrated or distracted user accidentally approves the fraudulent request. Modern implementations mitigate this by requiring the user to **match a specific number** displayed on the login screen within the app.

FIDO2 and Hardware Keys (Strongest)

FIDO2 and hardware-backed keys (such as **YubiKeys or Passkeys**) represent the gold standard of authentication. These methods use **asymmetric cryptography** and are cryptographically **bound to the specific domain** the user is visiting. Because the hardware device will only sign a challenge if the domain in the browser matches the registered domain, it is **technically un-phishable**. Unlike other factors, even if a user is lured to a perfectly cloned website, the hardware key will detect the domain mismatch and refuse to authenticate, effectively defeating sophisticated AiTM attacks.



MFA Real-World Bypass Techniques

- **Deployment Gaps and Attack Vectors**
- **Real-time Phishing (AiTM):** Attackers use proxy tools (e.g. Evilginx) to intercept the MFA session cookie in real-time, bypassing the need for the code itself.
- **MFA Fatigue (Prompt Bombing):** Attackers issue repeated push notifications late at night, hoping the user accepts one to stop the noise.
- **Browser Pivoting:** Once a machine is compromised, attackers use tools to dump session cookies or inject into the browser process, reusing the already-authenticated session without needing to re-challenge MFA.
- **Legacy Protocols:** Attackers bypass MFA policies by targeting legacy protocols (e.g. IMAP, SMTP) that do not support MFA challenges.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

While Multi-Factor Authentication (MFA) is a critical component of a **Zero Trust** security model, it is not a "silver bullet," and determined adversaries have developed sophisticated techniques to circumvent it.

Real-time Phishing (AiTM)

Adversary-in-the-Middle (AiTM) attacks represent a major evolution in phishing. Attackers use proxy tools such as **Evilginx 3.0** to sit between a victim and a legitimate login page, such as Microsoft Online. This setup allows the attacker to act as a reverse proxy, intercepting the victim's credentials and, more importantly, their **session cookies** or **Primary Refresh Tokens (PRTs)** in real-time. Because these tokens are issued only after a successful MFA challenge, they often contain an **MFA claim**. When the attacker injects these stolen cookies into their own browser, the service perceives the session as already authenticated, effectively **bypassing the MFA requirement**.

MFA Fatigue (Prompt Bombing)

MFA Fatigue, or **prompt bombing**, is a psychological attack that exploits user frustration rather than technical vulnerabilities. An attacker who has already obtained a user's password will trigger an **unusual or excessive number of MFA push notifications** to the user's mobile device. Adversaries often perform this "bombing" late at night, hoping the user will eventually **approve the request**.

simply to silence the persistent alerts or because they mistakenly believe their previous login attempt failed.

Browser Pivoting

Browser pivoting is a post-exploitation technique used once an attacker has already established a foothold on a victim's machine. Using frameworks like **Cobalt Strike**, an attacker injects a malicious HTTP proxy directly into the user's active browser process. This allows the attacker to route their own web traffic through the victim's browser, **inheriting all existing authenticated sessions and credentials**. Because the attacker is using the already-authenticated browser "from within," they can access sensitive cloud resources **without being prompted for a new MFA challenge**.

Legacy Protocols

Adversaries frequently target **legacy protocols** such as **IMAP, POP3, and SMTP**, which were designed in an era before modern authentication and often do not support MFA challenges. Attackers use tools like **MFASweep** to automate login attempts across various endpoints to identify **Conditional Access (CA) policy gaps** or MFA exceptions intended for legacy systems or mobile devices. By successfully authenticating via an unprotected legacy endpoint, an attacker can gain access to a mailbox even if the primary web portal is secured by MFA.



Authorization Models

- **Designing Access Control**
- **RBAC (Role-Based Access Control):** Assigns permissions to static roles (e.g. Admin, Editor); users are assigned roles. Simple to manage but lacks granularity for specific data ownership.
- **ABAC (Attribute-Based Access Control):** dynamic policies based on attributes of the user, resource, and environment (e.g. "Allow access if User Location=US AND Time=BusinessHours"); highly granular but complex to implement.
- **ReBAC (Relationship-Based Access Control):** Derived from social graphs (like Google Zanzibar); access is determined by relationships (e.g. "User is owner of Folder X, Document Y is inside Folder X, therefore User can view Document Y").

Authorization Models: Designing Access Control

Authorization is the process that determines what an authenticated identity is permitted to do within a system. Selecting the correct authorization model is critical for enforcing the **principle of least privilege**, ensuring entities only have the bare minimum permissions required for their tasks.

RBAC (Role-Based Access Control)

RBAC assigns permissions to **static roles** defined by job functions within an organisation, such as "Admin" or "Editor". There is a many-to-many relationship in this model: users are assigned to roles, and roles are assigned specific permissions. RBAC is **simple to manage** and reduces administrative overhead because permissions can be updated for an entire role rather than for individual users. However, its static nature means it **lacks granularity** for specific data ownership. This often leads to "**role explosion**," where an organisation creates an unmanageable number of roles to satisfy granular requirements.

ABAC (Attribute-Based Access Control)

ABAC is a **highly granular and flexible** model that makes access decisions dynamically. It evaluates rules based on **attributes** from four categories: the subject (e.g. job title), the resource (e.g. classification), the action (e.g. read/write), and the environment (e.g. time of day or location). Policies can

enforce complex conditions, such as "Allow access only during business hours if the user is in the UK". While powerful, ABAC is **complex to implement** and manage as user bases grow, and it requires significant computational overhead to evaluate multiple predicates for every request.

ReBAC (Relationship-Based Access Control)

ReBAC models users and resources as nodes in a **social graph**, where access is determined by the **relationships (edges)** between them. Derived from implementations like **Google Zanzibar**, it is used to manage complex ownership and hierarchical models. For example, if a user owns "Folder X" and "Document Y" is located inside that folder, the user automatically inherits access to the document via that relationship. ReBAC allows organisations to traverse the graph to answer not just if a user has access, but **who** has access to a specific resource.



Object-Level Authorization Failures (BOLA/IDOR)

- **The #1 API Vulnerability**
- **Definition: Broken Object Level Authorization (BOLA)**, also known as **Insecure Direct Object Reference (IDOR)**, occurs when an application fails to check if the authenticated user owns the specific data record being requested.
- **Mechanism:** An attacker manipulates the ID in an API endpoint (e.g. changing /receipts/1001 to /receipts/1002) to access records belonging to other users.
- **Root Cause:** Developers rely on the fact that the user is authenticated, forgetting to verify they are authorized for that specific object ID.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Broken Object Level Authorization (BOLA), traditionally known as **Insecure Direct Object Reference (IDOR)**, is consistently ranked as the **number one vulnerability** in the OWASP API Security Top 10,,,. This flaw is exceptionally prevalent because it exploits the core architecture of modern APIs, which typically expose endpoints that rely on user-controlled input to identify specific data objects,,.

The Mechanism of Attack

The primary mechanism for this vulnerability is **parameter manipulation**,. In a typical API, resources such as invoices, user profiles, or medical records are addressed via unique identifiers (IDs) in the URL path, query string, or request body,,. An attacker simply alters these identifiers, often switching a sequential integer like /receipts/1001 to /receipts/1002, to see if the system returns data belonging to another user,,. While some developers attempt to mitigate this by using **randomised IDs like UUIDs**, this is considered "security by obscurity" and is insufficient on its own, as UUIDs can often be leaked through other API responses or public documentation,,.

Root Cause: The "Authenticated Equals Authorized" Fallacy

The root cause is a fundamental logic flaw where developers rely on the fact that a user has a valid session or token but **fail to verify that the specific identity**

owns the specific record being requested,,. This often occurs when developers use **Object-Relational Mappers (ORMs)** that bind user input directly to database queries without an intervening authorization check,. They erroneously assume that if a user is authenticated, any ID they provide must be one they are entitled to see,,.

Real-World Impact and Prevention

The consequences of BOLA are devastating, leading to **mass data exfiltration** and massive privacy breaches,. A classic example is the **USPS Informed Visibility API breach**, where any authenticated user could query the account details of any other user simply by using wildcard parameters or manipulating IDs, exposing 60 million records,,.

Defending against BOLA requires a "deny-by-default" posture where every single request to a resource is **explicitly authorized** against the user's ID stored in the server-side session or token,,. Modern security architectures should leverage **authorization middleware or policy engines** to enforce these checks centrally, ensuring that the application logic never trusts an ID provided by the client without verifying ownership,,.



Function-Level Authorization Failures (BFLA)

- **Privilege Escalation**
- **Definition: Broken Function Level Authorization (BFLA)** occurs when a standard user can access administrative functions or API endpoints.
- **Mechanism:** An attacker discovers hidden endpoints (e.g. POST /api/admin/createUser) or changes the HTTP method (e.g. GET to DELETE) and the server executes the action because it lacks a role check.
- **Root Cause:** Reliance on "Security by Obscurity" (hiding the button in the UI) rather than enforcing strict role checks in the backend controller.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

These speaker notes provide an in-depth analysis of **Broken Function-Level Authorization (BFLA)**, its mechanisms, and why it remains a prevalent threat in modern API security.

Definition and Distinction

Broken Function-Level Authorization (BFLA) is a critical security flaw occurring when an application or API fails to apply proper **role-based access controls (RBAC)** to its specific functions or endpoints. While **Broken Object-Level Authorization (BOLA)** involves a user accessing data records that do not belong to them, BFLA is specifically concerned with the **unauthorized execution of actions or functions**, such as a standard user gaining the ability to delete accounts or modify system settings. This leads to **vertical privilege escalation**, where a user with lower privileges exploits a bug to grant themselves or others permissions they were never intended to have.

Mechanisms of Exploitation

Threat actors exploit BFLA by identifying functionality that lacks requisite access constraints. Common mechanisms include:

Discovering Hidden Endpoints: Adveraries often use **forced browsing**, automated spidering, or **administrative API documentation** to find "hidden" endpoints intended only for developers or moderators (e.g

/api/admin/createUser).

Method Manipulation: An API may correctly restrict access to a GET request but fail to validate the user's role for other **HTTP methods** like POST, PUT, or DELETE on the same resource.

Logic Flaws: In some instances, simply changing a parameter (like admin=true or modifying the case of a character in a URL) can bypass poorly implemented authorisation checks.

Root Cause: Security by Obscurity

The primary root cause of BFLA is a reliance on "**Security by Obscurity.**"

Developers often attempt to "hide" administrative features by simply **not exposing them in the User Interface (UI)**. This creates a false sense of security, as the **server-side attack surface** remains wide open; attackers can still discover and exploit these APIs via network scanning, proxy tools like **Burp Suite**, or by reviewing source code comments.

Prevention and Mitigation

To secure APIs against BFLA, organisations must move away from UI-level hiding and implement **strict backend enforcement**. Best practices include:

Enforcing RBAC at the Controller Level: Every administrative function must explicitly verify the user's role or **claims** (such as those within a **JWT**) before processing the request.

Deny by Default: Authorisation modules should be configured to **deny access by default**, only granting it to designated, verified roles for each specific function.

Consistent Auditing: Regularly checking API endpoints against the **business logic and group structure** is essential for identifying over-privileged accounts or forgotten "zombie" endpoints.



Why Broken Access Control Dominates

- **The Hardest Logic to Genericize**
- **Logic Complexity:** Unlike injection attacks (which follow patterns), authorization flaws are unique to the business logic of every application (e.g. "Can a sub-user view the billing data of the parent account?"), making them hard to detect with automated scanners.
- **Testing Gaps:** Traditional testing focuses on whether the feature works for the happy path (the Admin), rarely verifying that the negative path (the User trying to act as Admin) fails.
- **Scale:** Modern microservices architectures fragment authorization logic across dozens of services, increasing the likelihood that one service fails to validate a user's token scope or data ownership.

Broken access control has currently risen to the top of the OWASP Top 10 list, reflecting its status as the most critical risk to web applications. The primary reason for this dominance is that authorization is arguably **the hardest logic to genericize or automate**. While traditional vulnerabilities like SQL injection or cross-site scripting (XSS) often follow predictable, static patterns that can be detected via signature-based scanners, authorization is **intimately tied to the specific business logic** of every individual application. For example, a scanner cannot inherently determine if a "sub-user" should be permitted to view the billing data of a parent account without deep context regarding the application's intended business rules.

Testing Gaps and the "Happy Path"

A significant factor in the persistence of these flaws is the **gap in traditional testing methodologies**. Most quality assurance (QA) processes focus heavily on the **"happy path"**, verifying that features function correctly for authorised roles like administrators. Conversely, **"negative testing"**, which attempts to verify that an unauthorised user (the negative path) is blocked from performing sensitive actions, is frequently overlooked or poorly implemented. Furthermore, when an authorization flaw exists, the application often processes the request successfully without returning an error code; as far as an automated tool is

concerned, the transaction was valid, even if the user had no right to see the data.

Microservices and the Challenge of Scale

Modern **microservices architectures** have further complicated this landscape by fragmenting authorization logic across a wide array of loosely coupled services. In a traditional monolithic system, there were typically only a few primary trust boundaries to protect. In a microservices environment, this attack surface can explode to **over forty different connection points** between endpoint devices, services, and backend systems. This increased complexity makes it highly likely that a single service may **fail to validate a user's token scope** or verify data ownership (BOLA), essentially "dropping the end user's context" and allowing over-privileged access. Managing these fine-grained privileges consistently across multiple providers and services remains an **exceptionally cumbersome task** for security architects.



Application vulnerabilities (OWASP Top 10 and beyond)



Injection Attacks Across Modern Stacks

- **SQL Injection (SQLi):**

- Remains one of the most dangerous flaws, allowing attackers to search, update, or delete backend database data.
- ORM Failure Modes: While Object-Relational Mapping (ORM) tools reduce risk, they fail when developers bypass built-in protections by using "raw" SQL queries or improper string concatenation, inadvertently reintroducing injection flaws.

- **NoSQL Injection:**

- Targets non-relational databases (e.g. MongoDB) by manipulating JSON-based query structures.
- Attackers inject malicious operators (e.g. `$ne` for "not equal") into JSON input to bypass authentication or extract sensitive records without needing traditional SQL syntax.

- **OS Command Injection:**

- Occurs when an application passes unsafe user input to a system shell.
- Impact: Allows attackers to execute arbitrary operating system commands, potentially leading to full server compromise or the establishment of a foothold in the target environment.

SQL Injection (SQLi) and the ORM Fallacy

SQL Injection remains one of the most prevalent and dangerous threats, consistently ranking near the top of the **OWASP Top 10**. This flaw allows attackers to bypass authentication or extract, modify, and delete data by inserting malicious SQL syntax into input variables. While the adoption of **Object-Relational Mapping (ORM)** tools has significantly reduced these risks by providing an abstracted interface, vulnerabilities are frequently reintroduced when developers prioritise performance over security. In high-performance or serverless environments, developers may bypass the ORM to execute "**raw**" **SQL queries** or use improper **string concatenation**, which directly exposes the database to injection.

The Rise of NoSQL Injection

As modern stacks increasingly utilise non-relational databases like **MongoDB**, a new class of **NoSQL Injection (NoSQLi)** has emerged. These attacks target **JSON or BSON query structures** rather than traditional SQL syntax. Attackers exploit these systems by injecting malicious operators, such as **`$ne` (not equal)** or **`$gt` (greater than)**, to manipulate query logic. For instance, submitting a payload like `{"$ne": null}` can trick a vulnerable API into returning the first valid record in a collection, effectively bypassing authentication without a password.

OS Command Injection and System Compromise

OS Command Injection is arguably the most severe injection type, as it allows an adversary to execute **arbitrary operating system commands** directly on the host server. This occurs when an application passes unsanitized user input to a system shell, often using command separators like the **semicolon (;) on Linux** or the **ampersand (&) on Windows** to chain malicious instructions to legitimate ones. A successful exploit can lead to a **full server compromise**, allowing the attacker to establish a persistent **foothold**, exfiltrate sensitive files like `/etc/passwd`, or pivot deeper into the internal network.



Cross-Site Scripting (XSS)

- **XSS Classifications:**

- Reflected (Type 2): Malicious script is immediately returned by the web application in an error message or search result.
- Stored (Persistent/Type 1): The payload is permanently stored on the server (e.g. in a comment field) and served to victims later.
- DOM-Based (Type 0): The vulnerability exists entirely in client-side code where the DOM environment is modified, often without the payload ever reaching the server.

- **Impact Beyond Pop-ups:**

- Session Theft: Exploiting access to document.cookie to hijack authenticated user sessions.
- Redirection: Forcing user browsers to navigate to malicious phishing sites.
- Account Compromise: Leveraging frameworks like BeEF (Browser Exploitation Framework) to hook browsers and execute complex attacks against the user's local network or account.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

XSS Classifications

XSS is generally categorised into three primary types based on how the malicious payload is delivered and processed:

Reflected (Type 2): This is a non-persistent attack where the malicious script is immediately returned by the web application, typically via **HTTP GET parameters** in a URL. Exploitation requires **social engineering**, such as tricking a victim into clicking a crafted link in a phishing email, which then reflects the script from the vulnerable server into the victim's browser.

Stored (Persistent/Type 1): In this **opportunistic attack**, the payload is permanently stored on the server, often in a database, comment field, or user bio, and is served to any victim who later views the affected page. This allows a single injection to impact multiple users silently without requiring specific targeting.

DOM-Based (Type 0): Unlike traditional types, the vulnerability exists entirely in **client-side code** where the Document Object Model (DOM) is modified. The payload may never reach the server; instead, local JavaScript incorrectly handles data from a "sink" (like the URL fragment after a #) and writes it directly into the page.

Impact Beyond Pop-ups

The true danger of XSS lies in its ability to manipulate the browser to perform unauthorized actions:

Session Theft: By accessing `document.cookie`, an attacker can exfiltrate sensitive **session tokens** to a remote server they control. This allows the adversary to **hijack the authenticated session** and impersonate the user without ever knowing their password.

Redirection: Attackers can use the `document.location` element to force a victim's browser to navigate to **malicious phishing sites**, effectively bypassing user training that encourages hovering over links to verify hostnames.

Account Compromise & BeEF: Advanced attackers leverage the **Browser Exploitation Framework (BeEF)** to "hook" a browser, turning it into a **zombie bot** that periodically polls the attacker's server for new commands. Once hooked, the attacker can execute complex attacks, including **port scanning internal networks**, taking screenshots, or capturing input via keystroke loggers.

To defend against these threats, organisations should implement **output encoding**, use the **HTTPOnly** flag to protect cookies from JavaScript access, and enforce a strict **Content Security Policy (CSP)** to restrict which scripts the browser is permitted to load.



Server-Side Request Forgery (SSRF)

- **Internal Service Abuse:**
 - SSRF flaws allow an attacker to coerce the application server into sending crafted requests to unintended destinations, effectively bypassing firewalls to access internal systems.
- **Cloud Metadata Exploitation:**
 - A primary target in cloud environments (e.g. AWS, Azure) is the Instance Metadata Service (IMDS).
 - Attackers use SSRF to query local link-local addresses (e.g. 169.254.169.254) to steal temporary cloud credentials and gain unauthorized access to the cloud environment.
- **Mitigation and Bypasses:**
 - Defenses: Implementing strict allowlists for outgoing traffic and disabling HTTP redirections.
 - Bypasses: Attackers frequently bypass filters using DNS rebinding, alternative IP encodings (e.g. Decimal IPs), or open redirects.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Understanding SSRF and Internal Service Abuse

Server-Side Request Forgery (SSRF) is a critical vulnerability that occurs when a web application fetches a remote resource without sufficient validation of the user-supplied destination. In this scenario, an attacker coerces the application server into acting as a proxy to send crafted network packets to unintended locations. Because the request originates from the backend server itself, it can **effectively bypass perimeter firewalls** and network segmentation to access internal services that are otherwise unreachable from the public internet. Common targets for internal abuse include administrative interfaces, local databases (such as **Elasticsearch**), or other internal REST APIs that may lack authentication because they are assumed to be "safe" within the internal network.

Cloud Metadata Exploitation

In cloud environments, a primary and devastating target for SSRF is the **Instance Metadata Service (IMDS)**. All major cloud providers, including **AWS, Azure, and GCP**, host this service at the well-known link-local IP address **169.254.169.254**. By querying this endpoint, an attacker can retrieve sensitive information about the instance, including its hostname, security groups, and, most critically, **temporary cloud credentials** associated with the instance's IAM role. These

credentials, often delivered in clear text, allow the attacker to authenticate to the cloud provider's API from their own machine and perform actions as the compromised instance, leading to **mass data exfiltration**. A famous example of this exploit is the **2019 Capital One breach**, where an attacker used SSRF via a misconfigured Web Application Firewall (WAF) to steal credentials and access over 700 S3 buckets.

Mitigation and Defensive Strategies

Defending against SSRF requires a multi-layered approach:

Allowlisting: The most secure method is to implement strict **allowlists** that restrict the application to fetching resources only from a known set of trusted hostnames or IP addresses.

Disabling Redirections: Applications should disable the automatic following of HTTP redirections to prevent attackers from using a legitimate-looking URL to redirect the server to an internal address.

IMDSv2 Enforcement: On AWS, transitioning to **IMDSv2** is essential. Unlike version 1, version 2 requires a session-oriented **PUT request** to acquire a token before metadata can be accessed, a flow that is significantly harder to execute via standard SSRF.

Common Bypass Techniques

Adversaries frequently employ sophisticated bypasses to circumvent basic filters. **DNS Rebinding** is a particularly potent technique where an attacker configures a malicious DNS server to initially return a valid external IP to pass validation checks, only to switch the resolution to an **internal IP address** (like 127.0.0.1) when the actual HTTP request is made. Additionally, attackers may use **alternative encodings**, such as URL, double-URL, or Unicode encoding, to bypass filters that only look for literal strings like "169.254.169.254" or "localhost". Finally, **open redirects** on trusted domains can be chained with SSRF to bounce a request from an allowed site to a restricted internal resource.



Insecure Deserialization

- **The Mechanism:**
 - **Serialization** converts objects into a stream of bytes for storage or transmission; deserialization reverses this process.
 - Vulnerabilities arise when an application blindly deserializes untrusted data without validation.
- **Object Manipulation:**
 - Attackers tamper with serialized objects (e.g. Java objects, PHP serialization, Python pickles) to manipulate application logic.
- **Critical Impact:**
 - **Remote Code Execution (RCE):** By manipulating object properties or leveraging "gadget chains" in available libraries, attackers can force the application to execute arbitrary code upon deserialization.
 - **Privilege Escalation:** Modifying serialized user objects (e.g. cookies) to gain administrative privileges.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Mechanism: Serialization and Deserialization

Serialization is the common programming technique used to convert complex objects or variables in memory into a **stream of bytes or a string** suitable for storage or transmission,,. **Deserialization** is the inverse process, where the application reads this byte stream to **rebuild the identical object** in the current process's memory,,. While traditionally associated with binary formats in Java and Ruby (often termed **marshalling**), it also applies to hybrid formats like **PHP serialization** and **Python pickling**,,. Modern applications frequently use **JSON or XML** for these tasks, though they too can be vulnerable if integrated with libraries that support automated object instantiation,,.

The Root of Vulnerability

Security failures arise when an application **blindly deserializes untrusted data** from a client without proper validation,,. Because the deserializer often follows the instructions embedded within the serialized stream to determine which classes to instantiate, it effectively grants the data **control over the application's internal state**,. This "magic" of automated construction occurs **before the application logic** has an opportunity to inspect or verify the resulting object,.

Object Manipulation and Privilege Escalation

Attackers exploit this by **tampering with serialized objects**, often stored in cookies, hidden form fields, or URL parameters, to manipulate application logic,,. For example, in a PHP application, an attacker might decode a serialized user object, change a **role property** from a standard user value to an administrative one, and re-encode it,. Upon deserialization, the application accepts these **modified properties as authentic**, leading to **vertical privilege escalation**,,.

Gadget Chains and Remote Code Execution (RCE)

The most critical impact is **Remote Code Execution (RCE)**, achieved through **Property-Oriented Programming**,. Attackers identify and chain together **"gadget" classes**, legitimate classes already available in the application's classpath or libraries (like **CommonsCollections** in Java), that perform dangerous actions in their constructors or during the unmarshalling process,,. By carefully crafting a **gadget chain** within the serialized payload, an attacker can force the system to instantiate classes that execute **arbitrary shell commands** or write malicious files to disk,,,. Tools like **ysoserial** automate the generation of these complex chains for Java environments, while similar techniques exploit the **__reduce__** method in Python's pickle module to achieve code execution,,.



XML External Entity (XXE) Attacks

- **The Vulnerability:**
 - XXE flaws occur in older or poorly configured XML processors that allow the specification of an external entity (a URI that is dereferenced during processing).
- **Unsafe Parser Defaults:**
 - Many XML parsers enable external entity resolution by default, even if the feature is not required by the application.
- **Attack Vectors:**
 - **File Disclosure:** Attackers can define entities that reference local files (e.g. /etc/passwd), causing the server to return the file contents in the response.
 - **Internal Scanning:** The parser can be directed to connect to internal ports and systems, effectively acting as a proxy for port scanning behind the firewall.
 - **Denial of Service:** Recursive entity expansion (e.g. "Billion Laughs" attack) can exhaust server memory.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The Nature of the XXE Vulnerability

XML External Entity (XXE) injection is a specific class of **injection attack** that targets applications parsing XML input. The vulnerability arises when an **older or poorly configured XML processor** is used to handle untrusted XML data. These processors allow for the specification of an **external entity**, which is essentially a URI that is **dereferenced and evaluated** by the parser during processing. By hijacking the parser, an attacker can force the system to execute commands unintended by the developer.

Unsafe Parser Defaults

A critical contributing factor to this flaw is **unsafe default configurations**. Many older XML processors enable **external entity resolution by default**, even if the application does not require this feature to function. While newer parsers often disable **Document Type Definition (DTD)** parsing by default to improve security, the risk remains if developers manually enable these legacy features for backward compatibility.

Primary Attack Vectors

Adversaries leverage XXE through several high-impact attack vectors:

File Disclosure: Attackers can define malicious entities that point to **local files on the server's filesystem**, such as /etc/passwd. When the web service

resolves these entities, it loads the sensitive file contents directly into the application's response, exposing them to the requester.

Internal Scanning and SSRF: XXE allows a penetration tester to effectively turn the XML parser into a **proxy**. This enables **Server-Side Request Forgery (SSRF)**, where the parser is directed to make requests to internal services or perform **port scanning** on systems located behind the organization's firewall that are not directly accessible from the internet.

Denial of Service (DoS): An attacker can exhaust server resources by sending an **oversized XML payload** or using recursive entity expansion, commonly known as a "**Billion Laughs**" attack. This forces the parser to recursively resolve entities, leading to **exponential data expansion** that can quickly consume all available CPU and memory, causing the system to crash.

Mitigation

The most effective way to prevent XXE attacks is to **completely disable the use of DTDs** and external entity references within the XML parser configuration. Defenders should refer to language-specific security guidelines, such as the **OWASP XXE Prevention Cheat Sheet**, to ensure parsers are correctly hardened.



Security Misconfiguration

- **Default Credentials:**
 - Attackers routinely scan for administrative interfaces utilizing default usernames and passwords (e.g Tomcat, Jenkins, database accounts) to gain immediate access.
- **Exposed Admin Interfaces:**
 - Leaving management panels, cloud storage buckets, or diagnostic endpoints accessible to the public internet drastically increases the attack surface.
- **Verbose Errors:**
 - Applications configured to display detailed error messages (e.g stack traces) leak sensitive information about the backend architecture, installed libraries, and file paths.
- **Patch Gaps:**
 - Using components with known vulnerabilities or failing to apply security patches allows attackers to exploit publicly known CVEs.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Default Credentials

Adversaries frequently target the **"low-hanging fruit"** of default or guessable credentials. It is unbelievably common to find critical devices, such as **Tomcat**, **Jenkins**, and out-of-band management platforms like **Dell iDRAC**, still active with their factory-set administrator passwords (e.g admin/admin or root/calvin). Attackers use automated tools to scan for these authentication prompts, knowing that if they gain access, they often inherit **unrestricted administrative control** over the host operating system or the entire network infrastructure.

Exposed Admin Interfaces

Leaving management panels or diagnostic endpoints accessible to the public internet drastically expands an organisation's **attack surface**. Tools like **Shodan** constantly index internet-connected devices, revealing roughly 139,000 exposed vSphere authentication addresses and millions of systems with **Remote Desktop Protocol (RDP)** or **SSH** open to the entire world. Insecure defaults in cloud environments often lead to virtual machines (VMs) being spun up with wide-open administrative ports (0.0.0.0/0), inviting global scrutiny and automated brute-force attacks. Similarly, **cloud storage buckets** (e.g AWS S3 or Azure Blobs) that are accidentally configured for public read access remain a leading cause of massive data exfiltration.

Verbose Errors

Detailed error messages, such as **stack traces** or database syntax errors, act as a "**treasure map**" for threat actors. While intended to aid developers in debugging, these messages often leak sensitive information regarding the **backend architecture**, file paths, library versions, and even database schema details. For instance, a .NET Core stack trace might confirm the use of a specific SQL database, allowing an attacker to tailor their **injection payloads** to that exact technology.

Patch Gaps

Failing to apply security patches allows attackers to exploit publicly known **Common Vulnerabilities and Exposures (CVEs)**. Administrators often hesitate to update **firmware or legacy systems** out of fear that the update might disrupt mission-critical functions; however, adversaries rely on this "easy prey". High-severity vulnerabilities, such as those found in **Apache web servers**, can have base scores as high as **9.8**, yet millions of systems remain unpatched long after a fix is available. Maintaining an up-to-date **software inventory** and a rapid patch management cycle is essential to closing these windows of exposure.



API-specific security risks and failure modes



Why APIs Fail Differently

- The "Headless" Attack Surface
- **No UI Constraints:** Unlike browser-based apps, APIs lack a user interface to guide or constrain the user's path. Attackers interact directly with the backend logic using raw HTTP requests (scripts, curl, Postman).
- **Structured Data:** APIs typically consume and emit structured data (JSON, XML) rather than HTML. This machine-readable format makes it easier for attackers to automate attacks, scrape data, and parse internal object structures compared to scraping web pages.
- **Direct Database Access:** APIs often serve as a thin layer over the database. Vulnerabilities like Mass Assignment or Excessive Data Exposure occur because developers map database objects directly to API endpoints without filtering, exposing internal schemas and sensitive fields that a UI would normally hide.
- **Automation Friendly:** The predictable nature of APIs makes them ideal targets for automated bot attacks, credential stuffing, and scraping.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction Application Programming Interfaces (APIs) represent a unique challenge because their architecture is fundamentally different from traditional web applications. They are described as "**headless**" because they lack a graphical user interface (GUI) to guide or restrict user behaviour,. While a waiter in a restaurant serves as an intermediary following a set menu, an API allows other software components to talk directly to the "chef" (the backend logic),.

No UI Constraints Unlike browser-based apps that use buttons and menus to lead a user down a "**happy path**," APIs allow attackers to bypass these frontend restrictions entirely,. Adversaries interact directly with the backend using raw HTTP requests via scripts, **curl**, or tools like **Postman** and **Burp Suite**,,. This allows them to be much more creative and malicious with the requests they send.

Structured Data APIs typically consume and emit **structured, machine-readable data** such as **JSON** or **XML**,. While HTML is designed for human consumption, these formats are specifically designed for ease of computer parsing,,. This makes it significantly easier for threat actors to use automated tools to scrape data, analyze internal object structures, and identify potential weaknesses compared to the more complex task of parsing web pages,,.

Direct Database Access Modern APIs often function as a **thin layer over a**

database, with developers using **Object-Relational Mappers (ORMs)** to map database objects directly to API endpoints,,. This convenience leads to critical flaws:

Excessive Data Exposure: The API returns an entire data object from the database, relying on the client to filter what is displayed. This allows attackers to see sensitive fields that the UI normally hides,,.

Mass Assignment: The backend blindly assigns user-supplied input to internal objects. Attackers can guess sensitive field names (like `is_admin`) and overwrite them to elevate their privileges,,.

Automation Friendly The predictable, structured nature of APIs makes them ideal targets for **automated bot attacks**,. Because they are designed for high-speed machine-to-machine communication, attackers can launch **credential stuffing**, large-scale **scraping**, and **denial-of-service (DoS)** attacks much more efficiently than against standard web applications,,. Scaling an attack to thousands of endpoints can be done at a very low cost to the adversary.



Stateless APIs & Token-Centric Risks

- **The Consequences of "Sessionless" Architecture**
- **Statelessness:** REST APIs are designed to be stateless; the server does not store session state between requests. This forces the client to hold the state, usually in the form of a token (e.g. JWT).
- **Token Exposure:** Because the client holds the token, it must be stored securely (e.g. HttpOnly cookies vs. LocalStorage). If a token is stolen (via XSS or interception), the attacker gains persistent access until the token expires.
- **Revocation Difficulties:** Unlike server-side sessions, stateless tokens (like JWTs) are difficult to revoke instantly. A stolen token often remains valid until its expiration time, even if the user changes their password or is locked out, unless complex blocklisting or short-lived token flows are implemented.
- **Information Leakage:** JWTs are often unencrypted (encoded only). Sensitive claims (user roles, IDs, emails) put into a token can be read by anyone who intercepts it, leading to information disclosure.

The Nature of Statelessness

REST APIs are designed to be **stateless**, meaning that the server does not store session state or "remember" the client between independent requests. Each individual request must contain all the information necessary for the server to process it. To simulate a persistent session for the user, applications use **tokens**, most commonly **JSON Web Tokens (JWTs)**, which the client must hold and present with every request.

Token Exposure and Storage Risks

Because the client is responsible for managing the state, the method of **token storage** is a critical security decision.

LocalStorage: Storing tokens in **LocalStorage** is highly risky because this storage is directly accessible via JavaScript. This makes the token vulnerable to exfiltration through **Cross-Site Scripting (XSS)** attacks, where a malicious script can silently send the token to an attacker-controlled server.

HttpOnly Cookies: To mitigate XSS theft, tokens can be stored in cookies marked with the **HttpOnly** flag, which prevents JavaScript from accessing them. However, this shift introduces a different risk: **Cross-Site Request Forgery (CSRF)**, where a malicious site tricks the browser into automatically attaching the cookie to an unauthorized request.

Interception: If tokens are sent over an insecure channel, an attacker can use a **machine-in-the-middle (MITM)** attack to sniff the token and instantly impersonate the user.

Revocation Difficulties

The "Achilles' heel" of stateless tokens is the difficulty of **instant revocation**.

Unlike traditional server-side sessions, which can be deleted from a database to end a session immediately, self-contained JWTs are **valid until they expire**.

The Persistence of Stolen Tokens: A stolen JWT often remains valid even if a user changes their password, as the token's validity is usually checked locally by the API using a public key rather than by querying the central authority.

Mitigation Strategies: To handle revocation, defenders must implement complex workarounds like **blocklists** (storing revoked token IDs until they expire) or **allowlists** (requiring a database check for every request). Alternatively, using **short-lived access tokens** (e.g 5–15 minutes) paired with **refresh tokens** minimizes the window of opportunity for an attacker while maintaining a good user experience.

Information Leakage

A major misconception is that JWTs are "secure" out of the box. While digital signatures provide **integrity** (ensuring the token hasn't been tampered with), they provide **zero confidentiality**.

Ease of Decoding: JWTs are typically only **base64url encoded**, not encrypted. Anyone who intercepts a token, whether a malicious script, a proxy admin, or a rogue employee, can decode it using public tools like jwt.io to read sensitive claims, such as **user roles, internal IDs, and emails**.

Encryption (JWE): To prevent this information disclosure, developers must specifically implement **JSON Web Encryption (JWE)** to ensure that only the intended recipient can read the token's contents.



REST API Risks

- **Resource Exposure and Method Misuse**
- **Over-Permissive Endpoints:** Developers often expose administrative or debug endpoints (e.g. /admin, /test) without proper access controls, assuming they are "hidden" because they are not documented or linked in the UI.
- **Verb Misuse (BFLA):** Broken Function Level Authorization often manifests when different HTTP methods (verbs) have different access controls. A user might be denied POST access to a resource but allowed DELETE or PUT access because the developer forgot to secure all verbs for that endpoint.
- **Predictable Resource IDs:** REST APIs often use sequential integers for IDs (e.g. /users/101). This predictability allows attackers to enumerate resources by simply incrementing the ID (Resource Enumeration), facilitating BOLA/IDOR attacks.
- **Metadata Exposure:** Endpoints may inadvertently leak metadata (stack traces, server versions) in error messages (e.g. 500 Internal Server Error), helping attackers tailor their exploits.

REST APIs have become the standard for modern application connectivity, accounting for the vast majority of web traffic, yet their architecture introduces specific vulnerabilities that are often overlooked.

Over-Permissive Endpoints

Developers frequently deploy "shadow APIs" or debug endpoints (e.g. /admin, /test) under the assumption that they are safe because they are not documented or linked in the user interface. This reliance on **security by obscurity** is highly ineffective, as threat actors use automated discovery tools like **Kiterunner** or **Nmap** to identify these "hidden" routes through scanning and fuzzing. If these endpoints lack strict backend access controls, their mere discovery often grants attackers full access to sensitive functionality.

Verb Misuse (BFLA)

Broken Function Level Authorization (BFLA) often manifests when developers fail to apply consistent security checks across all **HTTP methods (verbs)** for a single endpoint. For example, a system might correctly restrict a GET request to authorized users but fail to enforce those same role-based checks for POST, PUT, or DELETE requests on the same resource. By simply manipulating the request method, an attacker can perform unauthorized actions, such as deleting records or modifying system settings.

Predictable Resource IDs

Many REST APIs utilize **sequential integers** for resource identifiers (e.g. /users/101), creating a predictable pattern that allows for **resource enumeration**. This predictability is a primary facilitator of **Broken Object Level Authorization (BOLA)**, also known as **IDOR**, where an attacker simply increments an ID to access data belonging to another user. While moving to randomized identifiers like **UUIDs** increases complexity for the attacker, true security still requires the API to explicitly verify that the authenticated user is authorized to access the specific object ID requested.

Metadata Exposure

Endpoints frequently leak sensitive system metadata through **verbose error messages**, such as stack traces or server version headers (e.g. X-Powered-By). These details often surface in **500 Internal Server Error** responses, providing attackers with a "treasure map" of the backend infrastructure, including the specific database engine or programming framework in use. This intelligence allows threat actors to refine their payloads and target known vulnerabilities within those specific technologies.



GraphQL Risks

- **Complexity and Introspection**
- **Introspection Abuse:** GraphQL includes an Introspection feature that allows clients to query the API for its entire schema. If left enabled in production, this gives attackers a complete map of all types, queries, and mutations, including hidden or internal fields.
- **Query Depth & Complexity:** Because clients define the query structure, an attacker can craft deeply nested queries (e.g. "Authors who wrote Books which have Authors who wrote Books...") to exhaust server CPU and memory resources, causing a Denial of Service (DoS).
- **Batching Attacks:** GraphQL supports batching multiple queries into a single HTTP request. Attackers can exploit this to bypass rate limits or perform brute-force attacks (e.g. trying 100 different passwords in one HTTP request), as the WAF/Gateway counts it as a single request.
- **Excessive Data Exposure:** GraphQL returns exactly what the client asks for. If the backend schema allows access to sensitive fields (e.g. PII, internal IDs) and relies on the frontend to just "not ask" for them, an attacker can simply modify the query to retrieve all available data.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to GraphQL Security GraphQL is a powerful query language that allows clients to define the exact structure of the data they require from a server. While this provides significant efficiency by reducing "over-fetching," the flexibility it grants to the client shifts the **computational burden to the server**, creating unique security challenges not typically found in traditional REST architectures.

Introspection Abuse

A primary risk is the **Introspection feature**, which allows a client to query the API for information about its own schema. When left enabled in production, introspection provides an adversary with a **complete map of all types, fields, queries, and mutations**. This acts as a "gold mine" for information disclosure, essentially providing a blueprint of the entire API that would otherwise require significant effort to discover.

Query Depth & Complexity

Because the client defines the query structure, an attacker can craft **deeply nested or recursive queries** (e.g. querying "Authors" who wrote "Books" which were edited by "Authors"). These complex requests can lead to **resource exhaustion**, as the server attempts to process recursive objects that consume excessive CPU and memory, resulting in a **Denial of Service (DoS)**. Mitigation

strategies often involve setting a **maximum query depth** or execution time limit to prevent such exhaustion.

Batching Attacks

GraphQL supports **batching**, allowing multiple queries or mutations to be sent within a **single HTTP POST request**. Attackers exploit this to bypass traditional rate-limiting mechanisms or Web Application Firewalls (WAFs), as the security layer often counts the single request rather than the **hundreds of individual queries** (such as password guesses) contained within the body. This makes GraphQL a prime target for automated **brute-force attacks**.

Excessive Data Exposure

While GraphQL is designed to return exactly what the client asks for, this relies on the assumption that the backend schema only contains data the user is authorised to see. If the schema exposes sensitive fields, such as **PII or internal database IDs**, an attacker can simply modify their query to retrieve all available fields. This vulnerability occurs when developers fail to implement **fine-grained access controls** at the resolver level, relying instead on the frontend to just "not ask" for sensitive data.



gRPC Security Challenges

- **Binary Blind Spots**
- **Reduced Visibility:** gRPC uses Protocol Buffers (Protobuf), a binary serialization format. Traditional WAFs and security scanners designed for text-based HTTP/1.1 (JSON/XML) may not be able to parse or inspect gRPC payloads for malicious content.
- **Schema Dependency:** To interact with or inspect gRPC traffic, one typically needs the .proto definition files (stubs). Without these, the traffic is opaque binary blobs, making it harder for security teams to audit traffic or for penetration testers to fuzz endpoints.
- **HTTP/2 Complexity:** gRPC relies on HTTP/2, which introduces complexities like multiplexing. Security tools that do not fully support HTTP/2 inspection may miss attacks tunneled through these connections.
- **Blind Spots:** The combination of binary encoding and complex transport means standard injection attacks or data leakage might pass undetected through perimeter defenses.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to gRPC and Protobuf gRPC is a modern **Remote Procedure Call (RPC)** framework that typically employs **Protocol Buffers (Protobuf)** as its default **binary serialization format**. Unlike traditional REST APIs that use text-based formats like JSON or XML, gRPC is designed for high efficiency and machine-to-machine communication, meaning the data is **intended purely for machine reading**.

The Challenge of Reduced Visibility

The shift to binary creates a significant **visibility challenge** for security infrastructure. Traditional **Web Application Firewalls (WAFs)** and security scanners are often highly optimised for text-based HTTP/1.1 traffic. Because these legacy tools may not be able to **parse or inspect** the compressed binary payloads of gRPC, they often fail to identify malicious content hidden within the stream.

Schema Dependency and Opaque Traffic

A critical aspect of gRPC is its **schema dependency**. To effectively interact with, audit, or decode gRPC traffic, an analyst typically requires the **.proto definition files**, which function as **stubs** for the client and server. Without these schemas, the network traffic remains **opaque binary blobs**. This dependency makes it exceptionally difficult for security teams to perform **fuzzing** on endpoints or for

forensic investigators to identify anomalous data patterns during an incident.

HTTP/2 Complexity and Blind Spots

gRPC's reliance on **HTTP/2** introduces further technical hurdles, specifically through **multiplexing**. Multiplexing allows for multiple concurrent **request and response streams** over a single TCP connection, which can **obscure the volume** and nature of individual exchanges. If perimeter defences, such as an **Intrusion Detection System (IDS)**, are not fully **HTTP/2-aware**, they may miss sophisticated attacks tunneled through these complex connections.

Ultimately, this combination of **binary encoding** and complex transport mechanisms creates dangerous **blind spots**. Standard threats, such as **injection attacks** or **unauthorised data leakage**, can pass through unhardened perimeter defences entirely undetected if the tools cannot **normalize or decode** the underlying binary application communication.



OWASP API Top 10 (Data & Auth)

- **Authorization and Data Leakage**
- **Broken Object Level Authorization (BOLA/IDOR):** The #1 API vulnerability. Attackers manipulate the ID in an API call (e.g. /orders/1234) to access resources belonging to other users. APIs often verify authentication (who you are) but fail to verify authorization (do you own this specific resource?).
- **Mass Assignment:** Modern frameworks allow binding client input directly to internal code objects. Attackers can inject extra fields (e.g. "is_admin": true, "balance": 1000000) into a request to overwrite internal variables that developers did not explicitly protect.
- **Excessive Data Exposure:** APIs often return full object representations (e.g. a full User object with address and phone number) to the client, relying on the frontend app to filter out sensitive data. Attackers intercept the raw response to view the private data.
- **Broken Authentication:** Weaknesses in session management, such as allowing weak passwords, failing to validate token signatures (e.g. accepting alg: none in JWTs), or permitting credential stuffing/brute-force attacks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Broken Object Level Authorization (BOLA), also historically known as **Insecure Direct Object Reference (IDOR)**, is currently ranked as the most common and devastating vulnerability in APIs. This flaw occurs when an API provider fails to implement sufficient security to enforce authorization at the resource level, allowing users to access data that does not belong to them. Attackers exploit this by **manipulating object identifiers** (such as IDs in a URL like /orders/1234) using simple scripts to probe for other valid resources. While these attackers may have valid authentication credentials, the system fails to verify if the specific user has the right to interact with the requested object, potentially leading to **full account takeovers** or massive data breaches.

Data Integrity and Mass Assignment

Mass Assignment vulnerabilities arise from the convenience of modern development frameworks that allow **"autobinding"** user input directly to internal code objects or database models. If developers do not explicitly use **whitelists** to restrict which fields can be updated, an attacker can inject extra parameters into a request, such as is_admin: true or balance: 1000000, to **overwrite internal variables**. This essentially grants the adversary write access to sensitive properties that were never intended to be exposed to the end user.

Information Leakage via Excessive Data Exposure

Excessive Data Exposure occurs when an API endpoint responds with a **full object representation**, relying on the client-side application to filter what is actually displayed to the user. This often happens because developers find it more efficient to map database objects directly to JSON responses using methods like `to_json()` rather than creating specific data transfer objects. Threat actors can use **interception proxies** to view the raw, unfiltered response, exposing sensitive private data like home addresses, phone numbers, or even plain-text passwords that the UI would normally hide.

Identity and Broken Authentication

Broken Authentication involves weaknesses in the mechanisms used to verify a user's identity, making it a favorite target for attackers. Vulnerabilities include permitting **credential stuffing** and brute-force attacks, as well as more technical flaws like failing to **validate JSON Web Token (JWT) signatures**. A notorious example is the **"none" algorithm attack**, where a vulnerable library is tricked into accepting an unsigned token simply by changing the `alg` field in the JWT header. Ultimately, compromising authentication undermines the entire security of the API, as the system can no longer reliably identify who is making requests.



OWASP API Top 10 (Config & Inventory)

- **Resource Management and Third-Party Risks**
- **Unrestricted Resource Consumption:** Missing rate limits or quotas allow attackers to exhaust system resources (CPU, memory, storage) or drive up cloud costs ("Denial of Wallet"). This includes sending large payloads or triggering expensive database queries.
- **Improper Inventory Management:** Organizations lose track of their APIs, leading to Shadow APIs (undocumented APIs) and Zombie APIs (deprecated/outdated versions). These unmanaged endpoints often lack security patches or monitoring.
- **Unsafe Consumption of Third-Party APIs:** APIs often implicitly trust data received from third-party integrations (e.g. payment providers, mapping services). If a third-party is compromised, it can inject malicious data (SQLi, scripts) into the consuming system, bypassing standard input validation.
- **Security Misconfiguration:** Default configurations, verbose error messages (leaking stack traces), and missing security headers (CORS, CSP) provide attackers with easy reconnaissance and exploitation paths

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Unrestricted Resource Consumption occurs when an API fails to set or enforce limits on the system resources required to satisfy requests, such as **network bandwidth, CPU, memory, and storage**. Without proper execution timeouts or memory allocation caps, attackers can trigger resource exhaustion, leading to a **Denial of Service (DoS)** for legitimate users. In cloud-native environments, this flaw can manifest as a "**Denial of Wallet**" attack; as the infrastructure automatically scales to meet the malicious demand, the organization is hit with massively inflated and potentially bankrupting cloud costs.

Improper Inventory Management (API9:2023)

Organizations frequently struggle with **API sprawl**, leading to a lack of visibility into their total attack surface. This results in two primary risks:

Shadow APIs: Undocumented endpoints or APIs created ad hoc without security oversight or quality checks.

Zombie APIs: Deprecated or outdated versions of an API that were forgotten and never properly retired, often running without the latest security patches. A proper **API catalog or inventory** is essential because security teams cannot protect assets they do not know exist.

Unsafe Consumption of Third-Party APIs (API10:2023)

Modern applications are increasingly interconnected, yet developers often

implicitly trust data received from third-party integrations, such as payment processors or identity providers. This category highlights that we are directly exposed to the **threat models of the third parties** we consume. If a third-party service is compromised, it can be used to inject malicious payloads, such as **SQL injection (SQLi)** or cross-site scripts, into our internal systems, bypassing standard validation because the data is assumed to be "safe".

Security Misconfiguration (API8:2023)

Security Misconfigurations are pervasive flaws resulting from insecure default settings, incomplete ad hoc configurations, or open cloud storage. These gaps provide attackers with a **"gold mine" for reconnaissance**. For instance, **verbose error messages** leaking stack traces can reveal sensitive information about the backend architecture, while missing security headers (like **CORS or CSP**) or informative headers (like X-Powered-By) can tip off an attacker to the specific versions of software and libraries in use. Organizations must implement **automated hardening procedures** and smoke tests to detect these misconfigurations before deployment.



Business Logic Abuse

- **Attacks That Bypass Technical Controls**
- **Workflow Manipulation:** Attackers analyze API workflows to find logical loopholes. For example, skipping a payment step in a checkout flow or reusing a one-time coupon code multiple times.
- **Scraping:** Automated bots use APIs to extract large volumes of proprietary data (e.g. pricing, user directories) faster than a human could, often utilizing endpoints that return bulk data (e.g. /users?limit=10000).
- **Scalping:** "Scalper bots" use APIs to instantly purchase limited-availability items (concert tickets, sneakers) the moment they go on sale, denying access to legitimate users and reselling for profit. This abuses the intended business logic without technically "hacking" the system.
- **Automation Attacks:** Credential stuffing and account takeover attacks leverage APIs because they provide immediate feedback (success/fail) without the overhead of loading UI assets, allowing for high-speed automated attacks.

Introduction Business logic abuse involves utilizing an application's intended features in an unexpected or unintended way to achieve a malicious outcome. Unlike traditional exploits that target coding errors like buffer overflows, business logic attacks exploit **logical loopholes** or design flaws within the application's legitimate functionality. These flaws are particularly pernicious because they often go undetected by automated vulnerability scanners, which lack the **contextual understanding** of how a specific business process is supposed to function.

Workflow Manipulation Attackers often analyze API workflows to identify steps that can be reordered, bypassed, or manipulated. Because APIs are "**headless**", meaning they lack a user interface (UI) to constrain the user's path, an attacker can interact directly with the backend logic via raw HTTP requests. This allows them to find loopholes, such as **skipping a payment step** in a checkout sequence or reusing a one-time discount code multiple times. Developers can mitigate this by using the **Arazzo Specification** to formally document and technically enforce the required sequence of API calls.

Scraping APIs are ideal for **scraping** because they emit **structured, machine-readable data** (JSON or XML) rather than formatted HTML. Automated bots can extract massive volumes of proprietary data, such as user directories or pricing

models, much faster than a human could. A notable real-world example occurred in 2024 when a threat actor scraped and leaked the personal details of over **15 million Trello users** by exploiting an unauthenticated endpoint.

Scalping "Scalper bots" exploit business flows to instantly purchase limited-availability items the moment they are released. These bots poll APIs every few seconds and execute transactions in **milliseconds**, effectively denying access to legitimate human users who might take several seconds to navigate the UI. This abuses the business intent of fair distribution without technically breaking any security protocols.

Automation Attacks APIs are highly susceptible to **automation attacks**, such as **credential stuffing** and account takeovers, because they provide immediate success/fail feedback without the overhead of loading graphical assets.

Attackers can even leverage cloud services like the **AWS API Gateway** to act as a proxy, rotating through thousands of IP addresses to bypass **Smart Lockout** mechanisms and remain anonymous during high-speed attacks.



Cloud-native and infrastructure security context



Cloud Shared Responsibility Model

- **The Responsibility Shift:** Security is a shared mandate, but the line drawn between the Cloud Service Provider (CSP) and the customer shifts depending on the service model.
- **Infrastructure as a Service (IaaS):** The customer retains the most responsibility, managing the operating system, middleware, runtime, data, and applications. The provider secures the physical virtualization, servers, storage, and networking hardware.
- **Platform as a Service (PaaS):** The provider takes ownership of the operating system and runtime environment (e.g. database patching). The customer focuses on securing their specific application code and data configurations.
- **Software as a Service (SaaS):** The provider manages the entire stack (application, runtime, OS). The customer is responsible primarily for data governance, user access management, and configuration settings.

The Fundamental Responsibility Shift The Shared Responsibility Model (SRM) is the foundational framework defining the security and compliance obligations between the **Cloud Service Provider (CSP)** and its customers. A critical industry pitfall is the "common misconception" that the provider handles all security; in reality, while an organisation can outsource its data and infrastructure management, it can never outsource its **legal liability**. As a rule of thumb, if a customer has the ability to "break" a component, they generally have the responsibility for securing it.

Infrastructure as a Service (IaaS) In an IaaS model, the provider secures the "fabric", the physical data centres, hardware, and the **hypervisor**. The customer retains the vast majority of the responsibility, functioning similarly to an on-premises environment. This includes the **guest operating system**, which must be manually patched, hardened, and updated, as VM templates are rarely current when first deployed. Customers also manage middleware, runtime, and the **virtual network** through constructs like Security Groups and NACLs.

Platform as a Service (PaaS) PaaS represents a shift toward higher-level abstraction where the provider manages the **operating system, middleware, and runtime** (e.g. database patching or application servers). This allows development teams to focus almost exclusively on their unique **application**

code. While the provider manages the underlying platform, the customer remains responsible for securing the code itself and ensuring appropriate **environmental settings** and data configurations are applied.

Software as a Service (SaaS) Under the SaaS model, the provider manages the **entire technology stack**, including the application code. However, the customer is "not entirely off the hook". They remain the **data owner** and are primarily responsible for **identity and access management**, ensuring only authorised users can access sensitive data. The customer must also meticulously manage **configuration settings**, such as sharing permissions and password complexity, as misconfigurations here are a leading cause of cloud data breaches.



Microservices Security Architecture

- **Service-to-Service Authentication:** In microservices, implicit trust is a vulnerability. Architecture must evolve to Zero Trust, requiring services to explicitly authenticate one another, often using Mutual TLS (mTLS) or PKI certificates, rather than relying on network location.
- **Traffic Directions:**
 - **North-South Traffic:** Communication entering or leaving the cluster (client to service). This is typically managed and secured by API Gateways or Ingress controllers.
 - **East-West Traffic:** Communication moving laterally between services within the cluster. Legacy networks often trust this traffic by default, allowing attackers to pivot easily once inside. Micro-segmentation is required to inspect and restrict these internal flows.

The Shift to Zero Trust in Microservices

Traditional security models relied on a "**castle-and-moat**" approach, where resources inside the network were **implicitly trusted** based on their physical or network location. In a **microservices architecture**, where applications are broken into loosely coupled, independently deployable units, this **implicit trust is a critical vulnerability**. Attackers who breach the perimeter can often roam freely because internal traffic is presumed innocuous. Modern architecture must evolve to **Zero Trust**, which operates on the principle of "**never trust, always verify**," assuming the network is already hostile.

Service-to-Service Authentication

In this model, **network location is no longer sufficient** for deciding trust. Instead, services must **explicitly authenticate** one another using strong cryptographic identities. This is typically achieved through **Mutual TLS (mTLS)**, where both the client and the server present **PKI certificates** to verify their identities before a connection is established. This process eliminates **machine-in-the-middle (MITM)** attacks and prevents **spoofing**, as trust is established through a central **Certificate Authority (CA)**. Frameworks like **SPIFFE** are often used to provide a standard way to issue these verifiable identity documents in dynamic environments.

North-South Traffic: Perimeter Security

North-South traffic refers to communication entering or leaving the cluster, such as a request from an external client to a service. This traffic is typically managed by **API Gateways** or **Ingress controllers**.

API Gateways act as a centralized entry point (choke point), enforcing security controls like **authentication, rate-limiting, and protocol translation** before traffic reaches the backend.

Ingress controllers specifically manage HTTP/HTTPS access from outside the cluster to internal Kubernetes services, often handling **TLS termination** to offload computationally expensive decryption from the application.

East-West Traffic: Micro-segmentation

East-West traffic describes lateral communication between services within the cluster. Legacy networks often leave this traffic unprotected, allowing attackers to **pivot laterally** to reach sensitive data stores once they gain an initial foothold. To mitigate this, **micro-segmentation** is required to divide the network into small, isolated zones. This limits the **attack surface** and ensures that a compromised service cannot easily communicate with unrelated workloads. These internal flows are often governed by a **Service Mesh**, which uses sidecar proxies to enforce fine-grained **authorization policies** and ensure all internal traffic is encrypted.



Container and Kubernetes Security

- **Image Scanning:** Security starts in the build pipeline. Container images must be scanned for Common Vulnerabilities and Exposures (CVEs) and configuration flaws (e.g. embedded secrets) before they are deployed to a registry.
- **Orchestration Controls (Kubernetes):**
 - **API Exposure:** The Kubernetes API server is the control plane for the cluster. Leaving it publicly exposed or allowing access to dashboards without strict authentication can lead to full cluster compromise.
 - **RBAC (Role-Based Access Control):** Granular permissions are essential. Policies should limit what actions a specific pod or user can perform (e.g. preventing a web server pod from listing all cluster secrets).
- **Runtime & Pod Security:**
 - **Boundaries:** Security contexts should be applied to prevent containers from running as the "root" user, reducing the impact of a container breakout.
 - **Runtime Protection:** Tools must monitor for anomalous behavior (e.g. a reverse shell spawn) in real-time, as static scanning cannot catch runtime exploits.

Image Scanning: Securing the Supply Chain

Security must be integrated directly into the **DevOps build pipeline**. Container images often serve as the "master" copy of an environment; therefore, any vulnerability or **embedded secret** (such as hardcoded API keys) present in the image will be replicated in every running instance. Security teams should use automated tools like **Trivy, Clair, or Snyk** to pull apart image layers, build a bill of materials, and scan for **Common Vulnerabilities and Exposures (CVEs)** before they are deployed to a registry. This "shift-left" approach prevents supply chain attacks where out-of-date or compromised libraries cascade into production.

Orchestration Controls: Protecting the Kubernetes Control Plane

The **Kubernetes API server** serves as the central "brain" or control plane of the cluster, making it a high-value target. Leaving management interfaces or dashboards publicly exposed without strict authentication can lead to **full cluster compromise**. A notable example is the 2018 Tesla breach, where an unauthenticated dashboard allowed attackers to access cluster secrets, including AWS credentials, which were then used to mine cryptocurrency. To mitigate these risks, **Role-Based Access Control (RBAC)** is essential to enforce the **principle of least privilege**. RBAC policies should be granular, defining specific actions (verbs) that users or pods can perform on particular

resources. For instance, a web server pod should never have permissions to list sensitive cluster **secrets** stored in the **etcd** database.

Runtime and Pod Security: Boundaries and Protection

Containers are essentially isolated processes running in a "jail," but they share the **host's operating system kernel**, which presents a larger attack surface than virtual machines. **Security contexts** must be applied to pod specifications to define strict boundaries. A critical control is ensuring containers run as **non-root users** (e.g. `runAsNonRoot: true`); this limits the "blast radius" and makes it significantly harder for an attacker to perform a **container breakout** to compromise the underlying host.

Because static scanning cannot identify exploits that only manifest during execution, **runtime protection** is required. Tools such as **Falco, Cilium, or KubeArmor** leverage **eBPF** technology to monitor kernel-level activity in real-time. These tools can detect and alert on anomalous behaviours, such as the spawning of a **reverse shell** or unauthorized file system calls, allowing for immediate containment or automated remediation.



Serverless API Security

- **Ephemeral Execution:** Functions (FaaS) run in temporary containers. While "cold starts" provide a fresh environment, high-volume functions may reuse "warm" environments, leading to potential data persistence risks where sensitive artifacts (like temporary files) remain accessible between invocations.
- **Event Triggers:** Serverless functions are driven by diverse events (e.g. HTTP requests, database changes, file uploads). Each trigger is an entry point that requires strict input validation to prevent injection attacks.
- **Over-Permissioned Roles:** Functions execute with an assigned IAM role. If this role is overly broad (e.g. Allow: * on S3), a compromised function grants the attacker those full privileges. Security relies heavily on the principle of least privilege for these execution roles.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Ephemeral Execution: Cold vs. Warm Starts

Serverless functions run in temporary, containerised environments managed by the provider. A "**cold start**" occurs when a function must be deployed in a freshly stood-up container. For performance efficiency, providers often keep these environments active, or "**warm**", for a period of time to reuse them for subsequent invocations.

The critical risk here is **data persistence**. Although these environments are intended to be short-lived, any sensitive data written to writable directories (such as /tmp in AWS or the root directory in Azure and GCP) remains accessible to future executions until the container is destroyed. For example, a warm AWS environment may persist for roughly **11 minutes** of inactivity, while GCP may only last **3 minutes**. Failing to explicitly clean up temporary files can allow an attacker who exploits a vulnerability (like **Local File Inclusion**) to recover sensitive artifacts from previous sessions.

Event Triggers and Injection Risks

Serverless functions are highly reactive and can be driven by a diverse array of events, including **HTTP requests via API Gateways, database changes, S3 uploads, and scheduled tasks**. Each trigger serves as a potential entry point for malicious data. Because developers often prioritise performance in serverless

environments, they may bypass secure abstractions like **Object-Relational Mappers (ORMs)** to interact with databases directly, inadvertently reintroducing **SQL injection** vulnerabilities. Furthermore, **command injection** attacks are particularly dangerous in FaaS because they can be used to extract sensitive environment variables, such as `AWS_SESSION_TOKEN` or `AWS_SECRET_ACCESS_KEY`, which are injected into the runtime to allow the function to interact with other cloud services.

Over-Permissioned Roles and Least Privilege

Functions execute using an assigned **IAM execution role**. When a function is triggered, the platform requests temporary credentials for this role and passes them into the environment. If this role is **over-permissioned**, for instance, using wildcards (*) to allow broad access to all S3 buckets, a compromised function grants the adversary those full privileges.

To minimise the "**blast radius**" of a compromise, organisations must strictly enforce the **principle of least privilege**, assigning a **unique IAM role to each function** that grants only the minimum permissions necessary for its specific task. Regularly auditing these roles against actual usage (using tools like **IAM Access Analyzer**) is essential to prevent privilege creep and lateral movement.



Infrastructure as Code (IaC) Security

- **Codified Risks:** Defining infrastructure as code (e.g. Terraform, CloudFormation) allows for rapid deployment, but also rapid propagation of errors. A single misconfigured template can deploy thousands of insecure resources.
- **Pre-Deployment Scanning:**
 - **Shift Left:** Security tools (e.g. Checkov, tfsec) should scan IaC templates in the CI/CD pipeline before deployment.
 - **Common Findings:** These scans detect critical risks such as hardcoded credentials, unencrypted storage buckets, or security groups allowing unrestricted SSH access (0.0.0.0/0).
- **Immutable Infrastructure:** By validating configurations early, organizations prevent "born insecure" infrastructure and ensure that production environments match the approved security baseline

Codified Risks and Rapid Propagation

Infrastructure as Code (IaC) allows developers to define complex environments using high-level languages like **Terraform, CloudFormation, or Azure Resource Manager**. While this enables rapid deployment, it introduces a significant risk: **security misconfigurations** can be propagated at high velocity. Because automation can create hundreds of resources across multiple cloud providers with a single command, a single error in a template can instantly instantiate thousands of vulnerable assets across an entire enterprise. Furthermore, these templates effectively provide a "blueprint" of the environment, making it easier for an attacker to identify weaknesses if they gain access to the source code repository.

Pre-Deployment Scanning: Shifting Left

To mitigate these risks, organisations must "**shift left**," integrating security testing as early as possible in the **Software Development Life Cycle (SDLC)**. Security teams can implement "**Policy as Code**" by utilizing Static Application Security Testing (SAST) tools designed specifically for IaC, such as **Checkov, tfsec, or KICS**. These tools should be integrated directly into the **CI/CD pipeline** to scan templates and provide immediate feedback to developers before any resources are provisioned in production. This proactive approach allows the pipeline to

"stop the line" and block non-compliant deployments until the identified vulnerabilities are corrected.

Common Security Findings

Automated IaC scanners are highly effective at detecting critical "low-hanging fruit" misconfigurations that frequently lead to breaches. **Common findings** include:

Hardcoded Credentials: Developers accidentally embedding AWS access keys, API tokens, or SSH keys directly into configuration scripts or user data.

Unencrypted Storage: Storage resources like **Amazon S3 buckets** or **Azure Blobs** configured without encryption at rest or, more dangerously, with public read access.

Permissive Networking: Security group rules or Network ACLs that allow unrestricted access (e.g **SSH on port 22** or **RDP on port 3389**) from the entire internet (0.0.0.0/0).

Immutable Infrastructure and Baselines

Early validation ensures the deployment of **immutable infrastructure**, where resources are never modified in place. If a change or patch is required, a new instance is created from a hardened "Golden Image" or template, and the old one is destroyed. This paradigm eliminates **configuration drift**, where a system's state deviates from its approved security baseline over time, and ensures that production environments remain consistent, repeatable, and secure by design. By validating these configurations early, organisations prevent "born insecure" infrastructure from ever reaching the public internet.



Secure API and application design principles



Least Privilege as a Design Constraint

- **Principle Definition:** Least privilege mandates that every process, user, and program operate with the absolute minimum permissions necessary to perform a task. It must be a foundational design constraint, not a retrofit, ensuring that access is "denied by default" and granted only through explicit authorization policies.
- **API Context:** In API design, this requires ensuring that clients receive tokens with the narrowest scope possible (e.g. read-only vs. read-write) and that backend services do not forward over-privileged tokens to upstream services.
- **Design Failures:**
 - Mass Assignment: Automatically binding client input to internal objects violates least privilege by allowing users to modify fields they shouldn't (e.g. `is_admin` or `account_balance`).
 - Excessive Data Exposure: Returning full data objects to the client relies on security by obscurity. Least privilege dictates that APIs return only the specific fields required by the client's view, preventing data leakage

Principle Definition The **principle of least privilege (PoLP)** mandates that every process, user, and program operates using the **absolute minimum set of permissions** necessary to complete a specific task. For this principle to be effective, it must be treated as a **foundational design constraint** rather than a "bolt-on" security retrofit, meaning it is integrated into the architecture from the very inception of the software development life cycle (SDLC). A practical implementation of this principle ensures that all access is "**denied by default**," where privileges are refused unless an explicit authorization policy is in place to permit them.

API Context In the context of API security, enforcing least privilege requires that clients are issued **tokens (such as JWTs)** with the **narrowest possible scope** for their intended use. For instance, a mobile application should be granted only a "read-only" token if it does not need to perform "write" operations, even if the user themselves possesses higher authority. Furthermore, within microservice architectures, it is critical that backend services do not forward **over-privileged tokens** to upstream services, as this limits the "blast radius" and prevents a compromised service from being used to move laterally through the network.

Design Failure: Mass Assignment **Mass assignment** represents a significant design failure where developers allow modern frameworks to **automatically**

bind untrusted client input directly to internal code objects or database models. This violates the principle of least privilege by granting users the ability to **overwrite internal variables**, such as `is_admin`, `account_balance`, or `role`, that they should not have the authority to modify. By failing to use **whitelists** to explicitly define which fields are updatable, the system inadvertently grants users administrative-level write access to the backend data store.

Design Failure: Excessive Data Exposure Similarly, **excessive data exposure** occurs when an API returns **full object representations** from the database, relying on the frontend user interface to filter what is actually displayed. This approach relies on **security by obscurity**, which is fundamentally flawed because threat actors can easily use interception proxies to view the raw, sensitive data that the UI hides. Least privilege dictates that APIs must be refactored to **filter and mask sensitive data** at the backend, returning only the specific fields required by the client's legitimate view to prevent critical **data leakage**.



Versioning Strategies and Backward Compatibility Risks

- **API Lifecycle Management:** Improper inventory management leads to "Zombie APIs", deprecated versions that remain accessible and unpatched, often lacking newer security controls.
- **Versioning Strategies:**
 - **URI Versioning:** Including the version in the path (e.g. /v1/, /v2/) allows for clear separation but can lead to code duplication and maintenance challenges.
 - **Deprecation Headers:** Using standard headers like Deprecation and Sunset (RFC 8594) communicates retirement dates to consumers programmatically.
- **Backward Compatibility Risks:** Maintaining older versions for backward compatibility increases the attack surface. Attackers frequently target these "Shadow" or "Zombie" endpoints because they often lack the rigorous monitoring and security fixes applied to the current production version.
- **Mitigation:** Automate the retirement of old versions using API gateways and enforce strict inventory management to prevent the accidental re-exposure of deprecated endpoints.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

API Lifecycle Management & The "Zombie" Threat

A primary challenge in API governance is **API sprawl**, where the rapid proliferation of endpoints leads to a lack of visibility into an organisation's total attack surface. This mismanagement frequently results in "**Zombie APIs**", deprecated or obsolete versions of an API that remain active and accessible but are **no longer maintained or patched**. Unlike current production versions, these abandoned endpoints typically **lack the latest security controls**, making them "easy prey" for attackers. A notable historical parallel is **SMBv1**, a protocol that "refused to die gracefully" despite being designed for a world without modern malicious actors, illustrating the long-term danger of failing to retire legacy interfaces.

Versioning Strategies: URI vs. Headers

To manage the transition between API generations, organisations must employ clear **versioning strategies**:

URI Versioning: This involves explicitly including the version identifier in the path (e.g. /v1/, /v2/). While this provides clear separation and helps consumers identify the interface they are using, it can lead to **code duplication and increased technical debt** if the underlying logic is not properly abstracted [Source slide].

Deprecation and Sunset Headers: Modern standards allow for programmatic communication of an API's retirement. Under **RFC 8594**, the **Sunset header** identifies when an endpoint will become unavailable, while the **Deprecation header** acts as a timestamp for when use of the API is discouraged. These headers allow client applications to anticipate retirement and migrate to newer versions autonomously.

Backward Compatibility & Attack Surface Risks

Maintaining backward compatibility is often a business necessity to prevent breaking integrations for existing users. However, this **significantly expands the attack surface**. Threat actors specifically target older versions because they are **frequently undocumented (Shadow APIs)** and often run on **vulnerable, outdated libraries** or frameworks that have already been patched in newer releases. Furthermore, configuration mistakes can lead to the **accidental re-exposure** of these legacy interfaces, such as during a cloud instance "reversion" or a failed rollback, providing an immediate foothold for an adversary.

Mitigation through Automation and Governance

To defend against these risks, organisations must move away from manual retirement processes, which are prone to human error.

API Gateways: These act as a **central entry point (choke point)** for all inbound traffic, allowing administrators to **programmatically enforce versioning policies** and disable unauthorized or deprecated endpoints.

API Cataloguing: A centralised **API inventory or catalog** is essential; if an organisation does not know an API exists, they cannot protect it.

Automated Retirement: By using a gateway to manage the lifecycle, organisations can **automate the sunseting** of old versions, ensuring they are deprovisioned the moment they reach their end-of-life date, thereby closing the window of opportunity for an attacker.



Error Handling and Information Leakage

- **The "Treasure Map" Risk:** Verbose error messages are a primary source of information leakage, providing attackers with details about the backend architecture, database type (e.g. SQL syntax errors), and file system paths.
- **Stack Traces:** Running production servers in debug mode often results in leaking full stack traces. This reveals internal logic, library versions, and configuration details that facilitate targeted attacks.
- **Safe Implementation:**
 - **Generic Responses:** APIs should return generic error codes (e.g. 400 Bad Request, 500 Internal Server Error) to the client while logging detailed diagnostic information internally for developers.
 - **Consistent Behavior:** Error responses must be consistent to avoid "side-channel" attacks, where subtle differences in error timing or content allow attackers to enumerate valid usernames or resources.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to the Risk of Information Leakage Error handling is a critical pillar of API security; however, poor implementation often leads to **information leakage**, where sensitive details about the application's internal environment are inadvertently shared with a client. While logs are vital for troubleshooting and forensics, displaying these details to an end user can facilitate a serious security breach.

The "Treasure Map" Risk The sources explicitly describe verbose error messages as a **"treasure map"** for adversaries. These messages provide reconnaissance data that allows an attacker to pinpoint the "key" to the application by disclosing **backend architecture, database types, and file system paths**. For example, a malformed SQL query might trigger a response revealing the specific database software (such as MariaDB or PostgreSQL) and the exact SQL syntax the server tried to execute, making an injection attack significantly easier to plan.

The Danger of Stack Traces and Debug Mode A common but dangerous configuration failure is leaving production servers in **debug mode**. This oversight typically results in the leakage of **full stack traces**, which reveal internal logic, class names, and specific library versions. Such details allow threat actors to identify the exact application runtime (e.g. PHP 7.4.33) and then research

publicly disclosed vulnerabilities (CVEs) related to those specific versions to mount highly targeted attacks.

Safe Implementation Strategies To mitigate these risks, developers must adopt a **generic response strategy**. APIs should be configured to return generic HTTP error codes, such as **400 Bad Request** or **500 Internal Server Error**, to the client. While the user receives a simple "site is unavailable" message, the system should log detailed diagnostic information **internally** for developers to audit. Modern frameworks support the ability to catch unexpected errors centrally, making the display of raw exceptions to a user inexcusable.

Consistent Behaviour and Side-Channel Attacks Finally, error responses must maintain **consistent behaviour** to prevent **side-channel attacks**. If an API responds differently to valid versus invalid inputs, such as varying response sizes, different status codes, or subtle timing differences, attackers can use this information to **enumerate valid usernames or resources**. For instance, if a login endpoint takes longer to process an existing username than a non-existent one, the timing difference reveals the valid account to the attacker. Consistent outward behaviour across all error states is essential to close these loops.



Idempotency, Replay Protection, and Transactional Integrity

- **Idempotency:** An operation is idempotent if performing it multiple times yields the same result as performing it once (e.g. PUT, DELETE). However, idempotency alone does not prevent race conditions or the resurrection of deleted data if requests are replayed out of order.
- **Replay Protection:**
 - **Nonces and Timestamps:** To prevent attackers from capturing and resending valid requests (Replay Attacks), APIs should require a unique nonce or timestamp in the request header.
 - **Challenge-Response:** A robust defense involves the server issuing a random challenge that the client must sign and return, ensuring freshness.
- **Transactional Integrity (ETags):** Using Entity Tags (ETags) allows for optimistic locking. The client sends an If-Match header with the current ETag; if the resource has changed since the client retrieved it, the server rejects the update (412 Precondition Failed), preventing "mid-air collisions" and replay attacks on state changes.

Understanding Idempotency An operation is defined as **idempotent** if it can be performed multiple times while yielding the same end result as if it had been performed just once. In the context of HTTP methods, **GET, HEAD, OPTIONS, PUT, and DELETE** are standard idempotent operations. While idempotency is a core principle for reliable distributed systems, it does not inherently prevent **race conditions** or the unintended "**resurrection**" of data. For instance, if a PUT request is delayed or replayed after a legitimate DELETE request has already been processed, the deleted resource could be recreated, potentially exposing sensitive information that was intended to be removed.

Replay Protection Mechanisms To defend against **replay attacks**, where an adversary captures and resends a valid request, APIs must ensure the **freshness** of communication. One common method involves the use of **unique nonces** (numbers used once) or **timestamps** in the request header. The server tracks these values and rejects any request that attempts to reuse a nonce it has already processed. A more robust approach is the **challenge-response protocol**, where the server issues a random, time-bound challenge that the client must sign and return. This ensures that the request was generated in direct response to the server's prompt, effectively guaranteeing it is not a captured recording.

Transactional Integrity via ETags Transactional integrity and **optimistic locking** are often managed using **Entity Tags (ETags)**. An ETag is a unique identifier, typically a version number or a cryptographic hash, returned by the server to represent a specific version of a resource. When a client wishes to update that resource, it sends the ETag back in an **If-Match** header. If the resource has been modified by another process since the client retrieved it, the ETag will no longer match, and the server will reject the update with a **412 Precondition Failed** status. This mechanism successfully prevents "**mid-air collisions**" and ensures that state changes are not overwritten by stale or replayed requests.



Pagination, Filtering, and Query Limits

- **Unrestricted Resource Consumption:** APIs that allow unconstrained queries are vulnerable to Denial of Service (DoS) and scraping. Attackers can request millions of items in a single call or trigger computationally expensive database joins.
- **Pagination Defense:**
 - **Force Pagination:** Always enforce server-side pagination limits (e.g. maximum: 100) regardless of the client's request parameters.
 - **Constraint Design:** Define minimum and maximum values for perPage or limit parameters in the API specification to prevent "Big Integer" attacks.
- **Filtering and Sorting:** Unconstrained sort parameters (e.g. sortBy) can allow attackers to enumerate database schema columns or trigger SQL injection if not properly sanitized or mapped to an allowlist.
- **Resource Limits:** Implement execution timeouts and limit query depth (especially in GraphQL) to prevent resource exhaustion attacks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The Risk of Unrestricted Resource Consumption

Unrestricted Resource Consumption (API4:2023) is a critical vulnerability where an API fails to enforce limits on the system resources, such as CPU, memory, and network bandwidth, required to satisfy requests. This flaw is frequently exploited to launch **Denial of Service (DoS)** attacks that can crash servers or to perform automated **scraping** to harvest proprietary data. In cloud-native environments, failing to constrain resource-intensive queries can lead to a **"Denial of Wallet" attack**, where automated scaling drives cloud operational costs to catastrophic levels.

Implementing Pagination Defenses

To mitigate these risks, APIs must move away from trusting client-supplied parameters and instead **force server-side pagination**. A common abuse case is the **"Big Integer" attack**, where a malicious user requests an absurdly high number of items (e.g. 100 million) in a single call to overwhelm the database. Secure **constraint design** requires developers to explicitly define minimum and maximum values for perPage or limit parameters within the API specification. If a client exceeds these boundaries, the API should ideally respond with an **HTTP 429 (Too Many Requests)** status code.

Securing Filtering and Sorting

Unconstrained **sorting parameters** (such as `sortBy`) are highly dangerous because they can be weaponised for **property enumeration**, allowing threat actors to discover hidden or sensitive database columns by observing server errors. Similarly, unvalidated **filter parameters** are a primary vector for **SQL injection**, where an attacker can nullify query conditions to gain unauthorized access to all records in a table. The most effective defense is to map these parameters to a strict **allowlist or enumeration** of valid fields, ensuring the API only processes expected values.

Enforcing Global Resource Limits

Finally, organisations must implement general **resource limits** to protect system integrity. This includes setting **execution timeouts** to kill runaway processes that hang for too long. For **GraphQL APIs**, which are particularly susceptible to resource exhaustion due to their flexible nature, it is essential to **limit query depth** and the number of nodes returned to prevent recursive, deeply nested objects from depleting server resources. Utilizing **API gateways** can help offload these checks from the backend, providing a centralized point to enforce rate limiting and data validation.



Client-side web security



XSS Defense Beyond Input Filtering

- **Input Filtering Limits:** Relying solely on input filtering is insufficient because malicious payloads can be encoded (e.g. UTF-7, hex) or obfuscated to bypass filters. Furthermore, valid input for one context may be malicious in another.
- **Context-Aware Output Encoding:** The most effective defense is encoding data when it is output to the browser. The encoding method must match the context (HTML body, JavaScript variable, CSS, or URL attribute) to ensure the browser interprets the data as text rather than executable code.
- **Framework Pitfalls:** Modern frameworks (e.g. React) auto-encode data, but developers can bypass these protections using dangerous functions like `dangerouslySetInnerHTML`, reintroducing XSS risks.
- **The DOM Sink:** In DOM-based XSS, the vulnerability exists entirely client-side. Attackers target "sinks" (execution points like `innerHTML` or `document.write`) by manipulating sources (like `location.hash`), requiring sanitization libraries such as `DOMPurify`.

The Limitations of Input Filtering

While input filtering is a traditional security measure, the sources highlight that it is often insufficient on its own. Malicious payloads can be **encoded** (using URL, Hex, or Unicode) or **obfuscated** (such as splitting the string "java\nscript" or using eval to dynamically assemble code) to bypass signature-based filters and Web Application Firewalls (WAFs). Furthermore, input that is considered safe in one context, such as a blog entry describing a script, can become a malicious execution vector if rendered without protection in another. Relying solely on input filtering is discouraged because the myriad of possible representations for malicious strings makes them nearly impossible to eliminate entirely at the input stage.

Context-Aware Output Encoding

The primary and most effective defence against XSS is **context-aware output encoding**, which ensures the browser interprets data strictly as text rather than executable code. This process involves converting metacharacters into their safe HTML entity counterparts; for example, converting the < character to < ensures it is displayed on the screen but not executed as the start of a tag. Crucially, the encoding method must match the **specific context** where the data is placed. Standard HTML encoding is sufficient for the HTML body, but more complex

attributes, such as those found in **CSS properties, URL parameters, or JavaScript variables**, require specialised escaping to prevent an attacker from "breaking out" of the attribute and executing code.

Framework Pitfalls and Modern Protections

Modern development **frameworks like React** significantly reduce the attack surface by providing "security by default" through the automatic encoding of all strings rendered to the DOM. However, the sources warn that these frameworks are not invincible. A common pitfall occurs when developers deliberately bypass these built-in protections using dangerous functions like

dangerouslySetInnerHTML to render raw HTML, which reintroduces the risk of XSS. Developers are urged to exercise extreme caution when using such features, as they effectively nullify the framework's security baseline.

The DOM Sink and Client-Side Sanitisation

In **DOM-based XSS** (also known as Type 0 XSS), the vulnerability exists entirely within the client-side environment. The malicious payload often targets "**sinks**", execution points where JavaScript updates the page, such as **innerHTML** or **document.write**, by manipulating "**sources**" accessible to the browser, such as `location.hash`. Because fragment parameters (data after the # symbol) are **never sent to the server**, server-side filtering is completely ineffective against these attacks. Mitigation requires the implementation of robust sanitisation libraries like **DOMPurify**. These tools leverage the browser's own JavaScript engine to parse and "disarm" untrusted data, removing dangerous attributes like `onerror` or `onload` before they are written to the DOM.



CSRF in a SameSite-Cookie World

- **CSRF Mechanics:** Cross-Site Request Forgery (CSRF) exploits the browser's tendency to automatically send cookies with cross-origin requests, allowing attackers to perform actions (e.g. money transfer) on behalf of an authenticated user.
- **SameSite Cookie Attribute:**
 - **Lax/Strict:** The SameSite attribute (set to Lax or Strict) restricts when cookies are sent on cross-site requests, offering a strong defense layer. However, Lax (the default in modern browsers) still allows cookies on top-level navigations, which can be exploited in certain scenarios.
 - **Defense in Depth:** SameSite is not a silver bullet (e.g. it does not protect against subdomain attacks).
- **Synchronizer Token Pattern:** The most robust defense remains the use of anti-CSRF tokens (random, unguessable values) that must be included in state-changing requests. Since the attacker cannot read the token due to the Same-Origin Policy (SOP), the attack fails.

CSRF Mechanics and the "Confused Deputy" Cross-Site Request Forgery (CSRF) is a vulnerability that exploits the inherent trust a web application places in a user's browser. The attack functions by tricking an authenticated user's browser into sending a forged HTTP request to a target application. Because browsers traditionally included all relevant session cookies with every request to a domain, regardless of where the request originated, the server would process these actions as if the legitimate user had intended them. This is a classic "confused deputy" attack, where the browser is tricked into using its elevated session privileges to perform tasks like transferring money or changing account details on behalf of an attacker.

The SameSite Cookie Attribute: Lax vs Strict To mitigate this, the SameSite attribute was introduced to give developers control over when cookies are sent during cross-site requests.

SameSite=Strict: This is the most secure setting, instructing the browser to only send cookies in a first-party context, meaning the request must originate from the same site that set the cookie.

SameSite=Lax: This is now the default in modern browsers and represents a balance between security and usability. It allows cookies to be sent during "safe" top-level navigations, such as when a user clicks a link from an external site, but

blocks them for "unsafe" cross-site sub-requests like auto-submitted POST forms or images intended to trigger an action.

Defense in Depth and Limitations While SameSite attributes provide a significant defense layer, they are not a "silver bullet" and should be part of a defense-in-depth strategy. A primary limitation is that they do not protect against **subdomain hijacking or takeover attacks**. Browsers determine "sameness" based on the registerable domain (eTLD+1); therefore, a compromised subdomain (e.g. dev.example.com) is considered the same site as the primary domain (e.g. www.example.com) and can still be used to launch authenticated requests that bypass SameSite restrictions.

The Synchronizer Token Pattern The most robust defense remains the **Synchronizer Token Pattern**, which requires a unique, cryptographically strong, and unguessable token to be included in every state-changing request. These tokens must be generated per-session or per-form and verified by the server upon submission. This defense is effective because, under the **Same-Origin Policy (SOP)**, a script running on an attacker-controlled origin is permitted to send a request but is strictly prohibited from reading the response or cookies of the target site. Consequently, the attacker has no way to acquire the required token, and the blind forged request will be rejected by the server.



Clickjacking and UI Redress Attacks

- **Clickjacking:** An attack where an invisible iframe containing the target application is overlaid on a malicious site. The user believes they are clicking a harmless button but is actually triggering an action on the target application.
- **X-Frame-Options (Legacy):** The X-Frame-Options header is an older but effective defense that instructs the browser to deny rendering the page in a frame (DENY) or allow it only from the same origin (SAMEORIGIN).
- **Content Security Policy (Modern):** The frame-ancestors directive in CSP supersedes X-Frame-Options, allowing for more granular control over which domains can frame the application content.
- **Framebusting Scripts:** Older defensive JavaScript methods ("framebusting") attempted to prevent framing but are generally considered unreliable compared to HTTP headers.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Understanding Clickjacking (UI Redressing)

Clickjacking, also known as **UI redressing**, is a frontend vulnerability that uses interface trickery to redirect a user's mouse clicks to buttons or links they did not intend to interact with. The attack operates by overlaying a malicious website with an **invisible iframe** (typically using an opacity setting of 0.0) that loads a target application. While the user believes they are clicking on harmless content, such as a game or a "Continue" button, **JavaScript** on the malicious page monitors the cursor and dynamically repositions the hidden iframe so the click lands on a sensitive target, such as a "Buy" button or a "Delete Account" link. Because the legitimate user is the one triggering the action, this attack can **circumvent anti-CSRF (Cross-Site Request Forgery) tokens**. Notable historical examples include hijacking clicks to grant attackers access to a victim's **microphone and camera** through Adobe Flash settings.

Legacy Defense: X-Frame-Options

To mitigate these risks, the security industry introduced the **X-Frame-Options** HTTP response header. This legacy defense instructs the browser whether it is permitted to render the page within a frame. It supports two primary directives: **DENY**: Completely prevents the page from being rendered in a frame, regardless of the site attempting to load it.

SAMEORIGIN: Allows the page to be framed only by other pages sharing the **same origin** (identical protocol, host, and port). While effective and still widely supported, this header is considered somewhat obsolete compared to modern standards.

Modern Defense: Content Security Policy (CSP)

The modern standard for clickjacking protection is the **frame-ancestors** directive within the **Content Security Policy (CSP)**. This directive **supersedes X-Frame-Options** when both are present in modern browsers. **frame-ancestors** provides more **granular control**, allowing a site owner to specify a whitelist of trusted domains that are permitted to embed the application content, or to set it to 'none' to mirror the DENY directive. Because it is enforced at the browser level, it provides a robust defense-in-depth mechanism.

The Unreliability of Framebusting Scripts

Prior to the standardisation of HTTP headers, developers often used **framebusting scripts**. These were JavaScript snippets, such as `if (top !== self) top.location.replace(self.location.href);`, designed to force the page out of a frame and into the top-level window. However, these scripts are **generally considered unreliable** because they can be circumvented by "**anti-framebusting**" techniques. For instance, an attacker can use the `security=restricted` attribute in older versions of Internet Explorer to **disable JavaScript** within the `iframe`, preventing the defense script from ever running. Additionally, attackers can use the `onbeforeunload` event combined with a **204 No Content** status code to cancel the browser's attempt to redirect away from the framed page.



Browser Storage Decisions and Compromise Paths

- **Local Storage vs. Cookies:**
 - **Local/Session Storage:** Accessible via JavaScript. If an application has an XSS vulnerability, attackers can easily dump the entire contents of Local Storage (including tokens) using a single line of script.
 - **HttpOnly Cookies:** Not accessible via JavaScript. Storing tokens in HttpOnly cookies mitigates the impact of XSS (tokens cannot be stolen), but introduces CSRF risks that must be managed.
- **The BFF Pattern:** For maximum security in Single Page Applications (SPAs), the Backend for Frontend (BFF) pattern is recommended. The BFF manages tokens server-side and issues encrypted HttpOnly, SameSite cookies to the browser, isolating the frontend from sensitive credentials.

Local and Session Storage vs. Cookies

Modern browsers provide **HTML5 Web Storage**, consisting of **LocalStorage** and **SessionStorage**, which can store up to 10MB of text-based data per domain. While convenient for developers, this storage is **accessible via JavaScript**. This accessibility represents a critical vulnerability; if an application suffers from a **Cross-Site Scripting (XSS)** flaw, an attacker can execute a single line of malicious code to "dump" or exfiltrate the entire contents of LocalStorage, including sensitive authentication tokens. Unlike cookies, Web Storage has no equivalent to the **HttpOnly** protection, meaning it cannot be shielded from scripts running in the browser's context.

The Role of HttpOnly Cookies

To mitigate the impact of XSS, developers often use **HttpOnly cookies**. This attribute specifically instructs the browser to block any client-side scripting language from accessing the cookie via the `document.cookie` property. This ensures that even if an attacker successfully executes an XSS attack, they cannot steal the session cookie to perform out-of-band impersonation. However, cookies introduce **Cross-Site Request Forgery (CSRF)** risks because browsers are designed to automatically attach them to every request made to the issuing domain. Attackers can exploit this "ambient authority" to perform **session riding**,

where a malicious site tricks the user's browser into making **unsolicited authenticated requests**.

The BFF Pattern: The Gold Standard for SPAs

For maximum security in **Single Page Applications (SPAs)**, the **Backend for Frontend (BFF)** pattern is recommended. In this architecture, the SPA functions as a public client that delegates sensitive **token management to a server-side BFF**. The BFF acts as a confidential client, storing the actual access and refresh tokens server-side in a database or cache.

Instead of exposing tokens to the browser, the BFF issues **encrypted HttpOnly cookies** to the frontend. By applying the **SameSite=Strict** attribute, the browser is instructed to only send these cookies if the request originates from the same registrable domain, effectively **neutralising the threat of CSRF**. This pattern ensures that the frontend never handles sensitive credentials directly, significantly reducing the attack surface for both token exfiltration and session hijacking.



Content Security Policy (CSP)

- **Damage Containment:** CSP is a defense-in-depth mechanism that does not prevent XSS vulnerabilities but limits their impact (exploitability). It restricts the sources from which a browser is allowed to load resources (scripts, images, styles).
- **Directives:**
 - **script-src:** Controls valid sources for JavaScript. Disallowing 'unsafe-inline' prevents the execution of malicious scripts injected into the HTML.
 - **connect-src:** Restricts the URLs the browser can connect to (e.g. via fetch or XHR), preventing attackers from exfiltrating stolen data to their own servers.
- **Bypasses:** Weak policies (e.g. allowing data: URIs or using unsafe-eval) can be bypassed by skilled attackers. CSP must be rigorously configured and tested.

CSP: A Layer of Damage Containment

It is vital to understand that a **Content Security Policy (CSP)** is not a primary tool for preventing the existence of vulnerabilities; rather, it is a powerful **defense-in-depth mechanism** designed to contain damage. While it does not stop a developer from writing code with an XSS flaw, it severely limits the **exploitability** of that flaw by restricting the sources from which a browser is permitted to load and execute resources, such as scripts, images, and styles. By declaring a trusted baseline of content sources, the site owner provides the browser with a criteria to reject anything originating from an unauthorised domain.

Key Directives for API and Web Security

The efficacy of a CSP relies on the precise configuration of its directives:

script-src: This is perhaps the most critical directive for mitigating XSS. It defines valid sources for JavaScript execution. A robust policy will explicitly disallow '**unsafe-inline**', which prevents the browser from executing malicious scripts that have been injected directly into the HTML body. This forces an attacker to attempt to host their payload on an already allowlisted external domain, a much higher bar for exploitation.

connect-src: This directive is essential for preventing **data exfiltration**. It restricts the URLs that the browser is allowed to connect to via APIs such as

fetch or XMLHttpRequest (XHR). If an attacker successfully executes a script but tries to send stolen session tokens or PII to a server they control, the browser will block the connection if that destination is not explicitly allowlisted.

Bypasses and Rigorous Configuration

A CSP is only as strong as its weakest rule. Skilled attackers frequently look for **weak policies** to bypass these controls. For example, allowing **data: URIs** can be exploited to run inline script content that slips past source restrictions.

Similarly, using **'unsafe-eval'** nullifies protections against DOM-based XSS by allowing the creation of code from strings (e.g through the eval() function).

Because an improperly configured CSP can offer no discernible security benefit, it must be **rigorously tested** using tools like the Content-Security-Policy-Report-Only header, which allows teams to monitor for potential breakages or attack attempts without impacting the user experience during the initial rollout.



Subresource Integrity and Third-Party Risk

- **Supply Chain Risk:** Modern web applications rely heavily on third-party scripts (analytics, ads, CDNs). If a third-party provider is compromised, they can serve malicious JavaScript to all consuming applications (Software and Data Integrity Failure).
- **Subresource Integrity (SRI):** SRI allows the browser to verify that a fetched resource (like a script from a CDN) matches a known cryptographic hash. If the file has been modified (tampered with), the browser refuses to execute it, neutralizing the attack.
- **Implementation:** Developers must generate the hash of the specific version of the library they intend to use and include it in the integrity attribute of the <script> tag. This ensures that the application executes only the code it explicitly trusts.

Modern web applications are increasingly complex, with research suggesting that developers write only **20–30% of the code**, while the remaining **70–80% is sourced from third-party libraries**, frameworks, and **Content Delivery Networks (CDNs)**. This reliance creates a significant **Software and Data Integrity Failure (OWASP A08:2021)**, as an application is only as strong as its weakest link. If an attacker compromises a provider, such as an analytics or customer support script, they can inject malicious JavaScript that cascades into every consuming application. A primary example is the **2018 Ticketmaster breach**, where attackers compromised **Inbenta**, a third-party support IT company, allowing them to serve malicious scripts to Ticketmaster's users to eavesdrop on sensitive keystrokes and payment data.

Subresource Integrity (SRI) as a Defence

SRI is a security feature that allows the browser to verify that a fetched resource, such as a script or stylesheet, has not been **tampered with or modified** by an unauthorised party. By providing an **integrity check value** (a cryptographic hash), the developer establishes a known-good baseline for the code. If an attacker modifies even a single bit of the script on the CDN, the resulting hash will no longer match the value expected by the browser, and **execution is refused**. This effectively neutralises **MageCart-style attacks** by ensuring the

browser only processes code that matches the organisation's explicit trust.

Technical Implementation and Constraints

To implement SRI, developers must generate a cryptographic hash, ideally using **SHA-384 or SHA-512**, for the specific version of the library they intend to use.

This hash is included in the **integrity attribute** of the <script> or <link> tag. For SRI to function, the browser must also include the **crossorigin="anonymous"** attribute, and the remote server must support **Cross-Origin Resource Sharing (CORS)** to allow the integrity check to proceed. If the remote host does not explicitly allow the request via a CORS response header, the resource load will fail with a security error. This proactive validation ensures that applications remain resilient even if their external suppliers are compromised.



Secrets and sensitive material management



Defining Secrets in Application and API Contexts

- **Definition:** Secrets are sensitive information acting as credentials used to authenticate and authorize humans, systems, and applications to access resources.
- **Common Examples:**
 - Credentials: Usernames and passwords for database connections or administrative accounts.
 - API Keys: Long-lived strings used to identify and authenticate calling applications or partners.
 - Cryptographic Keys: Private keys used for TLS certificates (X.509) or data encryption/decryption.
 - Tokens: OAuth access tokens, refresh tokens, and JSON Web Tokens (JWTs) used for session management and authorization.
 - Sensitive Attributes: Any character string providing a means to connect to external resources that must be kept confidential.

In an application and API context, **secrets** are defined as sensitive information that provide the necessary parameters to **connect to or interact with external resources**. These elements serve as the digital handshake required to prove the identity of a human, system, or application before granting access to protected data or services. Fundamentally, secrets are used to establish **explicit trust** in a zero-trust environment, ensuring that every request is validated regardless of whether it originates from an internal or external network.

Common Examples of Secrets

The sources categorise secrets into several distinct types based on their function and lifecycle:

Credentials: These typically consist of **username and password** combinations used for local authentication, database connections, or administrative account access. For example, in a web application, database secrets include the **username, password, and database name** required for the application to read or write data.

API Keys: These are unique, **long-lived strings** generated to identify and authenticate calling applications or partners. Unlike user credentials, API keys are designed for **automation rather than people** and are often valid for months or years.

Cryptographic Keys: This category includes **private keys** used for TLS certificates (X.509) to secure data in transit or private keys used for **data encryption and decryption** at rest. Storing a private key securely, often in a hardware security module (HSM), is critical because the holder can impersonate the server or decrypt sensitive session content.

Tokens: These include **OAuth access tokens, refresh tokens, and JSON Web Tokens (JWTs)**. Access tokens are typically **short-lived bearer credentials** that determine what resources a user can access, while refresh tokens allow for the acquisition of new access tokens without requiring the user to re-authenticate. JWTs are a standard format for **self-contained security tokens** that carry claims about a user in a cryptographically verifiable manner.

Sensitive Attributes: This refers to any confidential character string that provides a means to connect to external resources, such as **connection strings** or environment variables that may include system passwords.

Risks and Management Principles

Storing these secrets directly within **application code or public repositories** like GitHub is considered a dangerous **anti-pattern**. Because source code is often shared across teams and kept in version-controlled registries, hardcoded secrets are easily exposed to unauthorised parties. To mitigate this, organisations should utilise **dedicated secrets management services** (such as AWS Secrets Manager or Azure Key Vault) that allow for **secure storage, granular access control, and automated rotation** of credentials. Regular rotation is essential to limit the "**blast radius**" and the time window in which a compromised secret can be exploited by an adversary.



Hardcoded Credentials and Leakage Paths

- **The Hardcoding Anti-Pattern:** A common vulnerability where developers embed credentials directly into source code or scripts for ease of automation, making them readable to anyone with repository access.
- **Configuration Leakage:** Secrets often leak through plaintext configuration files (e.g. wp-config.php, web.config) or unencrypted backup files stored in publicly accessible web directories.
- **Version Control Exposure:** Secrets committed to version control systems (like Git) remain in the history even if deleted later; attackers use automated scanners ("GitHub dorks") to find these keys in minutes.
- **Client-Side Exposure:** Hardcoding API keys in mobile apps or JavaScript allows attackers to decompile the application and extract the credentials.
- **Toxic Logs:** Accidentally configuring applications to log full HTTP requests or debug information can write sensitive session tokens and API keys to log files.

The Hardcoding Anti-Pattern

Hardcoding involves embedding sensitive credentials, such as **AWS root keys**, **API tokens**, or **database passwords**, directly into source code or automation scripts for convenience,,,. This practice is a significant security "anti-pattern" because it assumes that the code repository itself is a secure boundary,. In reality, once a secret is embedded in code, it is readable by anyone with access to the repository, including developers, contractors, or any threat actor who compromises the version control system,.

Configuration and Backup Leakage

Secrets frequently leak through plaintext configuration files such as **wp-config.php** for WordPress or **web.config** for ASP.NET, which are often used to store database connection strings,,,. If these files are exposed via vulnerabilities like **Local File Inclusion (LFI)** or **Server-Side Request Forgery (SSRF)**, an attacker can steal the credentials and pivot from the web server to the backend database,,,. Furthermore, administrators sometimes leave **unencrypted backup files** (e.g. .bak, .old, or .zip) in publicly accessible web directories,,,. These "forgotten" files are prime targets for automated scanners and "forced browsing" attacks,,.

Version Control and "GitHub Dorks"

Committing secrets to version control systems like **Git** creates a permanent risk because the secret remains in the **commit history** even if it is removed from the most recent version of the file,. Adversaries utilize automated scanners and advanced search techniques known as "**GitHub dorks**" to find these keys in public repositories,. Research indicates that the mean time for an attacker to discover a new secret pushed to GitHub can be as little as **20 seconds**. To remediate this, organisations must use tools like **git-secrets** to scan for patterns before commits occur or tools like **BFG** to rewrite history and scrub the leaked data,.,.

Client-Side Exposure

Client-side technologies, including **mobile applications and JavaScript frameworks**, are defined as "**public clients**" because they operate in an environment entirely controlled by the end user,.,. It is impossible to keep a secret truly hidden in these environments; any API key or token embedded in a mobile app can be extracted by **decompiling the binary** or using browser developer tools,.,.

Toxic Logs and Debugging

"**Toxic logs**" represent a secondary leakage path where applications inadvertently write sensitive data to log files. This often occurs when production servers are left in **debug mode** or when logging systems are configured to capture full HTTP requests, including **Authorization headers and session tokens**,. Additionally, verbose error messages can act as a "treasure map," revealing internal file system paths or database schemas that facilitate subsequent attacks,. Professionals must divide logs into "sanitised" and "toxic" categories, ensuring that access to the latter is strictly restricted and monitored,.,.



Storage Strategies: Environment Variables vs. Vaults

- **Environment Variables:**

- Usage: Often used to decouple secrets from code, injecting them at runtime.
- Risk: While better than hardcoding, environment variables can still be exposed via server status pages, debugging endpoints, or compromised processes (e.g. /proc filesystem access).

- **Managed Secret Stores (The Secure Alternative):**

- Solutions: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager.
- Centralization: Secrets are stored in a secure, encrypted, centralized repository rather than scattered across config files.
- Programmatic Access: Applications retrieve secrets dynamically at runtime via API calls, ensuring the secret never touches the disk in plaintext.
- Auditing: Provides a complete audit trail of which entity accessed which secret and when.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The Role and Risk of Environment Variables

Historically, developers have used **environment variables** to decouple sensitive credentials from application code, injecting them at runtime rather than hardcoding them into the source. While this prevents secrets from being committed to version control systems like GitHub, the sources emphasize that environment variables are not a "set-and-forget" security solution.

On Linux systems, environment variables are inherently exposed via the virtual **/proc filesystem**. Specifically, an attacker with local access can brute-force or guess a process ID (PID) and read the contents of /proc/PID/environ to exfiltrate plaintext secrets. Furthermore, if an application has a **Local File Inclusion (LFI)** or **command injection** vulnerability, an adversary can simply run the env command to dump all active secrets. Verbose error messages and **debugging endpoints** often inadvertently leak these variables during application crashes, providing a "treasure map" for attackers.

Managed Secret Stores: The Secure Alternative

Managed secret stores, such as **HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, and Google Secret Manager**, offer a robust architectural alternative to vulnerable environment variables. These solutions address three critical security pillars:

Centralisation: Instead of "secrets sprawl," where credentials are scattered across various config files and servers, managed stores provide a **centralised, encrypted repository**. This allows security teams to manage the entire lifecycle of a secret from a single location.

Programmatic Access: Modern applications use **APIs or SDKs** to retrieve secrets dynamically at runtime. In the "best" implementation, the secret is held only in memory and **never touches the physical disk** in plaintext, significantly reducing the risk of theft via disk-level compromise. Advanced patterns like **envelope encryption** further protect data by using a Key Encrypting Key (KEK) to wrap Data Encryption Keys (DEK), ensuring that even if the storage is breached, the data remains unintelligible without the master key.

Auditing and Rotation: Unlike environment variables, managed stores provide a **complete audit trail**, recording exactly which entity accessed which secret and at what time. Furthermore, these services support **automated rotation**, which automatically replaces secrets at defined intervals. This ensures that even if a secret is leaked, it remains valid for only a limited window, thereby minimizing the attacker's "blast radius".



Lifecycle Management: Rotation and Revocation

- **Automated Rotation:**

- Static credentials (long-lived) pose a high risk if compromised; automated rotation limits the "blast radius" by regularly changing credentials (e.g. database passwords) without human intervention.
- Managed secret stores can automate the update of both the protected resource (e.g. RDS) and the stored secret.

- **Revocation Challenges:**

- Revoking compromised keys is critical but difficult in distributed systems; creating an "allowlist" or using short lifetimes is often required to effectively cut off access.

- **The Reality of Long-Lived Tokens:**

- API keys and IAM user access keys often have no default expiration, allowing attackers to maintain persistence indefinitely if the keys are stolen.
- Best Practice: Prefer temporary, short-lived credentials (such as AWS STS tokens) over static, long-term keys to enforce regular re-authentication.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Automated Rotation and Managed Secret Stores

The sources emphasize that **static, long-lived credentials**, such as database passwords or long-term API keys, pose a severe risk because if they are compromised, an attacker can maintain **indefinite persistence** within the environment. **Automated rotation** is the primary mitigation strategy, as it limits the "**blast radius**" by ensuring that even if a secret is stolen without the organization's knowledge, its useful lifetime for an adversary is strictly curtailed. Cloud-native solutions like **AWS Secrets Manager**, **Azure Key Vault**, and **Google Secret Manager** facilitate this by automating the update of both the protected resource (such as an **Amazon RDS database**) and the stored secret simultaneously. In AWS, for instance, this is often handled via **Lambda functions** that synchronise new credentials with the database and the secret store, eliminating the need for manual intervention and reducing the risk of downtime during the update.

Revocation Challenges in Distributed Systems

Revocation is frequently described as the "**Achilles' heel**" of stateless, distributed systems because the state is often held on the client side rather than in a central database. If a key or token is compromised, effectively cutting off access is difficult because the token remains valid until it reaches its natural

expiration. The sources outline two primary methods for managing this:

Allowlists: The system maintains a list of unique IDs for valid tokens in a database; if an ID is missing (meaning it has been revoked or deleted), access is denied.

Blocklists: Conversely, a blocklist records the unique IDs of revoked tokens. To keep these lists manageable in high-scale environments, **short lifetimes** are required so that IDs only need to be blocked until they would have naturally expired.

The Reality of Long-Lived Tokens vs. Best Practices

In practice, many **IAM user access keys (AKIA)** and **standard API keys** are issued with **no default expiration**, allowing them to exist in configuration files or code repositories for years. This persistence is a high-value target for attackers, who use automated scanners to find leaked keys on platforms like **GitHub** to gain an immediate foothold. A more extreme version of this persistence is seen in **Kerberos "Golden Ticket"** attacks, where stolen long-term keys allow an adversary to forge tickets valid for up to **10 years**.

The industry best practice is to move away from these static assets in favour of **temporary, short-lived credentials**, such as those provided by the **AWS Security Token Service (STS)**. These temporary credentials typically live for **minutes to hours** (e.g. a default of 1 hour in AWS), forcing regular **re-authentication**. By attaching identities directly to compute resources (like **IAM roles** for EC2) rather than hard-coding static keys, organisations ensure that credentials are **dynamically generated and automatically rotated**, significantly reducing the window of opportunity for an attacker.



Incident Impact of Leaked Credentials

- **Authentication Bypass:** A leaked API key often acts as a username and password, allowing an attacker to assume the identity of a legitimate consumer without further verification.
- **Data Exfiltration:** Attackers use stolen credentials to bulk download sensitive customer data (PII), financial records, or intellectual property from cloud storage buckets.
- **Resource Abuse:** Compromised cloud credentials are frequently used to spin up high-performance compute instances for cryptocurrency mining (cryptojacking), leading to "Denial of Wallet" attacks.
- **Lateral Movement:** Attackers use hardcoded secrets found in one application to pivot to other internal systems, databases, or cloud environments (e.g. pivoting from a web server to the cloud control plane).
- **Destructive Attacks:** In extreme cases (e.g. Code Spaces), attackers with administrative API keys can delete entire cloud environments, including backups, leading to total business cessation.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Authentication Bypass and Identity Assumption

In modern cloud architectures, a leaked API key or access token is frequently the functional equivalent of a username and password,. Because RESTful APIs are stateless, these credentials allow an attacker to assume the identity of a legitimate consumer without further verification,. This is particularly dangerous because stolen tokens, such as Microsoft's Primary Refresh Tokens (PRT), allow adversaries to **bypass Multi-Factor Authentication (MFA)** and other conditional access signals,. Once an attacker possesses these "crown jewels," they can operate within the environment as if they were the legitimate user, making their activity appear benign and difficult to detect,.,

Data Exfiltration and Massive Information Exposure

Adversaries use compromised credentials to perform **Collection and Exfiltration (TA0009 and TA0010)**, targeting high-value assets like Amazon S3 buckets or Azure Blob storage,. Because these storage services are accessible via public REST APIs, exfiltration becomes trivial once credentials are obtained,. Notable breaches, such as the 2019 Capital One incident, involved attackers using stolen instance role credentials to bulk download millions of customer records, including **Personally Identifiable Information (PII)** and financial data,. **Resource Abuse and the "Denial of Wallet"**

Compromised cloud credentials are a primary driver of **Resource Hijacking (T1496)**, where attackers provision high-performance compute instances for **cryptocurrency mining (cryptojacking)**,. A stark example is the USAID breach, where a password-spray attack on a global admin account resulted in **USD 500,000 in unauthorized cloud charges** before detection,. This creates a **"Denial of Wallet"** scenario, where the cloud's inherent scalability is weaponised to cause catastrophic financial harm to the victim,.

Lateral Movement and Pivoting

Leaked or **hardcoded secrets** found in source code (e.g on GitHub) or configuration files provide a roadmap for **Lateral Movement (TA0008)**,,. Attackers use these discovered keys to pivot from an initial entry point, such as a compromised web server, to more sensitive internal systems or the **cloud control plane**,. For instance, a developer accidentally pushing AWS root keys to a public repository can grant an attacker total control over the entire infrastructure within minutes,.

Destructive Attacks: The "Code Spaces" Scenario

In the most extreme cases, attackers with administrative API keys launch **Impact attacks (TA0040)** designed to destroy an organisation's entire digital existence,. The case of **Code Spaces** serves as a terminal example: after failing to extract a ransom, the attacker used administrative access to delete the company's entire cloud environment, including **all virtual machines, EBS volumes, and snapshots/backups**,. Because the backups were housed in the same environment accessible by the compromised keys, they were also destroyed, leading to **total business cessation**,. This underscores the critical need to store backups in separate, isolated accounts to prevent a single credential leak from becoming a "game over" event,.



Logging, monitoring, and detection



Designing Security-Relevant Application Telemetry

- **Structuring Logs for Analysis**
- **The "Five Ws" of Logging:** Effective telemetry must answer "Who did what, where, when, and why" for every event to support forensic analysis.
- **Critical Event Categories:** Design applications to emit specific security events, including:
 - **Input Validation Failures:** Logs regarding rejected data can indicate injection attempts or fuzzing.
 - **High-Risk Operations:** Log all Create, Read, Update, and Delete (CRUD) actions on sensitive data, administrative configuration changes, and system start/stop events.
- **Standardized Formats:** Use structured logging formats like JSON, Common Event Format (CEF), or Syslog to ensure interoperability with SIEM and analysis tools.
- **Contextual richness:** Ensure entries include relevant context such as source IP, User-Agent, and session IDs, while avoiding generic error messages that lack diagnostic value.

Introduction to Effective Telemetry Telemetry is the cornerstone of a sound security posture, providing a partial transcript of past events that is vital for troubleshooting and forensics. Effective logging ensures that system administrators can accurately identify what occurred during a breach to focus remediation efforts.

The "Five Ws" of Logging The primary goal of a security logging strategy is to answer "who did what, when, where, and why". A robust audit log must capture the identity of the actor, the specific request method, the resource being accessed, the timestamp, and whether the operation was successful. Without this level of detail, it is difficult to investigate incidents or identify those responsible.

Critical Event Categories Applications should be specifically designed to emit telemetry for several key security categories:

Input Validation Failures: Applications with strong input validation generate errors when an attacker probes or fuzzes the environment; these logs are critical for identifying reconnaissance and injection attempts.

High-Risk Operations: It is essential to log all **Create, Read, Update, and Delete (CRUD)** actions involving sensitive information.

System Lifecycle and Admin Changes: Telemetry must include system start

and stop events, as well as all configuration updates, user privilege changes, and authentication modifications on critical systems.

Standardised Formats and Interoperability To ensure interoperability with **Security Information and Event Management (SIEM)** tools, logs should use structured formats like **JSON, Syslog, or Common Event Format (CEF)**. JSON is particularly ideal because its key-value structure is easy for machines to parse, eliminating the need for complex internal parsing logic. Standardising these field names across the organisation allows analysts to correlate data from disparate sources more efficiently.

Contextual Richness vs. Diagnostic Value Logs must provide enough context to be useful, including **source IP addresses, User-Agents, and session IDs**. However, developers must avoid logging generic messages that lack diagnostic value. Crucially, while logs should be detailed, they must **never include sensitive data** such as passwords, API keys, or personally identifiable information (PII), as these increase the risk of accidental disclosure if the logs are compromised.



Authentication and Authorization Signal Generation

- **Monitoring the Identity Perimeter**
- **Success and Failure Tracking:** Log both successful and failed authentication attempts. Failures help detect brute-force attacks, while successes help audit misuse by legitimate credentials.
- **Authorization Context:** Log instances where access was denied (403 Forbidden). High volumes of these events often indicate an authenticated user attempting to escalate privileges or access unauthorized resources.
- **Session Lifecycle Monitoring:** Track the entire session lifecycle, including session creation, termination, and timeouts, to detect session hijacking or anomalies in session duration.
- **Contextual Anomaly Detection:** Capture attributes such as device fingerprints, geographic location, and time of day. Alerts should trigger on "impossible travel" (simultaneous logins from distant locations) or access from unknown devices.

Monitoring the Identity Perimeter In modern security architectures, **identity is defined as the new perimeter**, making the monitoring of authentication and authorization signals a primary defensive requirement. Because traditional network boundaries have become decentralised, organisations must rely on **continuous validation** and **real-time analytics** to secure resources.

Success and Failure Tracking It is a common pitfall that many infrastructures, including **Cisco IOS** and **Microsoft Active Directory**, do not log successful or failed login attempts by default; these must be explicitly enabled to ensure **non-repudiation**. **Failure tracking** is critical for detecting automated threats such as **brute-force attacks** or **password spraying**, where a high volume of failed attempts (Event ID 4625 in Windows) occurs within a short window. Conversely, **success tracking** (Event ID 4624) is essential to audit the misuse of legitimate credentials, allowing defenders to track a compromised account as it moves laterally through the environment.

Authorization Context While authentication verifies an entity's identity, **authorization** determines what actions that identity is permitted to perform. Monitoring instances where access is denied (**HTTP 403 Forbidden**) provides vital context. A 403 error indicates that while the user's credentials were valid, they lacked the permissions for the specific resource. A high volume of these

events is often a signature of **Broken Function Level Authorization (BFLA)** or an authenticated user attempting **privilege escalation**.

Session Lifecycle Monitoring Defenders should track the entire **session lifecycle**, including creation, termination, and timeouts. Insecure session management, such as failing to invalidate a session ID after logout or using predictable session variables, can lead to **session hijacking** or **fixation attacks**. Furthermore, because some sessions end via **timeout for inactivity** rather than explicit termination, tracking these intervals is necessary to detect anomalies in session duration that might indicate an active hijack.

Contextual Anomaly Detection Robust signal generation requires capturing contextual attributes such as **device fingerprints, geographic location, and time of day**. Using **User and Entity Behaviour Analytics (UEBA)**, systems can establish a baseline of "normal" activity and score deviations. One of the most effective alerts is for **"impossible travel,"** which triggers when a single user account logs in from two distant geographic locations in a timeframe shorter than is physically possible to travel. Access from **unknown devices** or IP addresses associated with **anonymising proxies** should likewise trigger high-priority alerts.



Detecting Enumeration, Probing, and Automation

- **Identifying Pre-Attack Reconnaissance**
- **Error Rate Analysis:** A spike in HTTP 404 (Not Found) or 500 (Server Error) codes often indicates forced browsing (directory enumeration) or fuzzing attempts seeking vulnerabilities.
- **User-Agent Analysis:** Monitor for unusual User-Agent strings. Attackers often use default tool signatures (e.g. "sqlmap," "curl," or scripting languages) or mismatched agents (e.g. a CLI tool claiming to be a browser),.
- **Traffic Volume & Frequency:** Detect automation through rate analysis. A single IP generating requests faster than a human could (e.g. directory brute-forcing) or consistent "beaconing" intervals indicates automated tooling or malware C2.,.
- **Protocol Anomalies:** Network-level enumeration can be detected via half-open connections (SYN scans) or unusual TCP flag combinations that deviate from standard RFC behavior.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Identifying Pre-Attack Reconnaissance

Detection efforts must shift from being reactive to being proactive, hunting for unknown threats through advanced analysis of anomalies and behaviours. Identifying reconnaissance early allows defenders to prioritise resources and mitigate threats before an actual breach occurs.

Error Rate Analysis

A primary indicator of reconnaissance is a significant spike in HTTP error codes.

Forced browsing (or directory enumeration) involves an attacker using an automated script to request a list of URL endpoints directly to discover unlinked content or sensitive files, such as .git/config or backup databases. While occasional 404 (Not Found) errors are normal, flurries of these requests arriving milliseconds apart are highly atypical of human behaviour.

Thresholds: Security tools like Zeek can be scripted to alert when a single IP address triggers ten or more 401 (Unauthorized), 403 (Forbidden), or 404 codes within a 60-second window.

Intent: A long bout of 400-series codes suggests general reconnaissance, whereas a sequence of 500-series (Server Error) codes against specific forms, often followed by a successful 200 response, can indicate **SQL injection** attempts where the backend database has crashed or responded to

manipulated input.

User-Agent Analysis

The User-Agent string identifies the web client and is a high-value data point for identifying automated tools.

Tool Signatures: Attackers often use default signatures that reveal their tooling, such as "**sqlmap**," "**Fuzz Faster U Fool**" (Ffuf), or "**Hydra**".

Mismatched Agents: Monitoring for "mismatched" agents is a proven detection winner. For instance, a request claiming to be from an iPhone (IPHONE8.5) while the user is known to be on a laptop, or a CLI tool (like curl or wget) claiming to be a standard browser, suggests malicious activity.

Long-Tail Analysis: Defenders should ignore common strings like "Mozilla" and inspect the rarest strings in the environment; malware often "fudges" these strings in conspicuous, easily detectable ways.

Traffic Volume & Frequency

Detecting automation relies on differentiating human "bursty" traffic from the consistent patterns of machines.

Rate Analysis: A single IP generating requests faster than humanly possible (e.g thousands per second during directory fuzzing) is a clear sign of automation.

Beaconing: Malware **Command and Control (C2)** often exhibits "beaconing", periodic communication at regular intervals (e.g every 15 seconds or every minute) to receive instructions. While some "jitter" may be added to avoid detection, the consistency in packet size and interval remains a fundamental indicator of a compromised host "phoning home".

Protocol Anomalies

Network-level enumeration is often detected by observing deviations from standard RFC behaviour in TCP/IP stacks.

SYN Scans (Half-Open): Tools like Nmap often use "**stealth scans**" where they send a SYN packet to a port but never complete the three-way handshake with an ACK; instead, they send a RST (Reset) to tear down the connection once a SYN/ACK is received. This avoids opening a full connection but is easily detected by **Intrusion Detection Systems (IDS)** monitoring for high volumes of RST packets or SYN/ACKs without data transfer.

TCP Flag Combinations: Illogical flag combinations, such as **SYN and RST** being set simultaneously, or "Christmas tree" packets (SYN, FIN, PSH, and URG all set), make no sense in normal communication and are strong indicators of active scanning or packet crafting.



Correlating Logs with Identity and Infrastructure

- **Cross-Layer Correlation:** Link application-layer logic (e.g. "User X purchased Item Y") with infrastructure signals (e.g. "Database Query Z executed"). This confirms whether a suspicious web request actually resulted in backend data access.
- **Identity-to-Network Mapping:** Correlate authenticated User IDs from application logs with source IP addresses in firewall or flow logs to attribute network traffic to specific identities.
- **Traceability IDs:** Implement unique Trace IDs or Transaction IDs that persist across microservices and infrastructure components. This allows analysts to follow a single malicious request as it traverses load balancers, web servers, and databases.
- **Centralized Aggregation:** Ship all logs (Application, OS, Network, Identity) to a centralized SIEM or Data Lake. Isolated logs prevent the detection of complex attacks that span multiple systems,.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Realising **holistic visibility** in a complex environment requires moving beyond siloed log repositories to a model where data from every architectural layer is interconnected. Maintaining a year of durable, searchable log data is considered a critical component of a robust security posture.

Cross-Layer Correlation **Cross-layer correlation** involves linking high-level application logic, such as a user purchasing an item, with low-level infrastructure signals like the execution of a specific database query. This is vital for confirming the success of an attack; for example, an analyst can determine whether a suspicious web request, such as an **SQL injection attempt**, actually resulted in unauthorized backend data access or modification.

Identity-to-Network Mapping **Identity-to-Network mapping** allows security teams to achieve definitive attribution by correlating authenticated User IDs or User Principal Names (UPNs) from application logs with the source IP addresses recorded in network telemetry, such as firewall or flow logs. This mapping is essential for tracking user activity across different devices and identifying anomalies, such as a legitimate identity suddenly being used from a **geographic outlier** or an unfamiliar IP address.

Traceability IDs To maintain continuity across distributed architectures and microservices, organisations implement **Traceability IDs** or Transaction IDs.

Tools like **Zeek** generate a unique connection identifier (uid) that acts as a "foreign key" to associate related entries across different log types, such as conn.log and http.log. Similarly, cloud services like **AWS Elastic Load Balancing** utilize a trace_id (X-Amzn-Trace-Id) to follow a single request as it traverses various infrastructure components.

Centralised Aggregation Finally, **Centralised Aggregation** ensures that all telemetry, Application, OS, Network, and Identity logs, is shipped to a single location, such as a **SIEM or a Data Lake**. This avoids the pitfall of isolated data silos, which can hide complex, multi-stage attacks that span multiple systems and cloud environments. Centralisation allows analysts to query a **single source of truth** and apply advanced correlation rules to identify lateral movement or data exfiltration.



Privacy-Preserving Logging

- **Balancing Visibility with Compliance**
- **Data Minimization:** Strictly prohibit logging sensitive data such as passwords, API tokens, PII (Personally Identifiable Information), or PCI (credit card) data. This prevents the logs themselves from becoming a data breach liability.
- **Sanitization and Masking:** Implement automated scrubbing or masking of sensitive fields (e.g. replacing credit card numbers with Xs) before data is written to disk.
- **Pseudonymization:** Use stable unique identifiers (such as GUIDs) instead of real names or email addresses in logs. Maintain a separate, tightly controlled mapping table to resolve identities only during active investigations.
- **Secure Storage and Integrity:** Encrypt logs at rest and in transit. Use integrity controls (hashing/digital signatures) to ensure logs have not been tampered with by attackers covering their tracks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Balancing Visibility with Compliance Logging is a fundamental security control, serving as a partial transcript of past events that is vital for accountability and forensic investigations. However, logs can quickly become a significant liability if they are not designed with privacy and compliance in mind. Effective telemetry must balance the need for detailed visibility into system activity with the requirement to protect sensitive information from accidental exposure.

Data Minimization: Reducing Liability The principle of **data minimization** dictates that organisations should collect and process only the data absolutely necessary for a specific purpose. In a logging context, this means strictly prohibiting the recording of sensitive data such as **passwords, API tokens, PII (Personally Identifiable Information), or PCI (credit card) data**. Storing such information increases the organisation's "blast radius" during a breach, as attackers who gain access to log repositories can exfiltrate plaintext credentials. Furthermore, including sensitive data puts the logs themselves into the scope of strict regulatory frameworks like **GDPR or PCI DSS**, necessitating additional complex access controls and backup procedures.

Sanitisation and Masking: Proactive Protection To prevent data leakage, developers should implement automated **scrubbing or masking** of sensitive fields before data is written to disk. For example, a proxy or a web application

firewall (WAF) can be configured to detect patterns like credit card or Social Security numbers and replace them with masking characters, such as "Xs". Modern cloud services, such as **AWS CloudWatch**, now offer data protection policies that automatically audit and mask sensitive data within log groups. While masking is effective, it presents a challenge for detecting attacks that occur within those sensitive fields; therefore, advanced threat detection tools like machine learning-based WAFs may be needed to flag malicious injections without exposing the underlying data.

Pseudonymization: Protecting Identities Pseudonymization is a privacy-enhancing technique where identifying information (such as real names or email addresses) is replaced with an alias or a stable unique identifier, like a **GUID**. This allows users to utilise services consistently and remain accountable for their actions without disclosing their true identity to every consuming system. For forensic purposes, organisations should maintain a separate, **tightly controlled mapping table** that correlates these GUIDs to actual entities, allowing identities to be resolved only during authorized investigations.

Secure Storage and Integrity: Preventing Tampering Logs must be **encrypted at rest and in transit** using strong cryptographic standards (e.g. TLS for transit and AES-256 for storage) to ensure confidentiality. Equally critical is **integrity management**. Attackers frequently attempt to **clear or modify logs** to cover their tracks after an intrusion. To counter this, defenders should employ integrity controls such as **cryptographic hashing or digital signatures** to verify that log data remains unaltered. Services like **AWS CloudTrail** can be configured for log file integrity validation, ensuring that any unauthorised modification or deletion of log files is immediately discoverable. Storing logs in **immutable storage** further ensures they are retained for their mandated lifecycle regardless of attacker activity.



Secure development lifecycle and DevSecOps



Threat Modeling for Web and API Systems

- **Systematic Risk Identification**
- **The Process:** Threat modeling involves decomposing the application (often using Data Flow Diagrams), identifying potential threats, ranking them, and determining mitigations. It is most effective when performed during the design phase or when major changes occur.
- **Methodologies :**
 - **STRIDE:** A framework categorizing threats into Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.
 - **DREAD:** A risk ranking model assessing Damage, Reproducibility, Exploitability, Affected Users, and Discoverability.
 - **PASTA & VAST:** Alternative process-centric and visual agile threat modeling methodologies.
- **Trust Boundaries:** A critical step is identifying where data crosses trust boundaries (e.g. between the internet and the API server, or the API server and the database), as these are primary targets for attack.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to the Threat Modelling Process Threat modelling is defined as a **systematic approach** to uncovering and documenting potential risks to a software system, shifting an organisation from a reactive to a **proactive defensive posture**. It is most impactful when performed early in the **design phase** of the Software Development Life Cycle (SDLC) or when major changes occur, as identifying flaws at this stage is significantly more cost-effective than remediating them post-deployment. The process is often described as a "**team sport**" that requires collaboration between developers, security experts, and business owners to challenge architectural assumptions and think like an attacker.

Decomposition and Visualisation The first step in the process is **application decomposition**, typically achieved through **Data Flow Diagrams (DFDs)**. DFDs allow the team to visualise how data moves through four essential elements:

Processes: Code or logic that manipulates data.

Data Stores: Databases or files where information is persisted.

Data Flows: The movement of data between components.

Trust Boundaries: The perimeters where data moves between different levels of trust.

Methodologies: STRIDE and DREAD To identify threats systematically,

professionals use established frameworks like **STRIDE**, which categorises threats into six logical groups:

Spoofing: Unauthorised access using another's identity.

Tampering: Malicious modification of data.

Repudiation: Denying an action was performed due to lack of evidence.

Information Disclosure: Leaking sensitive data to unauthorised parties.

Denial of Service: Affecting the availability of the system.

Elevation of Privilege: Gaining unauthorised administrative or higher-level access.

Once threats are enumerated, they are prioritised using a risk-ranking model such as **DREAD**. This model assigns a value (typically 1–10) to five factors: the potential **Damage**, the **Reproducibility** of the attack, the **Exploitability** (effort required), the percentage of **Affected Users**, and the **Discoverability** of the vulnerability. Alternative methodologies include **PASTA** (Process for Attack Simulation and Threat Analysis) and **VAST** (Visual, Agile, and Simple Threat), which provide different lenses for exploring the threat space, such as process-centric or visual agile approaches.

The Significance of Trust Boundaries A critical task in threat modelling is identifying **trust boundaries**, as these represent the most likely points of attack. Data flows that cross these boundaries, such as those between the public internet and an API server or between an application server and a backend database, are primary targets for exploitation. Identifying these boundaries early allows defenders to place essential **mitigations and controls**, such as authentication and input validation, at the exact points where trust is transitioned.



Secure Coding Standards and Review Priorities

- **Preventing Vulnerabilities at the Source**
- **Coding Standards:** Developers must adhere to secure coding standards to prevent the introduction of common flaws like injection or poor error handling. Standards should include specific guidance on avoiding banned or unsafe functions.
- **Peer Review:** Manual code reviews are essential for finding defects that automated tools miss, particularly logic flaws and backdoors.
- **High-Risk Code:** Reviews should prioritize "high-risk" areas, such as authentication routines, cryptographic implementations, and input validation logic.
- **Code Owners:** Systems should enforce approval workflows where specific subject matter experts must review changes to sensitive files (e.g. identity management code) before merging.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Establishing and adhering to **secure coding standards** is the critical first step in preventing common flaws such as injection and poor error handling. These standards should include explicit guidance on **avoiding banned or unsafe functions**, such as standard C library routines that are notoriously prone to buffer overflows. By mandating robust patterns, for instance, requiring **parameterized queries** for all database interactions, organisations can structurally eliminate entire vulnerability classes like **SQL injection** before the code is even compiled.

Peer Review: Addressing the Limits of Automation

While **Static Application Security Testing (SAST)** tools provide fast feedback, they often suffer from high false-positive rates and lack the contextual awareness to detect complex issues. **Manual peer code reviews** are therefore essential for identifying **logic flaws and backdoors** that automated scanners typically miss. Logic flaws are particularly difficult for machines to detect because they involve intended features being used in unintended, malicious ways.

Prioritising High-Risk Code

To maintain agility in rapid release environments, manual reviews must be **risk-based**, focusing on the most sensitive components of the application. **High-risk**

code typically includes:

Authentication and Session Management: Routines handling logins, password hashing, and token validation.

Cryptographic Implementations: The use of encryption libraries, random number generation, and key management.

Input Validation Logic: Code that handles untrusted data from users, APIs, or third-party providers.

Code Owners and Approval Workflows

Modern version control platforms like **GitHub** and **GitLab** allow teams to formalise oversight through **CODEOWNERS** files. This system enforces a mandatory **approval workflow** where changes to sensitive files, such as those governing **identity management** or deployment scripts, must be reviewed and signed off by designated **subject matter experts (SMEs)** before they can be merged. This ensures that critical security controls are never modified unilaterally, providing a vital layer of **separation of duties** and accountability.



Testing Methodologies: SAST, DAST, IAST

- **Strengths and Blind Spots**
- **SAST (Static Application Security Testing):**
 - **Strengths:** Analyzes source code early in the lifecycle (white-box); identifies specific lines of code causing issues; requires no running application.
 - **Blind Spots:** High false-positive rates; cannot detect runtime environment issues or configuration flaws; struggles with dynamic languages.
- **DAST (Dynamic Application Security Testing):**
 - **Strengths:** Tests the running application (black-box) from an attacker's perspective; technology-agnostic; finds runtime and environment vulnerabilities.
 - **Blind Spots:** Testing occurs late in the lifecycle; difficult to pinpoint specific code lines; cannot test uncompiled code.
- **IAST (Interactive Application Security Testing):**
 - **Strengths:** Instruments the application runtime to combine SAST and DAST benefits; provides code-level visibility with lower false positives.
 - **Blind Spots:** Requires the application to be running and interactions (tests) to trigger code paths; security coverage depends entirely on functional test coverage.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

SAST is a "white-box" testing methodology that scrutinises application source code, bytecode, or binary code for security deficiencies without executing the program.

Strengths: It is ideal for identifying **common coding vulnerabilities**, such as injection flaws, very early in the development phase, often integrated directly into the developer's **IDE** or commit pipeline. Because it identifies the **exact line of code** causing an issue, it allows for faster and cheaper remediation.

Blind Spots: A major drawback is a **high rate of false positives**, which can lead to "security fatigue" among engineering teams. Furthermore, SAST struggles with **dynamically typed languages** like JavaScript and cannot detect **runtime misconfigurations** or logic errors that only manifest when all components are integrated.

DAST (Dynamic Application Security Testing)

DAST employs a "black-box" approach, interacting with a **running application** or API to identify vulnerabilities from the perspective of an external attacker.

Strengths: It is technology-agnostic and excels at discovering **runtime and environment-specific vulnerabilities**, such as insecure cookies or server header flaws, that SAST might overlook.

Blind Spots: DAST testing occurs **late in the SDLC**, meaning identified flaws are

often more expensive to fix. It requires a **complete, compiled codebase** to run and generally cannot pinpoint the specific faulty code line, requiring further manual investigation.

IAST (Interactive Application Security Testing)

IAST is an evolving methodology that bridges the gap between static and dynamic analysis by **instrumenting the application runtime**.

Strengths: By using an agent embedded in the application (e.g. within the JVM or .NET CLR), it monitors code as it is executed, providing **code-level visibility** with a significantly **lower false-positive rate** than SAST.

Blind Spots: The most critical limitation is that its security coverage is **entirely dependent on functional test coverage**. If a specific code path or entry point is not exercised during functional or DAST tests, IAST will remain **blind to vulnerabilities** residing in those unreachable sections.



Software Composition Analysis (SCA) and Dependency Risk

- **Securing the Supply Chain**
- **The Risk:** Modern applications rely heavily on third-party open-source components, with estimates suggesting 70-80% of codebases are composed of libraries rather than custom code. Vulnerabilities in these dependencies (e.g. Log4j) are inherited by the application.
- **SCA Function:** SCA tools automatically inventory open-source components, map dependencies, and cross-reference them against vulnerability databases (like the NVD) to identify known risks.
- **SBOM (Software Bill of Materials):** A formal, machine-readable inventory of all components and dependencies in a product, essential for transparency and rapid response to new vulnerabilities.
- **License Compliance:** Beyond security, SCA tools track software licenses to prevent legal risks associated with using components that have unfavorable licensing terms.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Modern application development has shifted away from writing custom code for every function to a model of **assembly**. Past SANS surveys estimate that internal development teams now write only **20-30%** of an application's codebase; the remaining **70-80%** consists of third-party open-source libraries and frameworks,. While this increases efficiency, it means applications **inherit** any vulnerabilities present in those dependencies,.

A prime example is the **Log4j** vulnerability, where a flaw in a ubiquitous Java logging library allowed for **remote code execution (RCE)** across millions of systems,. Because these components are used so widely, a single exploit can be used against thousands of different applications, making them high-value targets for adversaries,.

The Function of SCA Tools

Standard Static Application Security Testing (SAST) tools often fail to scan third-party libraries. SCA tools fill this gap by **automatically inventorying** the codebase to identify every external package and version in use,. These tools then cross-reference this inventory against public and proprietary databases, such as the **National Vulnerability Database (NVD)**, to flag known risks,. Advanced SCA solutions can even identify **transitive dependencies**, the libraries that your libraries depend on, which are often invisible to developers but equally

dangerous,.

SBOM: The Software Ingredients Label

The **Software Bill of Materials (SBOM)** is a formal, machine-readable record of an application's "ingredients",. Much like a nutrition label, it provides transparency into the supply chain, allowing organisations to quickly determine if they are affected when a new vulnerability is announced,.

Following the **SolarWinds** breach, US Executive Order 14028 mandated the use of SBOMs to improve national cybersecurity,. Key industry standards for SBOMs include **SPDX** (an ISO standard focused on licensing) and **CycloneDX** (an OWASP standard focused on security and vulnerability management),,.

License Compliance and Legal Risk

Beyond security, SCA tools are essential for **license management**. Many open-source components carry restrictive licenses that could lead to legal disputes, monetary damages, or the forced disclosure of intellectual property (IP) if used improperly,. By tracking these licenses automatically, SCA tools ensure that an organisation's software remains compliant with its legal and regulatory obligations,.



Fuzzing for Protocol and Logic Failures

- **Automated Bug Discovery**
- **Mechanism:** Fuzzing involves injecting massive amounts of random, malformed, or unexpected data into application inputs to trigger crashes, memory leaks, or unexpected behaviors.
- **Protocol Fuzzing:** Effective for identifying implementation errors in how an application handles standards (e.g. HTTP, API inputs), potentially revealing buffer overflows or unhandled exceptions.
- **Logic Failures:** While automated fuzzers excel at finding crashes, complex business logic flaws typically require manual testing or "smart" fuzzing that understands the application context, as automated tools lack the context to determine if a specific data flow is logically invalid.
- **Tools:** Common tools include AFL (American Fuzzy Lop) for coverage-guided fuzzing and Burp Suite Intruder for web application input fuzzing.

Introduction to Fuzzing Fuzzing is a broad term used to describe the systematic process where an analyst or attacker sends **creative, unexpected, or malformed input** to an application endpoint to discover vulnerabilities. At its core, the mechanism involves "jamming" various combinations of junk data into every input an application accepts, including request headers, parameters, and payloads, to observe how the system reacts. The objective is to force the application into an **undefined state**, often triggered when the server returns a 500 response code or crashes due to memory exhaustion.

Protocol Fuzzing and Implementation Errors Protocol fuzzing is particularly effective at identifying implementation errors in applications designed around documented standards or **Requests for Comments (RFCs)**. While RFCs provide the "rules," they do not dictate how code should be written, leading to varied and sometimes faulty implementations. Fuzzing can reveal these defects, such as **buffer overflows**, by sending extraordinarily large strings (e.g. over 1,000 characters) into fields to see if the application fails to perform proper bounds checking. Beyond crashes, protocol-level fuzzing identifies unhandled exceptions and memory leaks by subjecting the API to diverse and unconventional input scenarios.

The Challenge of Logic Failures While automated fuzzers excel at finding "low-

hanging fruit" like crashes, **business logic flaws** represent a more "pernicious" class of vulnerability. Automated tools typically lack the **context** to understand the intended business process, making it difficult for them to determine if a specific data flow, such as skipping a payment step in a workflow, is logically invalid. Consequently, logic flaws often require manual testing by trained experts who can "reason on what they see" to identify patterns of abuse. However, "smart" fuzzing tools like **RESTler** attempt to bridge this gap by learning from past requests to predict potential issues in subsequent interactions.

Essential Fuzzing Tools

AFL (American Fuzzy Lop): AFL is a highly effective, open-source **coverage-guided fuzzer** that uses genetic algorithms to evolve test cases. It is instrumental for systematically identifying vulnerabilities in software handling API interactions by intelligently mutating payloads.

Burp Suite Intruder: Considered a "killer feature" for web penetration testing, **Intruder** automates custom payload delivery across various attack positions. It enables testers to perform "deep fuzzing" on individual requests to find complex flaws like **Broken Object Level Authorization (BOLA)**, mass assignment, and injection vulnerabilities.



Shift-Left Security as Design Accountability

- **Early Prevention Over Late Detection**
- **Concept:** "Shift-left" moves security activities (testing, review, design analysis) to the earliest possible stages of the development lifecycle rather than treating security as a final "gate" before deployment.
- **Design Accountability:** Security must be a design requirement, not just a tooling output. This involves defining secure user flows and constraints during the design phase, which prevents vulnerabilities that tools cannot easily fix later.
- **Cost Efficiency:** The cost of fixing a vulnerability during the design or coding phase is significantly lower than fixing it during testing or after release.
- **Continuous Feedback:** Implementing security as code creates a tight feedback loop, allowing developers to receive immediate notifications of security issues (e.g. via IDE plugins) rather than waiting for a security team report.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The "**shift-left**" paradigm represents a fundamental change in cybersecurity, moving security activities from the end of the **Software Development Life Cycle (SDLC)** to the earliest stages. Traditionally, security has been treated as a final "gate" or a bolt-on characteristic tested only against a running application, often resulting in hundreds of vulnerabilities being discovered too late to fix easily. Shifting left involves "**building security in**" rather than "bolting it on," ensuring that quality and safety are considered part of the daily work of development. This approach allows for a **shortened feedback loop**, enabling teams to proceed with confidence at every stage.

Design Accountability and Secure User Flows

Security must be a **design requirement**, not merely an output of automated tooling. By defining secure user flows and constraints, such as limiting input parameters or enforcing robust access controls, during the design phase, organisations can prevent fundamental flaws that are difficult or impossible to remediate later in production. **Threat modelling** at this stage allows teams to identify potential attack vectors before a single line of code is written, effectively addressing business logic vulnerabilities that scanners often miss. This proactive approach ensures that stakeholders are aligned on the risks the business is willing to accept.

Cost Efficiency and the Return on Security Investment (ROSI)

Addressing security during the design or coding phase is significantly more **cost-effective** than post-deployment remediation. Sources indicate that the **cost of repair increases exponentially** as a project moves through the lifecycle; for instance, a defect that costs \$25 to fix during coding could cost upwards of **\$16,000 to resolve post-release**. This "Return on Security Investment" (ROSI) is a powerful driver for management support, as it reduces the business impact and the time required for recovery.

Continuous Feedback through Security as Code

Implementing **Security as Code (SaC)** allows for the automation of critical compliance checks directly within the CI/CD pipeline. Tools such as **IDE security plugins** and **pre-commit hooks** create a tight feedback loop, acting like a spell-checker to notify developers of issues as they type. This provides **immediate, actionable notifications** on the line of code causing the issue, allowing for rapid triage and remediation before the code is even committed. This integration ensures that security testing keeps pace with the speed of DevOps, replacing manual, late-stage security reports with real-time defensive insights.



Security Ownership in Engineering and Platform Teams

- **Culture and Shared Responsibility**
- **Shared Responsibility:** In a DevSecOps culture, security is not solely the responsibility of a dedicated security team; it is shared by developers (secure code), operations (secure infrastructure), and security (governance and monitoring).
- **Security Champions:** Embedding "security champions", developers with specialized security training - within engineering teams helps bridge the gap between security requirements and development realities.
- **Removing Silos:** Collaboration reduces the friction between "speed of delivery" and "security assurance," moving away from the perception of security as a blocker.
- **Metrics and Goals:** Teams should define and track security metrics (e.g. mean time to remediation, test coverage) to drive continuous improvement and accountability.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Introduction to Security Culture The shift toward a **DevSecOps or Zero Trust culture** requires a fundamental change in how security is perceived: it must be viewed as a **"team sport"** where responsibility is democratized across the entire organisation. This approach moves away from traditional, siloed security teams and toward an integrated model where security is a baseline design goal for everyone involved in the Software Development Life Cycle (SDLC).

Shared Responsibility In this model, security is a **shared responsibility**.

Developers are accountable for writing secure code and performing peer reviews to identify flaws early. **Operations teams** are responsible for maintaining secure infrastructure, including automated patching and hardening. The **security team** shifts its role to become governance advisors and monitors, providing the necessary training, tools, and incident response frameworks to support the other teams.

Security Champions To scale these efforts, organisations often embed **"security champions"** (or "Mavens") directly into engineering squads. These are typically senior developers with specialized security training who bridge the gap between high-level security requirements and the day-to-day realities of coding. They evangelise security best practices, lead **threat modelling** sessions, and conduct code reviews for high-risk modules.

Removing Silos and Blockers A primary objective of this cultural shift is to **remove organizational silos** that cause friction. Historically, security teams were seen as "**blockers**" who inhibited speed of delivery. By "**shifting left**", addressing security concerns at the earliest stages of design, teams shorten feedback loops and ensure that security is an accelerant for innovation rather than an obstacle. This collaboration fosters a sense of collective accountability and trust.

Metrics and Continuous Improvement Finally, teams must define and track specific **security metrics** to ensure accountability and drive **continuous improvement**. Key indicators include **Mean Time to Remediate (MTTR)**, the time taken to resolve a vulnerability after detection, and **automated test coverage** for high-risk code. Tracking the **vulnerability escape rate** to production helps teams understand the effectiveness of their pre-commit and build-stage checks. These metrics provide a common language for executives and technical leads to measure the health of the security program.



Runtime protection and enforcement



Web Application Firewalls (WAFs)

- **Reliable Detection:** WAFs operate at Layer 7 (Application Layer) to inspect HTTP/HTTPS traffic, effectively blocking "low hanging fruit" and common attacks defined in the OWASP Top 10, such as SQL Injection (SQLi) and Cross-Site Scripting (XSS) via signature matching and anomaly detection,.
- **The Logic Gap:** WAFs struggle to detect business logic attacks (e.g. Broken Object Level Authorization) because malicious requests often appear technically valid and conform to the API schema, lacking recognizable attack signatures,.
- **Bypass Risks:** Attackers frequently bypass WAFs using obfuscation techniques (e.g. encoding payloads), IP masking, or by exploiting unmonitored subdomains and internal endpoints.
- **Operational Reality:** WAFs are often deployed in "monitor mode" due to the high risk of false positives blocking legitimate traffic, which reduces their utility as an active preventive control.

Reliable Detection at the Application Layer Web Application Firewalls (WAFs) represent a crucial evolution in perimeter security, operating at **Layer 7 (the Application Layer)** of the OSI model to scrutinise HTTP and HTTPS traffic. Unlike traditional firewalls that focus on ports and IP addresses, WAFs function like a "smart proxy," inspecting the payload of requests for malicious patterns. They are highly effective at blocking "**low-hanging fruit**" and common vulnerabilities identified in the **OWASP Top 10**, such as **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**, by using a combination of **signature matching and anomaly detection**. Many organisations leverage managed cloud services, such as AWS WAF or Azure WAF, which provide pre-built rule sets to mitigate these prevalent attack vectors.

The Logic Gap: Contextual Blindness Despite their strengths, WAFs suffer from a significant "**logic gap**" They struggle to identify **business logic attacks**, such as **Broken Object Level Authorization (BOLA)**, because these requests often appear technically valid and conform perfectly to the API's data schema. Because these attacks frequently involve legitimate, authenticated users manipulating object identifiers to access data that is not theirs, they lack recognizable "malicious" signatures. Research shows that **95% of API attacks originate from authenticated users** who mimic legitimate traffic patterns,

allowing them to remain virtually invisible to traditional WAF signatures.

Bypass Risks: Evading the Filter Adversaries employ several sophisticated **bypass techniques** to circumvent WAF protections:

Obfuscation: Attackers use various encoding methods, such as **URL, Base64, Unicode, or hexadecimal encoding**, to hide malicious payloads. If a WAF does not correctly decode these strings before analysis, the attack will bypass the signature match.

IP Masking: By using VPNs, Tor nodes, or services like **AWS API Gateway** (often through tools like FireProx), attackers can ensure each request originates from a different IP address, defeating rate-limiting and IP-based reputation blocks.

Architecture Gaps: Attackers often perform reconnaissance to find **unmonitored subdomains** (e.g. `ww2.example.com` instead of a protected `www.example.com`) where a WAF might not be enforced. Furthermore, if an attacker can compromise an internal host, they can often **route traffic around the WAF** to talk to the backend application directly.

Operational Reality: The Monitor Mode Conundrum In many production environments, WAFs are deployed in "**monitor mode**" (detection only) rather than as an active preventive control. This operational reality is driven by the high risk of **false positives**, where legitimate business transactions are mistakenly blocked. Halting business operations even once or twice due to a misconfigured rule can lead to significant management pressure to disable blocking features. Consequently, to provide true security value, WAFs require constant "**care and feeding**," including manual tuning and a deep understanding of the specific application logic they are meant to protect.



Runtime Application Self-Protection (RASP)

- **Instrumentation:** Unlike perimeter defenses, RASP agents embed directly into the application runtime environment (e.g. JVM, .NET CLR), hijacking function calls to inspect data and behavior in real-time as code executes.
- **Contextual Awareness:** RASP provides deeper visibility than WAFs by analyzing how the application processes input, allowing it to differentiate between actual attacks and harmless traffic that merely resembles an attack.
- **Limitations:**
 - **Performance Overhead:** Because RASP runs inside the application, it consumes compute resources (CPU, memory), potentially introducing latency to application transactions.
 - **Language Dependency:** RASP solutions must be specifically compatible with the language and runtime environment of the application, unlike WAFs which are language-agnostic.

Instrumentation: Moving Defense Inside the Runtime Unlike perimeter-based defenses like traditional firewalls, **Runtime Application Self-Protection (RASP)** represents a fundamental shift by embedding security directly into the application's runtime environment, such as the **Java Virtual Machine (JVM)** or **the .NET Common Language Runtime (CLR)**. RASP software works by **hijacking function calls** and instrumenting the application's internal components, including servlets, libraries, and virtual machines, to inspect and filter data parameters in real-time. This allow the system to monitor exactly how code executes and block malicious calls or strip out "bad" data before it can be processed by the business logic.

Contextual Awareness: Deeper Visibility than WAFs A primary advantage of RASP is its **contextual awareness**, which provides significantly deeper visibility than a Web Application Firewall (WAF). While a WAF acts as a "smart proxy" at the network edge to block known patterns, it lacks insight into the internal mechanisms of the application it is protecting. RASP, however, operates **inside the application context**, allowing it to analyze the internal logic flow and differentiate between a harmless request that merely looks suspicious and an actual attack that is successfully exploiting a vulnerability. This makes RASP particularly effective for **protecting internal microservices** where external

network visibility may be limited.

Performance Overhead and Latency The most significant limitation of RASP is the **performance overhead** it introduces. Because the security agent runs within the production environment alongside the application code, it consumes vital compute resources, specifically **CPU and memory**. This can introduce noticeable **latency** to transactions, which may be unacceptable for high-speed or resource-constrained environments. Consequently, many organisations initially deploy RASP in **monitoring or tracing mode** to gain confidence in its accuracy and measure the performance impact before enabling active blocking.

Language Dependency and Compatibility Finally, RASP solutions suffer from **language and stack dependency**. Because they must instrument the specific runtime environment, they are built to be compatible only with certain languages (such as **Java, C#, or Node.js**), whereas WAFs are generally language-agnostic because they operate at the network layer. This requirement can increase complexity in **polyglot environments** where multiple different development frameworks are used simultaneously. organisations must ensure their selected RASP tool supports the exact technology stack used by their developers to avoid gaps in protection.



API Gateways as Policy Enforcement Points

- **Policy Enforcement Point (PEP):** Gateways act as a centralized choke point for "North-South" traffic, enforcing coarse-grained policies such as rate limiting, TLS termination, and authentication verification (e.g. validating JWT signatures),.
- **Not Logic Validators:** Gateways are generally unaware of the underlying business logic and cannot effectively validate fine-grained authorization rules (e.g. "Can User A view Resource B?"), leaving applications vulnerable to BOLA if backend logic is not robust,.
- **Protocol Translation:** They serve as a translation layer, converting various protocols (e.g. converting HTTP/2 to HTTP/1 or SOAP to REST) to unify the interface presented to clients.

Policy Enforcement Point (PEP) In a modern cloud-native architecture, the **API gateway** serves as the primary "front door" and entry point for all external client traffic. Within the context of the **Zero Trust model**, the gateway acts as a **Policy Enforcement Point (PEP)**, creating a centralized choke point that governs all **"North-South" traffic**, the data flowing in and out of the infrastructure. Its core responsibility is to ensure that every incoming request conforms to a baseline set of **coarse-grained policies** before it is allowed into the internal network. This includes performing **TLS termination** to offload computationally expensive encryption tasks, enforcing **rate limiting or throttling** to mitigate Denial-of-Service (DoS) attacks, and conducting **authentication verification**, such as validating the digital signatures of incoming **JSON Web Tokens (JWTs)**.

Not Logic Validators A critical distinction for security teams is that while gateways are excellent at authentication, they are generally **not logic validators**. They handle **"Macro-authorization"**, checking if a token is valid and contains a required general scope, but they are typically unaware of the underlying **business logic**. For example, a gateway can verify that a user has a "customer" role, but it cannot determine if "User A" is authorized to view a specific "Resource B". This is known as **fine-grained authorization**, which must be enforced at the application or microservice layer using **claims-based logic**. If the backend logic

is not robust, the system remains vulnerable to **Broken Object Level Authorization (BOLA)** or **Insecure Direct Object Reference (IDOR)**, where a validly authenticated user can access data belonging to another user simply by manipulating identifiers.

Protocol Translation Beyond security, API gateways act as a vital **translation layer** that unifies the interface presented to clients. They provide **cryptographic and protocol agility** by converting modern client requests into formats compatible with legacy backend systems. This includes translating **HTTP/2 or HTTP/3** traffic into **HTTP/1.1** for inspection, or converting **SOAP/XML** requests into modern **REST/JSON** payloads. This abstraction allows developers to modernize the frontend experience without the immediate need to refactor complex legacy data sources or internal microservices.



Zero Trust Applied to Applications and APIs

- **Identity-First Access:** Zero Trust removes implicit trust based on network location (e.g. "inside the firewall"); instead, identity becomes the new perimeter, requiring every request to be authenticated and authorized regardless of origin,.
- **Continuous Verification:** Access is not a one-time gate; it is dynamic and re-evaluated per request. Trust is ephemeral and must be continually assessed based on context, asset state, and threat intelligence.
- **Least Privilege:** Users and services are granted the absolute minimum permissions required to perform a specific task, often utilizing Just-In-Time (JIT) access to reduce the attack surface and blast radius of a compromise,.

Introduction to the Zero Trust Shift The traditional "castle-and-moat" security model, which relies on a strong perimeter to protect an implicitly trusted internal network, is increasingly insufficient for modern, distributed cloud environments,. **Zero Trust** represents a paradigm shift from static, network-based perimeters to a focus on users, assets, and resources. It operates on the fundamental assumption that the network is always hostile and that threats exist both externally and internally at all times.

Identity-First Access: The New Perimeter In a Zero Trust architecture, **network location alone is never sufficient for deciding trust**. Whether a request originates from a public coffee shop or from a server inside the corporate data centre, it is treated as untrusted until proven otherwise,. This "identity-first" approach means that **identity has become the new perimeter**, serving as the primary fortification against malicious activity,,. Every single access request to an application or API must be **explicitly authenticated and authorized**, regardless of its origin,,.

Continuous Verification: Beyond the One-Time Gate Zero Trust moves away from "one-time" authentication at the beginning of a session,. Instead, **trust is ephemeral and dynamic**, requiring continuous evaluation,,. Access is **re-evaluated per request or session**, ensuring that if a user's context changes,

such as an employee being terminated or a device falling out of security compliance, their access can be revoked immediately,,. This verification process ingests multiple **real-time signals**, including:

User identity and roles,.

Device health and posture (e.g patch levels, antivirus status),.

Contextual factors like geo-location, time of day, and unusual request patterns,,.

Threat intelligence to identify known malicious IP addresses or compromised credentials,,.

Least Privilege and Just-In-Time Access The principle of **least privilege** ensures that users and services are granted only the absolute minimum permissions required to perform a specific task,. By restricting access to only what is strictly necessary, organisations significantly **reduce the potential blast radius** of a compromise,.

A critical component of this is **Just-In-Time (JIT) access**, which provides temporary, time-bound elevated privileges only when a user needs to perform a specific administrative task,,. JIT access automatically revokes these permissions once the task is complete or the time window expires, ensuring that management ports and sensitive data are not left exposed unnecessarily,. This proactive stance effectively **minimizes the attack surface** and prevents attackers from easily moving laterally through the network,,.



Third-party and supply chain APIs



Risks Introduced by Consuming External APIs

- **OWASP API 10:2023 (Unsafe Consumption):** Developers often implicitly trust data received from third-party APIs more than user input, adopting weaker security standards for these integrations.
- **The Trust Trap:** If a third-party API is compromised, attackers can inject malicious data (e.g. SQL injection payloads) into the consuming application. Because the data originates from a "trusted" source, it often bypasses standard input validation filters.
- **Cascading Failures:** When applications depend on third-party APIs, errors or malformed payloads from the upstream provider can cascade into the consuming system, causing unexpected behaviors, data corruption, or service outages.
- **Supply Chain Attacks:** Attackers increasingly target the supply chain (libraries, frameworks, and upstream APIs) to compromise downstream consumers, as seen in attacks against Python packages and npm libraries.

OWASP API 10:2023 (Unsafe Consumption) The modern development landscape is heavily reliant on integration, yet this connectivity introduces a significant blind spot known as **Unsafe Consumption of APIs**. Developers often fall into a psychological "trust trap," where they **implicitly trust data** received from a third-party API or an established partner more than they trust direct user input. Consequently, integrations frequently adopt **weaker security standards**, such as omitting rigorous input validation or failing to sanitize data before it interacts with the backend database.

The Trust Trap and Malicious Injection If a third-party API provider is compromised, the channels returning data to consumers become **pathways for attackers**. Because this data originates from a "trusted" source, it frequently **bypasses standard input validation filters**. This allows an adversary to inject malicious payloads, such as **SQL injection or Cross-Site Scripting (XSS)**, directly into the consuming application's database or execution context. For instance, an attacker could update their profile on a trusted identity provider with a SQL command that, when fetched by your API, drops a critical table.

Cascading Failures Relying on external APIs creates **fragile dependency chains** where a failure in an upstream service can lead to **cascading failures** downstream. If a third-party API returns an **unexpected or malformed payload**

(e.g. an unhandled enumeration value), the consuming system may suffer from unhandled exceptions, data corruption, or total **service outages**. Implementing patterns like the **circuit breaker** is essential to handle these malfunctioning dependencies gracefully without taking the entire application offline.

Supply Chain Attacks Adversaries increasingly target the **software supply chain**, including open-source libraries, frameworks, and upstream APIs, to compromise a vast number of downstream organizations simultaneously. Attacks against **Python packages (PyPI)** and **npm libraries** are common examples, where malicious code is inserted into widely used dependencies. Notable incidents like **SolarWinds** and **Log4j** highlight how inherited vulnerabilities in these "building blocks" can compromise an entire system's confidentiality and integrity. To combat this, organizations must utilize a **Software Bill of Materials (SBOM)** to maintain an accurate inventory of every component and dependency in their product.



Trust Failures Between Partner Systems

- **Implicit Trust Boundaries:** Organizations often assume that traffic coming from a partner's network or via a dedicated connection is benign, failing to apply Zero Trust principles such as explicit verification.
- **Business Logic Exploits:** Partners may use APIs in unintended ways that expose data. For example, in the Experian partner API leak, a partner integrated credit check functionality directly into a public-facing web application, inadvertently exposing the credit scores of millions of Americans.
- **Data Governance Gaps:** Suppliers often have access to customer information, and customers have access to supplier systems. Without rigorous data governance and defined access controls, this bidirectional access creates a high-risk environment for data leakage.
- **SLA and Security Misalignment:** Third-party providers may not adhere to the same security standards as the consuming organization. If a provider's spending limits or rate limits are exceeded, it can result in unexpected downtime for the consumer.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

A major vulnerability in partner ecosystems is the reliance on **implicit trust boundaries**. Organizations frequently assume that traffic originating from a partner's network or via a dedicated connection (like a VPN) is inherently benign. This "trust-the-network" approach is now considered an **antipattern**. **Zero Trust** mandates that no user, device, or network is implicitly trusted; instead, the system must **explicitly verify** every request regardless of its origin. Without this verification, a compromise in a partner's environment can allow an attacker to move **laterally** into your own network via these trusted links.

Business Logic Exploits

Trust failures often manifest as **business logic vulnerabilities**, where partners utilize APIs in ways the provider never intended. The **Experian partner API leak** is a canonical example: a partner was authorized to perform credit checks but integrated this functionality into a public-facing web application without proper controls. This allowed unauthorized users to query the API directly, exposing the credit scores of millions. This highlights why providers cannot rely on partners to act as their "security gatekeepers" and must enforce their own **granular access controls**.

Data Governance Gaps

Partner relationships create a **bidirectional risk** environment. Suppliers often

require access to customer systems, while customers host data on supplier infrastructure. Without rigorous **data governance**, such as clear data classification and defined ownership, this access can lead to significant **data leakage**. Organizations must remember that while they can outsource data management, they can **never outsource their liability** for its protection.

SLA and Security Misalignment

Finally, a mismatch between a provider's security standards and the consumer's expectations can lead to operational failures. For instance, if a third-party provider's **rate limits** or **spending limits** are reached, the API may become unavailable, causing unexpected **downtime** for the consuming organization. Proper **Service-Level Agreements (SLAs)** should not only define uptime but also align on specific security requirements and incident response protocols.



Token Reuse, Blast Radius, and Privilege Creep

- **Service-to-Service Token Forwarding:** In microservices and partner integrations, simply forwarding the original user access token to upstream services is a common anti-pattern. This results in "overprivileged" tokens where backend services possess more permissions than necessary.
- **Blast Radius:** If a reused or shared token is intercepted, the "blast radius" (potential damage) increases significantly because the token grants access to multiple downstream systems and APIs rather than just the immediate endpoint.
- **Privilege Creep:** Service accounts used for API integrations often accumulate permissions over time that are never revoked. Attackers actively scan for these over-permissioned credentials in code repositories and configuration files.
- **Confused Deputy Problem:** When a service makes a call to another service using its own credentials rather than the user's, it may be tricked into performing actions the original user was not authorized to do, exploiting the high privileges of the service account.

Service-to-Service Token Forwarding: The Overprivilege Trap

In microservice architectures, a single client request often triggers a chain of calls across multiple internal services. The "easiest" but most dangerous implementation is to simply forward the original user's access token to every upstream service. This is considered a **security anti-pattern** because it results in "overprivileged" tokens: every service in the chain receives a credential that grants the power to perform every action the original user is authorised for, regardless of whether that specific service needs such broad access. To mitigate this, organisations should use **token exchange** to provide upstream services with a new token that has a strictly reduced scope, ensuring they only possess the minimum necessary permissions.

The Expanding Blast Radius of Shared Tokens

The concept of the "**blast radius**" refers to the extent of potential damage resulting from a single security failure. When tokens are reused or shared across different trust boundaries, the blast radius increases exponentially. If a broadly scoped bearer token is intercepted at a single, potentially less-secure microservice, the adversary can immediately use that same token to impersonate the user across **multiple downstream systems and APIs**. Effective mitigation requires isolating accounts to create clear security boundaries and

using short-lived, uniquely assigned credentials for each integration point.

Privilege Creep and Machine Identity Risk

Privilege creep occurs when accounts, particularly non-human service accounts, are granted elevated permissions for a one-time task (such as a specific integration or deployment) that are **never reviewed or revoked** after the task is complete. Because machine identities often outnumber human ones in large systems, they represent a massive, often unmonitored attack surface. Adversaries actively hunt for these over-permissioned credentials, which are frequently left "in the clear" within **code repositories, configuration files, or environment variables**.

The Confused Deputy Problem

The "**Confused Deputy**" is a logic flaw that arises when a service (the deputy) uses its own high-privilege credentials to perform an action on behalf of a user without verifying if that user is authorised for that specific action. A critical real-world example occurred in **2018 within the Kubernetes API server**, where attackers could trick the server into using its own highly privileged service account to execute commands on backend services that the user should never have accessed. This highlights the necessity of ensuring that backend services always receive the **context of the original user** rather than relying solely on the service's own identity.



Webhook Security: Signature Verification

- **Authentication Challenge:** Webhooks are incoming HTTP requests triggered by events. Since the receiver does not initiate the connection, standard authentication flows (like user login) do not apply, requiring alternative verification methods.
- **HMAC Signatures:** To ensure that a webhook originated from the expected provider and was not tampered with, the provider should sign the payload using a shared secret (HMAC). The receiver must recalculate the signature and compare it to the header provided.
- **Replay Attacks:** Without proper verification mechanisms (such as timestamps or nonces included in the signature), an attacker can intercept a valid webhook payload and replay it to trigger duplicate actions or state changes in the receiving system.
- **Input Validation:** Even with signature verification, the payload of a webhook should be treated as untrusted input. Malicious payloads must be sanitized to prevent injection attacks against the receiver's database or internal logic.

Webhooks represent a unique security challenge because the standard **request-response model** is reversed: the receiver does not initiate the connection, meaning standard flows like user login or session cookie validation cannot be applied [User query, 434, 755]. Because these incoming **HTTP requests** are triggered by external events, the receiving endpoint must rely on alternative methods to prove the identity of the sender. In a **Zero Trust** environment, network location alone is never sufficient for trust, requiring every request to be explicitly authenticated despite its origin [User query, 1428].

HMAC Signatures: Ensuring Origin and Integrity

To verify that a payload originated from the expected provider and has not been tampered with, organisations use **HMAC (Hash-based Message Authentication Code) signatures**. This process involves the provider using a **shared secret key** and a hash function, such as **SHA-256**, to generate a fixed-length authentication tag for the payload. The receiver then re-calculates the hash using the same secret and compares it to the **Authorization or signature header** provided in the request; if the values match, the authenticity and integrity of the message are confirmed.

Mitigating Replay Attacks

Even when a message is signed, it remains vulnerable to **replay attacks**, where

an eavesdropper captures a valid payload and tag to resend them later, potentially triggering unintended duplicate actions. To ensure **freshness**, webhook signatures should include a **timestamp** or a **nonce** (number used once). The receiver must then implement a **replay window** to reject any requests that are too old or that attempt to reuse a nonce that has already been processed.

Input Validation: The Final Layer of Defence

It is a critical mistake to assume that a verified signature implies the payload is safe; even authenticated webhooks must be treated as **untrusted input**.

Malicious actors can use authenticated channels to deliver payloads designed for **SQL injection**, **Cross-Site Scripting (XSS)**, or **Server-Side Request Forgery (SSRF)** against the receiver's internal logic. Receivers must therefore perform rigorous **input validation and sanitisation** against a strict schema to ensure that data is treated as information rather than executable code.



Webhook Security: SSRF and Internal Scanning

- **The Webhook Vector:** Webhooks are a primary vector for Server-Side Request Forgery (SSRF). Attackers configure webhooks to point to internal IP addresses (e.g. localhost, 127.0.0.1) or private network segments instead of legitimate external listeners.
- **Internal Scanning:** By analyzing the response times or error messages from webhook requests, attackers can fingerprint internal infrastructure, identifying running services like Jenkins, databases, or administrative portals that are not exposed to the internet.
- **Cloud Metadata Exploitation:** A critical SSRF risk involves pointing a webhook at the cloud instance metadata service (e.g. http://169.254.169.254). If successful, this can reveal temporary cloud credentials and allow the attacker to take over the cloud account.
- **Mitigation Strategy:** Webhook receivers should implement strict allowlists of permitted domains and IP ranges, and use a network proxy to prevent the application server from directly connecting to internal or sensitive loopback addresses.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Webhooks are a pervasive feature in modern APIs, allowing users to provide a custom URL that the server calls when specific events occur. However, this creates a primary vector for **Server-Side Request Forgery (SSRF)**. SSRF occurs when an application accepts a user-supplied URL and retrieves the content without sufficient validation. Attackers abuse this by configuring webhooks to point to internal or private IP addresses, such as **localhost (127.0.0.1)**, rather than legitimate external listeners. This effectively turns the trusted server into a proxy for the attacker's malicious requests.

Internal Scanning and Fingerprinting

Because the request originates from within the trusted network, it can often bypass firewall barriers that block external traffic. Adversaries use this to perform **internal network scanning**, identifying services that are not exposed to the internet. By carefully analyzing HTTP response codes (e.g. 200 OK vs. 404 Not Found) or detailed error messages, attackers can **fingerprint** internal infrastructure. Common targets identified through this method include **Jenkins servers**, administrative portals, or databases like **PostgreSQL**, which can then be targeted for further exploitation.

Cloud Metadata Exploitation

In cloud environments, a critical SSRF risk involves targeting the **Instance**

Metadata Service (IMDS), traditionally accessible at the link-local address `http://169.254.169.254`. If a webhook is successfully pointed at this service, it can reveal highly sensitive information, including **temporary cloud credentials** (AccessKeyId and SecretAccessKey) associated with the instance's IAM role. This was the exact vector used in the **2019 Capital One breach**, where an attacker retrieved credentials through a misconfigured WAF to exfiltrate data from over 100 million customers.

Mitigation Strategies

To secure webhooks, developers should move beyond simple denylists and implement **strict allowlists** of permitted domains and IP ranges. Other essential strategies include:

Outbound Proxies: Use an outbound request proxy to inspect and evaluate the risk of API calls before they are executed.

Network Isolation: Execute webhook calls from a sandboxed environment that has no access to sensitive internal segments.

Enforcing IMDSv2: AWS organizations should mandate **IMDSv2**, which requires a session token and a PUT request, making it significantly more resistant to standard SSRF attacks.

Input Validation: Ensure all user-supplied URLs are strictly validated and that the application does not follow arbitrary redirects.



API testing and security assessment



Manual API Testing vs. Automated Scanning

- **Balancing Speed and Context**
- **Automated Scanning (DAST/SAST):**
 - **Strengths:** Speed and scale. Essential for finding "low-hanging fruit" like injection flaws, configuration errors, and known CVEs across a broad attack surface.
 - **Weaknesses:** Lacks context. Scanners struggle with complex authentication flows and often produce false positives or miss logic flaws because they cannot understand the application's intent.
- **Manual Testing Mindset:**
 - **Strengths:** Critical for identifying business logic vulnerabilities (e.g. skipping payment steps) and complex authorization issues (BOLA/BFLA) that require an understanding of the workflow.
 - **Human Element:** "There is still no better web application tester than a well-trained person with a brain" to identify logic errors and validate findings.
- **Hybrid Approach:** The most effective strategy combines automated scanning for breadth with manual testing for depth, using manual findings to tune automated tools.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Automated Scanning: Speed and Scale Automated scanning, which includes **Static Application Security Testing (SAST)** and **Dynamic Application Security Testing (DAST)**, is essential for identifying "low-hanging fruit" across a broad attack surface,. SAST allows developers to find flaws and specific lines of problematic code early in the development cycle, though it is often **prone to false positives** and struggles with complex, dynamically typed languages,. Conversely, DAST interacts with the running application to detect **runtime-level errors** and dynamic vulnerabilities without requiring access to the original source code,. These automated tools are highly valued for their ability to perform the **millions of interactions** required to test a multitude of inputs for common vulnerabilities like **SQL injection and Cross-Site Scripting (XSS)**,. However, scanners are fundamentally limited as they generally only find **known vulnerabilities** and lack the intelligence to understand the underlying business intent,.

The Limitations of Automation Automated tools frequently struggle with **complex authentication flows** and often miss nuanced security issues because they cannot determine if a successful response is logically valid,. For example, if an application successfully processes an unauthorized request to a funds-transfer page without returning an error, a scanner will typically assume the

transaction is perfectly acceptable. This lack of **contextual awareness** means scanners may fail to identify when data that was returned should have been restricted. Furthermore, over-reliance on these tools can provide a **false sense of security**, as they are often fooled by misleading banners or inconsistent configurations,.

The Manual Testing Mindset The human element remains the most critical component for identifying **business logic vulnerabilities** and complex authorization issues such as **Broken Object Level Authorization (BOLA)** and **Broken Function Level Authorization (BFLA)**,. These flaws are effectively **zero-day vulnerabilities** unique to a specific API, requiring a trained professional to reason about what they see and identify patterns of abuse,. A skilled human tester can discover where a weakness lies faster than a fuzzer because they can understand the **application's purpose** and the intended workflow,. As the sources emphasise, "**there is still no better web application tester than a well-trained person with a brain**" to identify these "pernicious" logic errors that scanners routinely overlook,,.

The Hybrid Approach: Breadth and Depth The most effective security strategy is a **hybrid approach** that combines the **breadth** of automated scanning with the **depth** of manual expert testing,. Automation is leveraged to perform the repetitive task of **fuzzing all inputs** for common defects, while manual assessments are prioritised for high-risk components, recent changes, or upcoming major releases,. Manual findings should be used to **tune and validate** automated tools, ensuring that true positive results are identified while noise and false alerts are reduced,. Ultimately, this combination ensures a **rigorous assessment** that catches both the common "low-hanging fruit" and the complex, business-specific flaws,.



Leveraging OpenAPI Specifications (OAS)

- **Contract-Driven Security Testing**
- **Seeding Scanners:** Importing OAS definitions (Swagger files) into tools like OWASP ZAP, Burp Suite, or Postman ensures 100% coverage of endpoints, including those not linked in the UI.
- **Design-Time Auditing:** Using linters (e.g. Spectral) to analyze the OAS definition for security flaws - such as missing authentication directives or unconstrained data schemas - before code is even written.
- **Contract Testing (Fuzzing):** Tools like Schemathesis or Microsoft's RESTler use the OAS definition to automatically generate test cases, sending malformed payloads to verify the implementation strictly adheres to the security contract.
- **Mocking:** OAS definitions allow security teams to build mock servers to test client-side security handling without waiting for the backend to be fully implemented.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The **OpenAPI Specification (OAS)**, formerly known as Swagger, serves as a formal, machine-readable **contract** between API producers and consumers. By defining every endpoint, parameter, and constraint in a standard format, it becomes the single source of truth for both development and security testing. This contract-driven approach is fundamental to a **shift-left security** strategy, allowing teams to identify and remediate vulnerabilities before they are integrated into the codebase.

Seeding Scanners for 100% Coverage

A significant challenge in API security is **shadow or zombie endpoints** that are not linked in the User Interface (UI) and therefore remain invisible to traditional web spiders. Importing OAS definitions directly into tools such as **OWASP ZAP, Burp Suite, or Postman** ensures complete coverage by providing the scanner with a comprehensive map of all intended endpoints and methods. This pre-populates the scanners with the correct data structures and parameters, allowing for more precise and aggressive vulnerability scanning.

Design-Time Auditing: Catching Flaws Early

Security testing can begin even before code is written by using **linters like Spectral** to audit the OAS definition itself. These tools can automatically flag **design-time security flaws**, such as endpoints missing mandatory

authentication directives. Furthermore, they detect **unconstrained data schemas**, such as integers without minimum/maximum values or strings without specific regex patterns, which could be exploited for SQL injection, property enumeration, or Denial of Service (DoS) attacks.

Contract Testing and Fuzzing

To prevent "**API drift**" where the server implementation differs from the intended design, organisations utilise **contract testing**. Fuzzing tools like **Schemathesis** or **Microsoft's RESTler** ingest the OAS definition to dynamically generate hundreds of test cases per endpoint. These tools send malformed, invalid, or extreme payloads to the running API to verify that the implementation strictly adheres to the security constraints defined in the contract and correctly rejects dangerous input.

Mocking for Parallel Security Testing

OAS definitions facilitate the automated generation of **mock servers**, which simulate the behaviour of the actual API backend. This allows security teams to build and test client-side security handling and integrations in a controlled environment. By using mocks, developers and testers can validate security logic, such as how a mobile app handles an expired token or a 403 Forbidden error, without needing a fully implemented or stable backend.



Authorization Testing in Multi-Tenant Systems

- **Validating Isolation and Access Control**
- **The BOLA/IDOR Challenge:** The most critical test in multi-tenant systems is verifying that User A cannot access User B's data by simply manipulating an ID in an API call.
- **Testing Strategy (A-B Testing):**
 - **Setup:** Create resources as "Tenant A" and attempt to access/modify them using a token from "Tenant B".
 - **Matrix:** Test vertical privilege escalation (User to Admin) and horizontal escalation (Tenant A to Tenant B).
- **Token Analysis:** Verify that multi-tenant applications correctly validate the "audience" (aud) and "issuer" (iss) claims in JWTs to prevent cross-tenant token usage.
- **Automation:** Use tools (e.g. Burp's Autorize extension) to automatically replay requests with different session tokens to detect authorization bypasses at scale.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The BOLA/IDOR Challenge in Multi-Tenant Systems

In multi-tenant architectures, where data from multiple organisations may be stored in the same database tables, enforcing robust isolation is the most critical security requirement. The most prevalent threat to this isolation is **Broken Object Level Authorization (BOLA)**, historically known as **Insecure Direct Object Reference (IDOR)**. BOLA occurs when an API fails to validate that an authenticated user has the specific right to access the object requested. Because APIs often use predictable identifiers (like numerical IDs or UUIDs) in the URL path or payload, threat actors can easily manipulate these IDs to probe for loopholes that permit them to view, modify, or delete data belonging to other tenants.

A-B Testing Strategy and Escalation Matrix

To identify these flaws, security professionals employ an **A-B testing strategy**. This involves a structured setup: a resource is created using "User A" from "Tenant A", and the tester then attempts to access or modify that same resource using an authentication token belonging to "User B" from "Tenant B". A robust testing program must evaluate a matrix of access levels:

Horizontal Escalation: Testing if a user can access the data of a peer in a different tenant.

Vertical Escalation: Testing if an unprivileged user (e.g. a student) can access administrative endpoints (e.g. /admin/students) or privileged functions intended for a higher role.

Token Analysis: Validating aud and iss

A core component of multi-tenant security is the rigorous validation of **JSON Web Tokens (JWTs)**. It is a "catastrophic mistake" for an application to assume a JWT is valid without checking its signature and claims. In a multi-tenant context, the application must verify the **issuer (iss)** claim to confirm the token was generated by the correct tenant's authorization server. Simultaneously, the **audience (aud)** claim must be checked to ensure the token was intended for the specific API being called; failure to validate the audience can lead to a token meant for one context being replayed to gain unauthorized access in another.

Automation and Scalability

Manually testing every endpoint for authorization flaws is tedious and does not scale in complex environments. Automated tools such as **Burp Suite's Authorize extension** help solve this by automatically replaying requests with different session tokens as the tester spiders the application. Authorize highlights any instances where a request succeeds despite using an unauthorized token, allowing teams to detect BOLA and access control bypasses at scale. Implementing these checks within a **CI/CD pipeline** ensures that new code does not introduce "API drift" or regress into previous vulnerabilities.



Business Logic Testing Techniques

- **Exploiting the Workflow, Not the Syntax**
- **Understanding Context:** Logic flaws are unique to the application; automated tools miss them because they are "features" used maliciously (e.g. using a "gift card" endpoint to generate unlimited credit).
- **Workflow Manipulation:**
 - **Skipping Steps:** Attempting to jump directly to a "payment success" endpoint without submitting payment credentials.
 - **Quantity Manipulation:** Testing for mass assignment issues, such as modifying the `is_admin` or `account_balance` fields in a profile update request.
- **Constraint Testing:** Validating that the API enforces business limits (e.g. max order quantity, coupon reuse) rather than relying on client-side logic.
- **Documentation Analysis:** Reviewing API documentation (e.g. Arazzo specifications) to understand intended flows and create specific "abuse cases".

Business logic represents the core objectives and rules that drive an application. Unlike technical flaws such as SQL injection, **logic flaws are unique to the specific business process** of the application. Automated vulnerability scanners frequently miss these issues because the malicious requests often use perfectly **valid syntax** and conform to API schemas; as far as a tool is concerned, the interaction is a legitimate "feature". For instance, an attacker might use a gift card endpoint to generate unlimited credit, which appears as a technically correct transaction but subverts the intended business goal. Identifying these risks requires "thinking like an attacker" to find gaps where developers assumed users would only behave benevolently.

Workflow Manipulation: Reordering and Skipping

Many APIs are designed with an **intended sequence of operations**, such as requiring a user to validate their identity before resetting a password. Testing involves **skipping steps** or reordering them in ways the developer never envisioned. A classic exploit is attempting to jump directly to a "payment success" endpoint without providing valid credentials, or adding more items to a shopping cart after the total has been calculated and authorised. Because these individual steps might be stateless, the backend may fail to verify that preceding mandatory actions were completed.

Quantity Manipulation and Mass Assignment

Attackers frequently use **Mass Assignment** to manipulate quantities or account attributes. This occurs when an API backend blindly binds all client-supplied parameters to internal code variables or database objects. In a profile update request, a tester should attempt to inject sensitive fields like **is_admin: true** or **account_balance: 9999**. If the server processes these extra parameters without checking them against a whitelist of permitted fields, it can lead to immediate **privilege escalation** or financial data corruption.

Constraint Testing: Breaking Server-Side Logic

A significant design pitfall is **relying on client-side logic** to enforce business limits. Security must be enforced on the server to prevent users from bypassing restrictions. Testers should validate that the API strictly enforces constraints such as **maximum order quantities** or prevents **coupon reuse**. For example, a system might fail to check if a one-time voucher has already been claimed, allowing a user to register for multiple courses using the same code.

Documentation Analysis and Abuse Cases

Effective testing begins with reviewing API documentation, such as **Arazzo specifications**, to understand intended user journeys. By mapping out the data flow between resources, testers can create "**abuse cases**" that specifically target weak links in the chain. A key tactic is to **disobey instructions** found in the documentation; if the manual says "only use feature X for function Y," a tester should immediately try to use it for function Z to see if technical controls actually exist.

Missing Context

While the sources extensively cover web and REST API logic flaws, there is less detail on how these techniques differ specifically for **GraphQL or WebSocket** interfaces compared to REST, though the general principles of reordering and mass assignment likely still apply.

Source Contradictions

There are no significant contradictions in the sources; they consistently agree that business logic testing is a manual, human-centric effort that cannot be fully automated due to the need for contextual business understanding.



Red Team vs. Compliance-Driven Assessments

- **Compliance Assessments (Audits):**

- **Focus:** Verifying the existence and configuration of controls against a standard (e.g. PCI DSS, CIS Benchmarks).
- **Goal:** Proving adherence to policy; often relies on "point-in-time" evidence like screenshots or configuration exports.

- **Red Team Operations:**

- **Focus:** Simulating a realistic, goal-oriented adversary (e.g. "exfiltrate the customer database") using real-world Tactics, Techniques, and Procedures (TTPs).
- **Goal:** Testing the organization's resilience and the Blue Team's ability to detect and respond to an active breach.

- **Outcome:** Compliance proves you followed the rules; Red Teaming proves whether the rules actually stop an attacker.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Compliance Assessments: Verifying the Plan Compliance-driven assessments, often referred to as **security audits**, are fundamental for many organisations to meet regulatory or legal requirements, such as **PCI DSS, HIPAA, or CIS Benchmarks**. The primary focus of an audit is a **line-for-line comparison** of a device or application configuration against a published standard. This process aims to prove adherence to a defined security policy and typically relies on **"point-in-time" evidence**, such as configuration exports, logs, or screenshots, to demonstrate that specific controls are in place. While audits can improve a component's security posture, they are not comprehensive security assessments; a compliant company is **not necessarily a secure one**. If an organisation is not compliant, it is likely insecure, but even breached companies are often found to have been technically compliant with their selected standards prior to the incident.

Red Team Operations: Simulating the Adversary In contrast, **Red Team operations** shift the focus from a checklist-driven approach to a **goal-oriented, opportunistic emulation** of a realistic adversary. These teams use real-world **Tactics, Techniques, and Procedures (TTPs)**, such as spear-phishing or credential hijacking, to achieve a specific objective, like exfiltrating a customer database. Unlike traditional penetration tests, which are often scope-driven to

find as many flaws as possible, Red Teaming evaluates the **holistic maturity** of an organisation's security program. It specifically tests the **Blue Team's ability** to detect, respond to, and contain an active threat in real-time.

The Outcome: Rules vs. Resilience The critical distinction lies in the outcome: **Compliance proves you followed the rules**, providing a baseline of "basic cyber hygiene" and ensuring that administration processes are adhered to. However, **Red Teaming proves whether those rules actually stop an attacker**. By emulating the "enemy," Red Teams identify gaps in detection and response that a static audit would likely miss, providing a more accurate gauge of actual risk after the test is complete. Organizations that combine both, using audits for baseline consistency and Red Teaming for resilience testing, are best positioned to defend against sophisticated, persistent threats.



Real-world API and application breach analysis



How Large-Scale Breaches Actually Unfold

- **The Attack Lifecycle: From Reconnaissance to Exfiltration**
- **The Kill Chain Reality:** Breaches are rarely a single event but a sequence of steps; adversaries start with reconnaissance to identify exposed assets (OSINT), move to initial access (e.g. credential theft), and then pivot to lateral movement and data exfiltration.
- **Case Study: Capital One:** The attacker exploited a misconfigured Web Application Firewall (WAF) via Server-Side Request Forgery (SSRF) to query the AWS Instance Metadata Service (IMDS), stealing credentials to access and exfiltrate data from S3 buckets.
- **Case Study: Code Spaces:** Attackers conducted OSINT to find exposed portals, gained credentials (likely via brute force or key theft), and accessed the AWS control plane to demand ransom; when the company resisted, the attackers deleted backups and resources, effectively destroying the business.
- **Supply Chain Vectors:** Attackers increasingly compromise third-party libraries or CI/CD pipelines (e.g. SolarWinds) to inject malicious code, turning trusted software updates into initial access vectors.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The Kill Chain Reality Large-scale cyberattacks are rarely isolated, spontaneous events; instead, they represent a **calculated sequence of steps** known as the **Cyber Kill Chain** or the attack lifecycle,. This progression typically begins with **Reconnaissance**, where adversaries leverage **Open-Source Intelligence (OSINT)** to gather data from public sources, such as social media, DNS records, and technical search engines like Shodan, to identify exposed assets and potential vulnerabilities,,,. Once a target is identified, the attacker moves to **Initial Access**, often through methods like **credential theft, phishing, or exploiting public-facing applications**,,. After gaining a foothold, the adversary pivots to **Lateral Movement**, navigating the internal network to locate "crown jewels" before concluding with **Data Exfiltration** or systems impact,,,. **Case Study: Capital One** The **2019 Capital One breach**, which compromised over **100 million customer records**, is a canonical example of this technical chain,,. The attacker exploited a **misconfigured Web Application Firewall (WAF)**,,. Using a **Server-Side Request Forgery (SSRF)** attack, the adversary tricked the application into querying the **AWS Instance Metadata Service (IMDS)**,,. Because the EC2 instance was **over-privileged**, the attacker successfully retrieved **temporary security credentials**,,. These stolen tokens allowed the attacker to impersonate the instance and **exfiltrate massive**

volumes of sensitive data directly from **Amazon S3 buckets**,,,.

Case Study: Code Spaces The **Code Spaces attack of 2014** demonstrates the devastating potential of **Management Plane** compromise,. Attackers conducted **OSINT** to discover exposed portals and gained **administrative cloud credentials**,. With access to the **AWS control plane**, the attackers attempted to **extort the company** for a ransom,. When Code Spaces resisted and attempted to regain control by changing passwords, the attackers, who had already established **backdoor resources** for **Persistence**, responded by **deleting critical resources**, including virtual machines, block storage, and all backups,,. This **data destruction (T1485)** effectively **destroyed the business**, forcing it to cease trading,.

Supply Chain Vectors Modern adversaries increasingly target the **Software Supply Chain**, compromising the third-party libraries and **CI/CD pipelines** that organisations implicitly trust,,. In the **SolarWinds (SUNBURST) attack**, threat actors injected **malicious code** directly into the **build tasks** of the CI/CD server,. This allowed the malware to be **digitally signed** by the build pipeline and distributed as a legitimate software update to over **18,000 customers**,,. Such attacks are particularly dangerous because they turn **trusted update mechanisms** into initial access vectors, bypassing traditional perimeter defences,,.



Repeating Architectural and Authorization Failures

- **Why the Same Vulnerabilities Keep Reappearing**
- **BOLA/IDOR Dominance:** Broken Object Level Authorization (BOLA) remains the primary API vulnerability, as seen in breaches like Peloton and USPS, where authenticated users could simply change an ID parameter to view other users' private data.
- **Broken Authentication:** High-profile breaches often stem from weak credential management, such as the T-Mobile breach where an unauthenticated or poorly secured endpoint allowed mass data access, or the use of hardcoded tokens in mobile apps.
- **Mass Assignment:** Developers often bind client input directly to internal data models for efficiency, allowing attackers to modify sensitive fields (e.g. `is_admin: true`) if the architecture does not explicitly filter input, as seen in the Ruby on Rails GitHub compromise.
- **Excessive Data Exposure:** APIs frequently return full data objects (including PII) to the client, relying on the frontend to filter what is displayed; attackers intercept this traffic to scrape sensitive data that the UI hides but the API leaks.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Broken Object Level Authorization (BOLA), also known as **Insecure Direct Object Reference (IDOR)**, remains the most prevalent and devastating API vulnerability. It occurs when an API provider fails to implement user-based access controls at the resource level, allowing one user to access data belonging to another simply by manipulating identifiers like names, numbers, or UUIDs in the request. High-profile breaches, such as the **USPS Informed Visibility API** leak affecting 60 million users, were caused by this lack of granular authorization. Similarly, **Peloton** was found to have leaked the personal data of millions of subscribers because its API allowed unauthenticated users to query workout details and user profiles through unvalidated identifiers. These flaws persist because they reside in the **business logic** rather than the syntax; traditional security scanners often miss them because the requests themselves appear legitimate.

Broken Authentication and Credential Mismanagement

High-profile breaches frequently stem from **Broken Authentication**, which involves weaknesses in verifying user identity. This includes poor credential management, such as the **T-Mobile** breach where weak authorization checks allowed the theft of PII for 37 million customers. Furthermore, developers often commit the "unforgivable" error of using **hardcoded tokens** or API keys within

mobile app binaries or public code repositories for the sake of convenience. Because mobile apps are easily decompiled, these static strings provide a direct pathway for attackers to bypass the entire authentication layer.

Mass Assignment: The Efficiency vs Security Trade-off

Mass Assignment occurs when an API backend blindly binds client-supplied input to internal data models or code variables for developer efficiency. Many modern frameworks include "autobinding" features that map HTTP parameters directly to database objects. Attackers exploit this by injecting sensitive fields, such as **is_admin: true**, into request payloads. A canonical example is the **Ruby on Rails GitHub compromise**, where a researcher was able to add his own SSH key to the Rails repository by manipulating the attributes passed during a profile update.

Excessive Data Exposure: Relying on the Frontend

A critical design anti-pattern is **Excessive Data Exposure**, where APIs return full data objects to the client, trusting the frontend (web or mobile UI) to filter what is actually displayed to the user. Developers often use efficient helper methods like **to_json()** on full database objects, inadvertently leaking sensitive fields like PII or internal IDs that the user does not need to see. This reliance on client-side masking is fundamentally flawed; once data is transmitted, it can be easily intercepted using basic browser developer tools or interception proxies. This was a core factor in the **Venmo** data scraping incident, where 200 million transactions were harvested because the API returned full details that the UI merely hid.



Why Perimeter and Gateway Controls Failed

- **The Limits of WAFs and Gateways**
- **The "Authenticated" Attacker:** Most API attacks (95%) come from authenticated users who appear legitimate to WAFs and Gateways because they have valid credentials but are abusing business logic logic flaws.
- **Context Blindness:** API Gateways and WAFs typically inspect traffic for known signatures (like SQL injection) but lack the context to detect BOLA or logic abuse, such as a user scraping data they technically have permission to view but are accessing at an impossible volume.
- **Misconfiguration & Permissiveness:** Perimeter controls like WAFs are often misconfigured or set to "log only" to avoid blocking legitimate traffic, or they fail to inspect specific traffic types (like internal metadata calls), creating bypass opportunities for attacks like SSRF.
- **Internal Trust Zones:** Gateways usually protect North-South (external) traffic but ignore East-West (internal) traffic; once an attacker breaches the perimeter, they can often move laterally between microservices without further inspection.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The primary failure of perimeter controls is the assumption that threats only originate from unauthenticated external actors. In reality, research indicates that **95% of API attacks come from authenticated users** who possess valid credentials. These actors appear legitimate to Web Application Firewalls (WAFs) and API Gateways because they are not "breaking in" but rather **abusing the permissions** they have already been granted within a valid session.

Context Blindness

WAFs and Gateways typically rely on **signature-based detection**, which is effective for identifying well-known patterns like SQL injection or Cross-Site Scripting (XSS). However, these tools suffer from **context blindness**; they cannot understand the unique business logic of a specific application. Consequently, they fail to detect **Broken Object Level Authorization (BOLA)**, where a user might be accessing data they are technically permitted to view but at an impossible or suspicious volume, such as a scraper harvesting millions of records.

Misconfiguration and Permissiveness

Perimeter controls are frequently **misconfigured** or operated in **"log only" (DetectionOnly)** mode to prevent the accidental blocking of legitimate business traffic. Furthermore, many traditional WAFs do not inspect specific types of

traffic, such as calls to the **AWS Instance Metadata Service (IMDS)**. This creates bypass opportunities for **Server-Side Request Forgery (SSRF)**, where an attacker tricks a trusted server into querying internal endpoints to retrieve sensitive cloud access keys.

Internal Trust Zones

A significant architectural limitation is that Gateways generally govern **North-South traffic** (requests entering or leaving the network) but ignore **East-West traffic** (communication between internal services). This relies on **implicit trust** within the network interior, which is an antipattern. Once an attacker breaches the perimeter, this lack of internal inspection allows them to perform **lateral movement**, pivoting between microservices without being detected by the edge controls.



Detection Gaps Revealed Post-Incident

- **Dwell Time and Visibility Failures**
- **Excessive Dwell Time:** The mean time to identify a breach is often over 200 days, with many organizations only discovering the compromise when notified by a third party (e.g. law enforcement or a fraud processor) rather than through their own monitoring.
- **Logging Without Analysis:** Organizations frequently collect logs but fail to analyze them for security signals; logs often contain evidence of failed logins or anomalies that are ignored until a forensic investigation occurs post-breach.
- **API Sprawl & Shadow IT:** Breaches often occur on "Shadow" or "Zombie" APIs, undocumented endpoints that security teams are unaware of and therefore do not monitor, leaving them as open backdoors for attackers.
- **Lack of Business Logic Alerts:** Traditional monitoring flags system errors (500s) but misses successful (200 OK) malicious activity, such as a scraper sequentially accessing millions of customer records without triggering rate limits.

Dwell time represents the duration an adversary remains undetected within a network, and it remains a critical metric for measuring security maturity. While global median dwell times have decreased in some reports, the **average time to identify a breach often remains over 200 days**. Post-incident analysis frequently reveals that organisations are **incapable of detecting their own breaches**, even when they result in massive data loss. Instead, a staggering **54% to 63% of victims first learn of a compromise through third-party notification**, such as a call from law enforcement (e.g. the FBI) or a fraud detection system at a bank. This reliance on external discovery proves that current reactive measures, such as waiting for signature-based alerts, are fundamentally insufficient.

Logging Without Analysis: Data Graves

A pervasive failure revealed post-breach is **"logging without analysis"**; many organisations collect vast quantities of data but **fail to look at it until a service interruption occurs**. Forensic investigations often uncover that **evidence of an attack**, such as anomalies in network flow or patterns of failed logins, had been recorded for months but were simply ignored. Furthermore, many infrastructure components **do not log successful or failed logins by default**, requiring manual configuration that is often overlooked. This results in a "data grave" where

organisations **consolidate data merely to ignore it more efficiently**, failing to use it for real-time alerting or proactive hunting.

API Sprawl and Shadow IT: The Invisible Backdoor

Breaches often exploit "**Shadow APIs**" or "**Zombie APIs**", endpoints that are undocumented, unmonitored, or deprecated but still operational. **API sprawl** is a massive challenge, with large enterprises managing upwards of **25,000 APIs**, many of which are released "under the radar" without security oversight.

Because these endpoints are **invisible to security teams**, they typically lack the patches, rate limiting, and access controls applied to the main production environment, serving as **open backdoors for attackers**.

Lack of Business Logic Alerts: Malice Masked as Success

Traditional monitoring is often tuned to flag **system errors (500-series)** but fails to detect **malicious activity that returns a "200 OK" success code**. Automated tools and scanners frequently miss these **business logic exploits** because the malicious requests use valid syntax and appear to be legitimate features. For example, **scrapers and bots** can exfiltrate millions of customer records through sequential pagination or ID manipulation without triggering error-based alerts or standard rate limits. Because these transactions **appear normal**, they can persist for months without ever being flagged by traditional security tools.



Design Lessons That Persist

- **Moving Beyond Patching to Secure Architecture**
- **Zero Trust is Mandatory:** The "castle and moat" strategy is obsolete; architecture must assume the network is compromised and require authentication and authorization for every request, even between internal microservices.
- **Secure by Design:** Security must be "shifted left" into the design phase (e.g. defining strict API schemas and access controls) rather than attempting to bolt on security via WAFs after deployment.
- **Least Privilege:** Identities and services should only have the permissions necessary for their function; over-permissioned roles (like the WAF role in the Capital One breach) allow minor exploits to become major data disasters.
- **Fail-Safe Defaults:** Systems should be designed to deny access by default; explicit permission must be granted for any resource access, ensuring that a failure in configuration results in a locked state rather than an open one.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Traditional security often relies on reactive patching, but true resilience requires a **fundamental shift toward secure-by-design architecture**. While patches fix known vulnerabilities, a defensible architecture aims to eliminate entire classes of unknown threats.

Zero Trust Is Mandatory

The outdated "**castle and moat**" or "**crunchy shell**" strategy, which implicitly trusts internal traffic, is now obsolete. Modern architecture must **assume the network is hostile** and that adversaries are already present within the LAN. Consequently, **Zero Trust** requires that **every request**, whether from a remote user or between internal microservices, be **explicitly authenticated and authorised** using multiple data points like identity, device health, and context.

Secure by Design: Shifting Left

Security must be "**shifted left**" into the initial design phase rather than being "bolted on" via Web Application Firewalls (WAFs) after deployment. This involves defining **strict API schemas** and access controls before code is even written, which is significantly more cost-effective than fixing flaws later. Relying solely on a WAF is risky, as they can be bypassed by sophisticated attackers; therefore, security must be **baked into the application's DNA**.

The Principle of Least Privilege

Every identity, service, and process must operate with the **bare minimum permissions** necessary for its function. This limits the "**blast radius**" of a compromise, ensuring that one breached component does not lead to a total system failure. A critical example is the **Capital One breach**, where an **over-privileged WAF role** allowed an attacker to exfiltrate data from millions of customer records.

Fail-Safe Defaults

Systems should be built with **fail-safe defaults**, meaning the default state is to **deny access**. Explicit permission must be granted for any resource interaction, ensuring that if a configuration fails or an error occurs, the system **remains in a locked, secure state** rather than failing open. This "deny by default" philosophy is a cornerstone of robust access control.



Governance, standards, and operational alignment



OWASP Top 10: Web Applications vs. APIs

- **Different Attack Vectors:** While the traditional OWASP Top 10 focuses on broad web risks like Injection and Broken Access Control, the API Security Top 10 targets specific flaws inherent to structured data exchange and object manipulation.
- **The Number One Threat:** Broken Object Level Authorization (BOLA) consistently ranks as the top API threat, enabling attackers to manipulate IDs to access unauthorized data, a risk less prominent in traditional web apps due to server-side session management.
- **Data-Centric Vulnerabilities:** The 2023 API list introduced "Broken Object Property Level Authorization" (BOPLA), combining mass assignment and excessive data exposure, reflecting how APIs expose underlying data models more directly than browser-based apps.
- **Business Logic Risks:** New categories such as "Unrestricted Access to Sensitive Business Flows" highlight how APIs are prone to automated abuse (e.g. scalping) that technically follows protocol but subverts business intent.
- **Infrastructure Attacks:** Unlike the general web list, the API Top 10 explicitly calls out "Server Side Request Forgery" (SSRF) and "Unsafe Consumption of APIs," acknowledging the interconnected nature of modern microservices.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

While the traditional OWASP Top 10 focuses on broad web risks like **Injection** and **Broken Access Control**, the API-specific list targets flaws inherent to **structured data exchange** and **object manipulation**. Unlike browser-based applications, APIs are **headless** and designed for machine-to-machine communication, which shifts the attack surface toward the underlying **data logic** and the way objects are addressed and transferred.

The Number One Threat: BOLA

Broken Object Level Authorization (BOLA) consistently ranks as the primary threat in the API landscape. It occurs when an API provider fails to implement user-based access controls at the resource level, enabling attackers to manipulate **object identifiers** (such as IDs in a URL) to access data belonging to other users. This is a more significant risk for APIs than traditional web apps, where **server-side session management** and stateful browser interactions often mask these direct resource references.

Data-Centricity and BOPLA

The 2023 API list introduced **Broken Object Property Level Authorization (BOPLA)**, a category that merges the previous risks of **Mass Assignment** and **Excessive Data Exposure**. This change reflects a move toward data-centric security, acknowledging that APIs often expose underlying **data models** too

directly. Threat actors exploit BOPLA to either read sensitive properties they shouldn't see or to overwrite restricted fields, such as changing an `is_admin` flag, by injecting additional properties into a request.

Automated Abuse of Business Logic

A significant addition to the API Top 10 is **Unrestricted Access to Sensitive Business Flows**. This category highlights how APIs are uniquely prone to **automated abuse**, such as **scalping** tickets or **data scraping**. Unlike traditional implementation bugs, these attacks are elusive because the malicious requests may be technically valid and follow standard protocols, but they ultimately subvert the **intended business intent** at a volume and speed impossible for human users.

Infrastructure and the Interconnected Web

Finally, the API Top 10 explicitly addresses infrastructure-level risks like **Server-Side Request Forgery (SSRF)** and **Unsafe Consumption of APIs**. These categories acknowledge the **interconnected nature of modern microservices**, where a single compromised or improperly validated internal request can lead to a cascading failure across the entire backend network. By elevating these risks, OWASP encourages developers to move toward a **Zero Trust** model where every interaction, even internal service-to-service calls, is explicitly verified.



NIST and ISO Perspectives

- **NIST Zero Trust (SP 800-207):**
 - Adopts a "never trust, always verify" model where no asset is implicitly trusted based on network location.
 - Requires dynamic, per-request access decisions based on the observable state of the client identity, application, and requesting asset.
- **NIST SSDF (Secure Software Development Framework):** Focuses on "shifting left" by integrating security into the early stages of the software lifecycle, requiring secure coding practices and automated testing within CI/CD pipelines.
- **ISO/IEC 27001 & 27002:**
 - Provides a management framework (ISMS) for identifying and treating risks, with specific controls for access restriction (Annex A.9) that align with least privilege principles.
 - Emphasises network segregation and secure development environments to contain breaches.
- **ISO/IEC 27036:** specifically addresses security in supplier relationships, critical for managing risks when APIs consume third-party cloud services or external data.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

NIST Special Publication 800-207 defines **Zero Trust (ZT)** as a paradigm shift focused on resource protection where **trust is never granted implicitly**, regardless of a user's physical or network location. Traditional architectures relied on a "chewy center" protected by a "crunchy shell," but NIST mandates a "**never trust, always verify**" approach, assuming that the internal network is just as hostile as the public internet.

Crucially, NIST requires that **access to resources be determined by dynamic policy**. This process involves a **Policy Decision Point (PDP)** and a **Policy Engine (PE)** that calculate a trust score based on the observable state of the **client identity**, the **requesting asset's health**, and the **application's sensitivity**. Decisions are made on a **per-request or per-session basis**, ensuring that being authorised once does not grant perpetual access.

NIST SSDF: Shifting Security Left

The **NIST Secure Software Development Framework (SSDF)**, or SP 800-218, provides a structured approach to "**shifting left**" by integrating security into the earliest stages of the **Software Development Life Cycle (SDLC)**. The goal is to identify and remediate vulnerabilities, such as those found in the **OWASP Top 10**, before code enters production.

The SSDF emphasizes **secure coding practices** and the use of **automated**

testing within **CI/CD pipelines**. This includes **Static Application Security Testing (SAST)** to scrutinize source code for flaws like memory leaks, and **Dynamic Application Security Testing (DAST)** to analyze the running application for runtime errors. It also mandates the use of a **Software Bill of Materials (SBOM)** to inventory and manage risks associated with third-party libraries and dependencies.

ISO/IEC 27001 & 27002: The Management Framework

While **ISO/IEC 27001** specifies requirements for an **Information Security Management System (ISMS)**, **ISO/IEC 27002** provides the practical guidelines for implementing security controls. These standards align closely with Zero Trust through **Annex A.9 (Access Control)**, which requires that access to information and facilities be restricted according to business and security requirements. This mandates the **principle of least privilege**, ensuring users only have the minimum permissions necessary for their specific role.

Furthermore, **Annex A.13 (Communications Security)** emphasizes **network segregation** to isolate sensitive systems and prevent **lateral movement** following a breach. To contain potential compromises, ISO recommends creating **secure development environments** (Annex A.14) where security is baked into the implementation rather than added as an afterthought.

ISO/IEC 27036: Securing the Supply Chain

ISO/IEC 27036 addresses the critical risks inherent in **supplier relationships**, particularly as organizations increasingly move data and services to **Cloud Service Providers (CSPs)**. Part 4 of the standard specifically provides guidelines for **cloud service security**, highlighting that while organizations can outsource data management, they can **never outsource their liability**.

This is vital for modern **API-driven architectures** where internal systems frequently consume third-party cloud services or external data feeds. ISO 27036 requires a robust mechanism for **objectively evaluating the security profile of providers** and establishing **cloud service agreements** that define clear security obligations, incident response SLAs, and data destruction policies upon contract termination.



Regulatory Expectations for APIs (1/2)

- **GDPR (Privacy):**
 - Mandates "Data Protection by Design," requiring APIs to implement strict data minimization (not exposing excessive data fields) and user consent management.
 - Enforces the right to erasure and access, meaning APIs must have capabilities to locate and delete specific user data across distributed systems.
- **HIPAA (Healthcare):**
 - Requires strict encryption of Protected Health Information (PHI) both in transit and at rest, along with granular access controls to ensure only authorised personnel interact with specific patient data.
 - Mandates detailed audit trails for all API access to PHI to facilitate forensic analysis and compliance verification.
- **PCI DSS v4.0 (Payments):**
 - Explicitly targets API security, requiring strong authentication (MFA) and protections against business logic abuse.
 - Mandates regular vulnerability scans and penetration testing specifically designed to identify flaws in API logic and implementation.
- **PSD2 (Open Banking):** Requires financial institutions to expose data via secure APIs, mandating Strong Customer Authentication (SCA) to ensure transactional integrity and interoperability.

API Regulatory Landscape In the current digital economy, APIs have moved from purely technical components to central subjects of **stringent regulatory frameworks**. Compliance with these regulations is a baseline requirement to protect sensitive information, build customer trust, and avoid significant financial penalties, which can reach up to **4% of a company's global turnover**.

GDPR: Privacy and Data Protection The **General Data Protection Regulation (GDPR)** mandates "**Data Protection by Design and by Default**" (Article 25), requiring organizations to integrate privacy and security controls into their APIs throughout the entire system development life cycle. APIs must implement **strict data minimization**, collecting and processing only the data absolutely necessary for a specified purpose. Furthermore, APIs must facilitate individual rights, specifically the **right to erasure** (the "right to be forgotten") and the **right to access**, meaning the system must have the functional capability to locate, provide, and delete specific user data across distributed environments.

HIPAA: Healthcare Data Integrity In the healthcare sector, **HIPAA** requires the protection of **electronic Protected Health Information (ePHI)** through administrative, physical, and technical safeguards. This necessitates **strict**

encryption of PHI both **in transit and at rest** to maintain confidentiality and integrity. APIs must enforce **granular access controls**, typically through role-based access control (RBAC), ensuring that only authorized personnel have access to specific patient records. Additionally, detailed **audit trails** must be maintained for all API access to PHI, documenting who accessed the data and when to facilitate forensic investigations and compliance audits.

PCI DSS v4.0 and PSD2: Financial Security **PCI DSS v4.0** has introduced specific requirements that **explicitly target API security**, mandating **strong authentication (MFA)** for privileged access and protection against **business logic abuse**. It also requires **regular vulnerability scans and penetration testing** tailored to the API's unique logic and implementation to identify flaws that traditional tools might miss. Similarly, the **Revised Payment Services Directive (PSD2)** in the EU mandates that financial institutions provide secure APIs for data sharing. This includes **Strong Customer Authentication (SCA)** to ensure the integrity of transactions and secure interoperability between diverse financial platforms.



Regulatory Expectations for APIs (2/2)

- **PDPA (Singapore – Personal Data Protection Act):**
 - Requires organizations to implement reasonable security arrangements to protect personal data processed via APIs, including authentication, authorization, and secure transmission controls.
 - Imposes accountability on API owners to prevent unauthorized access, use, or disclosure, making excessive data exposure, weak access tokens, or undocumented APIs a compliance risk.
 - Supports data access and correction obligations, implying APIs must support traceability and controlled data retrieval workflows.
- **MAS TRM (Singapore – Financial Sector):**
 - Explicitly requires secure API design for systems handling financial data, including strong authentication, authorization, encryption, and protection against abuse and denial-of-service.
 - Emphasizes defense against logical abuse, meaning APIs must be resilient not only to technical exploits, but also to misuse of valid business logic.
 - Requires comprehensive logging and monitoring of API activity to support incident detection, investigation, and regulatory reporting.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

PDPA (Singapore – Personal Data Protection Act)

The PDPA mandates that organisations take proactive steps to safeguard personal data, which is defined as any information that can identify an individual. Under the Act, APIs are considered a primary conduit for **processing PII**, which includes data collection, storage, and retrieval.

Reasonable Security Arrangements: Organisations must implement robust **authentication and authorization** mechanisms to verify identities and define access rules. Secure transmission via protocols like **HTTPS (TLS)** is essential to ensure data confidentiality and integrity during transit.

Accountability and Risk Mitigation: API owners are held accountable for preventing unauthorized disclosure. This makes **excessive data exposure** (returning more PII than necessary) and **weak access tokens** (which lack sufficient entropy or validation) significant compliance risks. Furthermore, **undocumented (shadow) APIs** represent a major liability, as they typically bypass standard security oversight and monitoring.

Data Rights and Traceability: The PDPA supports rights such as **data access and correction**. To fulfil these, APIs must be designed with traceability, using **audit logs** to record who accessed what data and when, thereby supporting the "right to request your data" and ensuring accountability.

MAS TRM (Singapore – Financial Sector)

For the financial sector, the MAS TRM guidelines set a high bar for system resilience and data protection.

Secure by Design: Financial systems must adopt a "**secure-by-design**" approach, integrating security into every stage of the development lifecycle. This includes enforcing **strong authentication** (e.g. OAuth 2.0 or MFA) and protecting against **Denial-of-Service (DoS)** attacks that could impact the availability of critical financial services.

Defense Against Logical Abuse: APIs in finance are particularly susceptible to **business logic vulnerabilities**, where threat actors abuse intended features (like referral rewards or negative quantity inputs) rather than technical bugs. MAS TRM expects APIs to be resilient to these "zero-day" logic flaws through thorough **threat modeling** and the validation of all user-supplied data.

Comprehensive Logging and Monitoring: Logging is described as the "cornerstone of a sound security posture". Organisations must maintain detailed logs of **CRUD operations** on sensitive information to facilitate incident detection and forensic analysis. These logs should be fed into a **centralised system (SIEM)** for real-time analysis and regulatory reporting.



Translating Frameworks into Engineering Behaviour

- **Guardrails over Gatekeepers:** Security teams should shift from being blockers to advisors by setting "guardrails", pre-approved, secure-by-default infrastructure and patterns that allow developers to deploy safely without manual approval for every change.
- **Security as Code:** Integrate compliance checks (like SAST, DAST, and secret scanning) directly into the CI/CD pipeline, automating the enforcement of standards like NIST or OWASP before code reaches production.
- **API Governance via Gateways:** Centralise policy enforcement (rate limiting, authentication validation, logging) at the API Gateway to ensure consistent behaviour across all microservices without relying on individual service developers to implement controls correctly.
- **Security Champions:** Embed security-conscious developers within engineering teams to translate abstract compliance requirements into practical coding standards and to act as a bridge between security leadership and development squads.
- **Design-First Approach:** Use API specifications (like OpenAPI) as a contract to define data schemas and security requirements upfront, allowing automated tools (linters) to validate compliance before code is even written.

The traditional security model often relies on "gatekeepers" who perform manual reviews at the end of the development cycle, frequently acting as a bottleneck that delays releases. Shifting to **guardrails** transforms security teams into advisors who provide **pre-approved, secure-by-default infrastructure patterns** and **landing zones**. This "paved road" approach empowers developers to deploy services autonomously within a safe environment, for example, permitting the creation of virtual machines while automatically preventing the attachment of a public IP address. This shift is essential because security that acts as a blocker is likely to be ignored or circumvented by teams pressured to meet deadlines.

Security as Code: Automating the Pipeline

Security as Code (SaC) integrates compliance requirements directly into the **CI/CD pipeline**, treating security tool configurations and policies as software artifacts that can be reviewed and version-controlled. By automating **SAST (Static Analysis)**, **DAST (Dynamic Analysis)**, and **secret scanning**, organisations can enforce standards such as the **NIST Secure Software Development Framework (SSDF)** or **OWASP API Top 10** with every code commit. This provides immediate feedback to engineers, ensuring that vulnerabilities are remediated early in the **SDLC**, which is significantly more cost-effective than fixing them in production.

API Governance via Gateways: Centralised Enforcement

Utilising **API Gateways** as a central "choke point" is critical for maintaining **uniform policy enforcement** across a microservices architecture. Instead of relying on individual developers to implement controls correctly in every service, the gateway handles **rate limiting, authentication validation, and logging** transparently. This prevents the emergence of "spaghetti code" and ensures that backend APIs are protected from excessive resource consumption and common web-based attacks before a request even reaches the microservice subnet.

Security Champions: Bridging the Gap

Because centralised security teams are often outnumbered by development squads, **Security Champions** are embedded within engineering teams to act as a local bridge. These are security-conscious developers who translate abstract compliance mandates into **practical coding tasks** and lead activities such as **threat modelling** or peer code reviews. This distributed model helps scale security expertise throughout the organisation and fosters a collaborative culture rather than an antagonistic one.

Design-First Approach: Contracts over Code

A **Design-First approach** uses API specifications, such as **OpenAPI (OAS)**, as a single source of truth and a formal contract between producers and consumers. This allows security to be considered before any implementation code is written, enabling **automated linters like Spectral** to validate the specification for security flaws, such as missing authentication schemes or unconstrained input parameters. By leveraging a **positive security model**, organisations can ensure that only data and operations explicitly defined in the contract are permitted, radically reducing the risk of mass assignment and excessive data exposure.